
PROJECT GUIDE

Nexusphere

Verifiable evidence of what AI agents do, for whom and with whose permission, across organizations.

Version 1.0.0 · October 2026

Contents

Part 1 Introduction	4
Nexusphere at a Glance	5
Project Description	9
Part 2 Getting Started	13
Ledger Guide	14
Local Environment Setup	19
Part 3 Nexusphere Ledger	21
Nexusphere Ledger	22
Nexusphere Ledger Chain	39
Nexusphere Ledger Mandate	40
Nexusphere Ledger Server	41
Nexusphere Ledger Verifier	43
Nexusphere Ledger Demo MCP	44
Nexusphere Ledger Conformance Vectors	45
Nexusphere Ledger Changelog	47
Part 4 SDKs and Integrations	49
Nexusphere Ledger Python SDK	50
Nexusphere Ledger TypeScript SDK	54
Nexusphere Ledger OpenTelemetry Exporter	58
Nexusphere Ledger Adapters	62
Nexusphere Ledger for LangGraph	64
Nexusphere Ledger for the OpenAI Agents SDK	67
Nexusphere Ledger for Amazon Bedrock AgentCore	70
Nexusphere Ledger for agentgateway	73
Nexusphere Package Verifier for the Browser	75
Part 5 Web UI	77
Nexusphere Frontend	78
Part 6 Operations	80
Operations	81
DevOps Step by Step	87
Part 7 Testing	90
Nexusphere Ledger End-to-End Tests	91

Part 8 Build and Delivery Toolkit	95
mvn-devops	96
Installation	100
Installing the prerequisites	102
What your Maven project needs	104
GitHub tokens and SSH keys	106
Getting started, step by step	108
Ready-made pipelines	113
Modules and stages	115
Orchestrators	117
Where the tools run	119
Container images and deployment	121
Project site on GitHub Pages	130
Releasing your project	131
Reaching tools on your machine from the internet (ngrok)	132
IDE settings (IntelliJ IDEA)	133
Troubleshooting	134
Design	136
Writing a module	139
Development	142
Changelog	143

PART 1

Introduction

What Nexusphere is, the problem it solves and how it is organized.

Nexusphere at a Glance

Verifiable evidence of what AI agents do, for whom and with whose permission, across organizations.

Documentation

Read the documents in this order. The first three take a newcomer from nothing to a verified evidence package; the rest are references to open when they are needed. All of them are also in one book, [Nexusphere Guide \(PDF\)](#).

Step	Document	What it gives
1	This page	What the ledger is and the quick start
2	Project Description	The problem, the concepts, the architecture and the standards
3	Ledger Guide	Install, register agents, grant, call tools and verify step by step
4	Nexusphere Ledger	Every setting, format and endpoint, and one README per module
5	Nexusphere Frontend	The web UI for operators and principals
6	Operations	Secrets, signing keys, backup, monitoring, alerts and rate limits
7	Local Environment Setup	Developer machine settings for Windows, shells and the IDE
8	DevOps Step by Step	The build pipelines with mvn-devops
9	Ledger End-to-End Tests	What the end-to-end tests cover
10	Conformance Vectors	Test data for another implementation of the formats
11	Python SDK	Record evidence from Python and verify packages and mandates
12	TypeScript SDK	The same for Node and browsers
13	OpenTelemetry Exporter	Evidence from the agent spans an OpenTelemetry Collector receives
14	Adapters	LangGraph, OpenAI Agents SDK, AgentCore and agentgateway
15	Browser Verifier	Verify a package in a browser, from one file, with no network
16	Changelog	What each release contains

Nexusphere Ledger

The Nexusphere Ledger is a self-hosted evidence layer for AI agents: every action an agent takes for a person or an organization becomes an entry that a third party can verify offline, without trusting the operator. It does not replace the agent platforms and gateways that already decide what an agent may do; it sits beside them and keeps the proof. A gateway or agent platform sends its decisions and outcomes to `POST /api/v1/evidence`, and where no gateway exists the ledger's own MCP and A2A gateways decide and record.

Need	Already provided by	What the ledger adds
Agent identity and tool-call decisions	Amazon Bedrock AgentCore, Google Agent Gateway, Microsoft Entra Agent ID, agentgateway	Records their decisions as evidence, or decides itself through its MCP and A2A gateways
Evidence a third party can verify	Logs and audit trails the operator controls	Hash chain, Merkle transparency log with witness cosignatures, SCITT statements and receipts, offline verifier
Proof of authority across organizations	AP2 and Verifiable Intent, for payments only	Grants with principal consent, SD-JWT VC mandates and signed A2A receipts on both sides
Disclosure to an auditor or a counterparty	Exports of raw logs	Evidence packages that disclose only the selected entries and prove the rest of the log

The formats follow open standards so other tools can check them: RFC 9162 Merkle trees, C2SP signed notes and the `tlog-witness` protocol, IETF SCITT signed statements with RFC 9942 receipts, SD-JWT VCs and the IETF Token Status List. [Conformance Vectors](#) let another implementation test itself against the ledger.

Quick Start

These steps start the ledger with its database, Keycloak, the web UI and a demo MCP server, run the demo and stop everything again. Every command runs in the root of the repository.

Step 1. Check the tools; each command must print a version:

Tool	Version	Check
Java	21	<code>java -version</code>
Maven	3.9	<code>mvn -version</code>
Docker	any	<code>docker compose version</code>
jq	any	<code>jq --version</code>

Windows, shell and IDE settings are in [Local Environment Setup](#).

Step 2. Build the server, the verifier and the demo MCP server:

```
mvn -pl ledger/server,ledger/verifier,ledger/demo-mcp -am package -DskipTests=true
```

Step 3. Start the services:

```
docker compose --file ledger/compose.yaml --project-name ledger up -d --build
```

Step 4. Check that the ledger is up; the answer must be `{"status":"UP"}`:

```
curl -X GET http://localhost:8090/actuator/health
```

Step 5. Run the demo; it must end with `Result: VALID` for the package and `Result: INVALID` for the changed copy:

```
ledger/demo-mcp/demo.sh
```

Step 6. Open the web UI on <http://localhost:5173>, sign in with the operator key `nexusphere-ledger-development-key-change-me`, and look at the evidence the demo recorded. Sign out and sign in again as `alice` with the password `alice` to see her grants.

Step 7. Stop everything and remove the data:

```
docker compose --file ledger/compose.yaml --project-name ledger down
docker volume prune -f
```

Next, the [Ledger Guide](#) registers your own agents, grants them tools and connects real agents such as Claude Code.

Build and Test

Step 1. Compile every module:

```
mvn validate clean compile
```

Step 2. Run the unit tests:

```
mvn test
```

Step 3. Run every test, including the integration and end-to-end tests; Docker must be running for Testcontainers:

```
mvn verify
docker volume prune -f
```

Step 4. Build the jars without tests:

```
mvn package -DskipTests=true
```

The web UI is built and tested with npm, as described in [Nexusphere Frontend](#). The SDKs, the exporter, the adapters and the browser verifier use their own tools, as described in [Python SDK](#), [TypeScript SDK](#), [OpenTelemetry Exporter](#), [Adapters](#) and [Browser Verifier](#).

Kubernetes

`ledger/kube-dev.yaml` runs the ledger, PostgreSQL, Adminer, Keycloak, the web UI and the demo MCP server in the `dev` namespace. The steps are in [Nexusphere Ledger](#).

DevOps

The project ships `mvn-devops` 1.0.0 in the `mvn-devops` folder. It starts the chosen tools in Docker, configures them and runs the pipeline stages against this Maven project, either from the menu or from one of its 16 ready-made pipelines. Step-by-step use for this project, from `maven-sonarqube-`

nexus up to jenkins-complete , is in [DevOps Step by Step](#); its own guide is [mvn-devops/README.md](#). Values and tokens are kept in `.devops/` , which is never committed.

```
mvn-devops/devops.sh doctor
mvn-devops/devops.sh pipelines
mvn-devops/devops.sh setup --pipeline maven-sonarqube-nexus
mvn-devops/devops.sh stages
mvn-devops/devops.sh run
mvn-devops/devops.sh urls
```

Windows:

```
mvn-devops\devops.bat setup
```

Upgrade with `mvn-devops/devops.sh upgrade` , which replaces the folder with the latest release after checking its `SHA256SUMS` , or by deleting the `mvn-devops` folder and extracting the new release zip in its place under the same name.

Service URLs

Service	URL	Sign in
Web UI	http://localhost:5173	Operator key, or <code>alice / alice</code> and <code>bob / bob</code>
Ledger API	http://localhost:8090	<code>Authorization: Bearer {key}</code>
Health	http://localhost:8090/actuator/health	None
Demo MCP	http://localhost:8091/mcp	None, reached through the ledger at <code>/mcp/demo</code>
Keycloak	http://localhost:8180	<code>admin / admin</code>
Adminer	http://localhost:8092	Server <code>ledger-postgresql</code> , user and password <code>ledger</code>
Prometheus	http://localhost:9090	None
Alertmanager	http://localhost:9093	None
OTLP	<code>localhost:4317 (gRPC)</code> , <code>http://localhost:4318 (HTTP)</code>	None, the Collector sends with the operator key

Repository Layout

Folder	Content
<code>ledger</code>	The ledger: chain, mandate, server, verifier, demo MCP, tests, conformance, SDKs, adapters
<code>frontend</code>	The web UI in React and TypeScript
<code>docs</code>	The documents above and the PDF guide
<code>mvn-devops</code>	The build and delivery toolkit

Project Description

The problem the Nexusphere Ledger solves, its concepts, guarantees, architecture and standards.

Problem

AI agents now read files, send email, place orders and talk to agents of other companies on behalf of people. When something goes wrong, three questions come up: what did the agent do, for whom, and who allowed it. Platform logs answer them only for whoever runs the platform, and that party can change its own logs. An auditor, a regulator, the person the agent acted for or a company on the other side of an exchange has to take the operator's word.

The Nexusphere Ledger turns every agent action into evidence that anyone can check without trusting the operator. It is self-hosted and sits beside the agent platforms and gateways an organization already uses, such as Amazon Bedrock AgentCore, Google Agent Gateway, Microsoft Entra Agent ID or agentgateway; it keeps the proof of what they decided. Where nothing decides yet, its own MCP and A2A gateways decide and record.

Concepts

Concept	Meaning
Agent	An identity with its own API key; the program that uses the key is the agent
Principal	The person or organization the agent acts for, such as <code>alice</code>
Operator	Whoever runs the ledger and holds its operator key
Grant	What a principal lets an agent do: actions, targets, validity and a use limit
Consent	The principal's approval of a grant with their own sign-in at an OpenID Connect provider
Decision	ALLOW or DENY for one action, with a reason code, followed by the reported outcome
Evidence entry	One record in the hash chain: who, what, for whom, the decision and the outcome
Checkpoint	The chain head signed with the ledger key, so later changes are detectable
Transparency log	A Merkle tree over all entries whose signed checkpoints prove that an entry is in the log
Witness	An independent party that cosigns log checkpoints and refuses a log that rewrites its history
Signed statement	One entry as an IETF SCITT COSE statement, with a receipt that proves it is in the log
Package	Selected entries with the proof that they belong to the signed chain and log, for an auditor or counterparty
Mandate	A signed SD-JWT VC an agent carries to another organization to prove what it may do
Receipt	The signed answer of a receiving ledger in an A2A exchange, so both sides hold matching evidence

Guarantees

ID	Guarantee
G-1	Evidence is append-only: the database refuses any change or deletion, and the hash chain shows any change
G-2	Every checkpoint is signed; a retired key stays published, so everything it signed still verifies
G-3	A package verifies offline with the ledger's public key, without the server
G-4	Witnesses refuse a smaller or forked log, so a ledger cannot hide or rewrite entries they have seen
G-5	A decision is allowed only by an active grant that covers the action and target; the default is DENY
G-6	A grant created by the operator or an agent allows nothing until the principal approves it, when OIDC is on
G-7	Grant, consent, mandate and key changes are evidence too
G-8	A receiving ledger accepts an A2A request only with a valid mandate, and both sides record the exchange
G-9	The ledger refuses to start with the published development secrets outside the <code>dev</code> profile

Actors

Actor	Does
Operator	Runs the ledger, registers agents, creates grants, exports packages
Agent	Asks for decisions, reports outcomes, or calls tools and agents through the gateways
Principal	Signs in, approves, denies, creates and revokes grants, and reads the evidence about them
Gateway or platform	Sends the decisions it made and their outcomes to the ledger as evidence
Auditor	Verifies packages, statements and receipts offline
Counterparty ledger	Checks mandates of incoming A2A requests and signs receipts
Witness	Cosigns log checkpoints of the logs it watches

How It Works

1. The operator registers an agent; the agent gets its own API key.
2. A grant names the principal, the agent, the actions, the targets, the validity and the use limit.
3. With principal sign-in on, the principal approves the grant in the web UI.
4. The agent calls a tool through the MCP gateway, or an external gateway decides and sends the decision as evidence.
5. The ledger decides against the grant, forwards only allowed calls, and records the decision and the outcome.
6. Each entry is linked to the previous one by its hash and added to the Merkle log.
7. At a fixed interval the ledger signs the chain head and the log checkpoint, and witnesses cosign the checkpoint.
8. An auditor receives a package of selected entries and verifies it offline with the ledger's public key.
9. For another organization, the agent carries a mandate; the receiving ledger checks it and answers with a receipt.

Architecture

The ledger is a Maven multi-module project in `ledger/`. The formats live in plain Java libraries without framework dependencies, so the verifier and other organizations use exactly the code the server uses. The server is one Spring Boot application on PostgreSQL. The web UI in `frontend/` is a separate single-page application that talks to the server's API.

Module	Responsibility
<code>ledger/chain</code>	Evidence format, hash chain, Merkle log, signed notes, COSE statements and receipts
<code>ledger/mandate</code>	SD-JWT VC mandates, status lists, JWK keys and the mandate verifier SDK
<code>ledger/server</code>	Evidence API, gateways, grants, consent, checkpoints, witnesses and verification
<code>ledger/verifier</code>	Offline verifier for packages, statements and mandates, as a library and a jar
<code>ledger/demo-mcp</code>	Demo MCP server and the demo script
<code>ledger/ledger-e2e-tests</code>	End-to-end tests against the server and PostgreSQL
<code>ledger/conformance</code>	Conformance vectors for every format
<code>ledger/sdk/python</code>	Python client and offline verifier, on the same conformance vectors
<code>ledger/sdk/typescript</code>	TypeScript client and offline verifier for Node and browsers
<code>ledger/otel-exporter</code>	OpenTelemetry Collector exporter from agent spans to evidence
<code>ledger/adapters</code>	Adapters for LangGraph, OpenAI Agents SDK, AgentCore and agentgateway
<code>ledger/verifier-web</code>	Package verifier in the browser, as one HTML file with no network access
<code>frontend</code>	Web UI for operators and principals

Stack

- Java 21, Spring Boot 4.1, Maven
- PostgreSQL 18 with the `ledger` schema and Flyway migrations
- Spring JDBC, Spring Security OAuth2 JOSE for principal tokens, Micrometer with Prometheus
- React and TypeScript for the web UI and the browser verifier
- Python and TypeScript SDKs that implement every format independently
- Go for the OpenTelemetry Collector exporter
- JUnit 5, AssertJ and Testcontainers for tests; Vitest for the web UI

Server Packages

```
com.nexusphere.ledger
├── evidence      evidence entries, the hash chain, checkpoints and packages
├── transparency  Merkle log, log checkpoints, proofs, witnesses, SCITT statements and receipts
├── agent         agents and their API keys
├── authorization grants, consent, decisions and mandates
├── mcp          MCP gateway
├── a2a          A2A gateway, inbound verification and receipts
└── server       configuration, security, signing keys, rate limits and errors
```

Standards

Standard	Used for
RFC 9162	Merkle tree, inclusion and consistency proofs
C2SP signed-note, tlog-checkpoint	Log checkpoints
C2SP tlog-witness, tlog-cosignature	Witness cosigning
IETF SCITT, RFC 9052 COSE	Signed statements
RFC 9942	Receipts for signed statements
IETF SD-JWT VC	Mandates with selective disclosure
IETF Token Status List	Mandate revocation
RFC 8037, RFC 8032	Ed25519 keys and signatures
OpenID Connect, OAuth 2.0 PKCE	Principal sign-in
MCP, A2A	Gateways for tool calls and agent-to-agent calls
OpenTelemetry GenAI conventions	Evidence from agent spans
AP2, Verifiable Intent	Mapping of mandates to payment intents

Out of Scope

- Running agents or hosting models; the ledger only records and decides their actions
- Replacing agent platforms and gateways; the ledger sits beside them
- A policy language; grants are explicit lists of actions and targets
- Payments; AP2 and other payment protocols carry the money, the ledger keeps the proof
- Certification for a regulation; the ledger provides evidence that audits can use

PART 2

Getting Started

Trying the ledger with agents, and setting up a machine for development.

Ledger Guide

Install the Nexusphere Ledger, run agents through it, and present it.

What It Does

An agent is software that acts for a person. The ledger stands between the agent and the tools it uses. Every tool call is checked against a grant the person gave, allowed or denied, and written to a hash chain that nobody can change unnoticed, not even the operator. Anyone can later check what an agent did, for whom and with whose permission, with a package of evidence and a verifier that runs offline.

Word	Meaning
Agent	An identity with its own API key; the program that uses the key is the agent
Principal	The person or organization the agent acts for, such as <code>alice</code>
Grant	What a principal lets an agent do: actions, targets, expiry and a use limit
Decision	ALLOW or DENY for one call, with a reason code
Evidence	One entry in the hash chain: who, what, for whom, the decision and the outcome
Checkpoint	The chain head signed with the ledger key, so later changes are detectable
Package	The evidence of one agent or principal with the proof that it belongs to the signed chain
Mandate	A signed token an agent carries to another organization to prove what it may do

Install

Prerequisites

Tool	Version	Check
Java	21	<code>java -version</code>
Maven	3.9	<code>mvn -version</code>
Docker	any	<code>docker compose version</code>
jq	any	<code>jq --version</code>

Start

Every command runs in the root of the repository.

Step 1. Build the server, the verifier and the demo MCP server:

```
mvn -pl ledger/server,ledger/verifier,ledger/demo-mcp -am package -DskipTests=true
```

Step 2. Start the services:

```
docker compose --file ledger/compose.yaml --project-name ledger up -d --build
```

Step 3. Check the ledger; the answer must be `{"status": "UP"}` :

```
curl http://localhost:8090/actuator/health
```

Service	URL	Sign in
Web UI	http://localhost:5173	Operator key, or <code>alice / alice</code> and <code>bob / bob</code>
Ledger API	http://localhost:8090	<code>Authorization: Bearer {key}</code>
Demo MCP	http://localhost:8091/mcp	None, reached through the ledger at <code>/mcp/demo</code>
Keycloak	http://localhost:8180	<code>admin / admin</code>
Adminer	http://localhost:8092	Server <code>ledger-postgresql</code> , user and password <code>ledger</code>
Prometheus	http://localhost:9090	None
Alertmanager	http://localhost:9093	None

The development operator key is `nexusphere-ledger-development-key-change-me` . It only works with the `dev` profile; production keys and secrets are described in [Operations](#).

Run the Demo

Step 4. Run the demo; it must end with `Result: VALID` for the package and `Result: INVALID` for the changed copy:

```
ledger/demo-mcp/demo.sh
```

The script registers an agent, asks `alice` for a grant and approves it with her Keycloak sign-in, calls two allowed tools and one denied tool through the gateway, lists the evidence, exports a package, verifies it offline and shows that a changed entry fails verification.

Agents

These steps use the stack started above. Keep one terminal open for them, since step 1 sets a variable the later steps use.

Register Agents

Step 1. Keep the operator key in a variable and register two agents:

```
OP="Authorization: Bearer nexusphere-ledger-development-key-change-me"
curl -s -X POST http://localhost:8090/api/v1/agents -H "$OP" -H "Content-Type: application/json" -d '{"agentId": "invoice-agent", "name": "Invoice agent", "ownerId": "acme"}'
curl -s -X POST http://localhost:8090/api/v1/agents -H "$OP" -H "Content-Type: application/json" -d '{"agentId": "support-agent", "name": "Support agent", "ownerId": "acme"}'
```

Each answer carries the agent's `apiKey` once. Keep it; a lost key is replaced with `POST /api/v1/agents/{agentId}/key` . The Agents page of the web UI does the same.

Grant Permissions

Step 2. Create a grant for each agent:

```
curl -s -X POST http://localhost:8090/api/v1/grants -H "$OP" -H "Content-Type: application/json" -d '{"principalId":"alice","agentId":"invoice-agent","actions":["tools/call"],"targets":["demo/read_file","demo/send_email"],"expiresAt":"2026-12-31T00:00:00Z","maxUses":20}'
curl -s -X POST http://localhost:8090/api/v1/grants -H "$OP" -H "Content-Type: application/json" -d '{"principalId":"alice","agentId":"support-agent","actions":["tools/call"],"targets":["demo/read_file"],"expiresAt":"2026-12-31T00:00:00Z"}'
```

The compose ledger asks principals for consent, so both grants start `PENDING`. Sign in to the web UI as `alice` and approve them; both must then show `ACTIVE`. Targets are `{server}/{tool}`; a trailing `*` matches a prefix, such as `demo/*`.

Call Tools

Step 3. Call one granted and one not granted tool with the `apiKey` of `invoice-agent`:

```
curl -s http://localhost:8090/mcp/demo -H "Authorization: Bearer {apiKey}" -H "X-Ledger-Principal: alice" -H "Content-Type: application/json" -d '{"jsonrpc":"2.0","id":1,"method":"tools/call","params":{"name":"read_file","arguments":{"path":"/invoices/2026-10.txt"}}}'
curl -s http://localhost:8090/mcp/demo -H "Authorization: Bearer {apiKey}" -H "X-Ledger-Principal: alice" -H "Content-Type: application/json" -d '{"jsonrpc":"2.0","id":2,"method":"tools/call","params":{"name":"delete_file","arguments":{"path":"/invoices/2026-10.txt"}}}'
```

The first call reaches the demo server and returns its answer. The second is denied with JSON-RPC error `-32003` and never reaches the server. `X-Ledger-Principal` names the principal the agent acts for; the call is allowed only when that principal gave the agent a matching grant.

See the Evidence

Step 4. List the evidence of `invoice-agent`; the denied call is there too:

```
curl -s "http://localhost:8090/api/v1/evidence?agentId=invoice-agent" -H "$OP" | jq '.items[] | {sequence, action, target, decision, outcome}'
```

Step 5. Export a package and verify it offline; the verifier must print `Result: VALID`:

```
curl -s -X POST http://localhost:8090/api/v1/packages -H "$OP" -H "Content-Type: application/json" -d '{"agentId":"invoice-agent"}' -o package.json
java -jar ledger/verifier/target/verifier-1.0.0-SNAPSHOT-exec.jar --public-key "$(curl -s http://localhost:8090/api/v1/keys -H "$OP" | jq -r '[] | select(.status == "ACTIVE") | .publicKey')" package.json
```

Real Agents and MCP Servers

Claude Code

```
claude mcp add --transport http ledger-demo http://localhost:8090/mcp/demo --header "Authorization: Bearer {apiKey}" --header "X-Ledger-Principal: alice"
```

Claude Code then lists the demo tools and every call it makes goes through the ledger: granted tools run, others are denied, and both are evidence. Any MCP client that speaks Streamable HTTP and can send headers works the same way, such as Claude Desktop or an agent framework.

Your Own MCP Server

Each upstream MCP server is configured on the ledger by name and served at `/mcp/{name}`. The server must speak Streamable HTTP; a stdio server needs an HTTP bridge in front of it. A key the upstream needs is sent by the ledger, so the agent never sees it.

```
LEDGER_MCP_SERVERS_FILES_URL: http://files-mcp:3000/mcp
LEDGER_MCP_SERVERS_FILES_AUTHORIZATION: Bearer upstream-secret
```

Another Organization

When an agent works with an agent of another organization, both sides run a ledger. The sending ledger decides the call and attaches a mandate, the receiving ledger checks it and answers with a signed receipt, and both keep matching evidence. The A2A gateway and its settings are in [Nexusphere Ledger](#).

Beside an Existing Gateway

When agents already run behind a platform or gateway that decides their tool calls, such as Amazon Bedrock AgentCore, Google Agent Gateway or agentgateway, keep it and let the ledger hold the proof. Register one ledger agent per real agent, then have the gateway, a hook of the platform or the agent itself send every decision and its outcome as evidence with that agent's key. The entries get the same chain, log, witnesses, statements and packages as calls that pass through the ledger's own gateway.

```
curl -X POST http://localhost:8090/api/v1/evidence -H "Authorization: Bearer {agentApiKey}" -H
"Content-Type: application/json" -d '{"agentId":"invoice-
agent","principalId":"alice","action":"tools/call","target":"send_email","decision":"ALLOW","reason":"agentcore-policy","outcome":"SUCCEEDED","correlationId":"session-42","attributes":
{"gateway":"agentcore"}}'
```

When the platform already sends OpenTelemetry traces, nothing has to call the API: point its OTLP exporter at the Collector of the [OpenTelemetry Exporter](#), or add the `nexusphere` exporter to an existing Collector. Every `execute_tool`, `invoke_agent` and MCP `tools/call` span becomes an evidence entry, with the trace id as its correlation id, and other spans are ignored.

For LangGraph, the OpenAI Agents SDK, Amazon Bedrock AgentCore Gateway and agentgateway, the [Adapters](#) connect the ledger in a few lines and can also let it decide each call.

Present It

Step	Show	Point
1	The problem in one sentence	Agents act, but nobody can prove what they did
2	<code>demo.sh</code> : two tools allowed, <code>delete_file</code> denied	The decision happens before the tool runs
3	The web UI as <code>alice</code> : approve, then revoke a grant	The person, not the vendor, decides
4	A package verified in the browser verifier, then one changed entry fails	Nobody can change the record, not even the operator
5	Claude Code calling tools through the ledger	Works with real agents without changing them

Step	Show	Point
6	Two ledgers and a mandate, if the audience works across organizations	Trust between companies, not only inside one

Start the stack before the talk and run `demo.sh` once, so images and Keycloak are warm. Keep the web UI open in a second window signed in as `alice`.

Stop

Step 1. Stop the services and remove the data:

```
docker compose --file ledger/compose.yaml --project-name ledger down
docker volume prune -f
```

Next Steps

- Production secrets, backups and monitoring: [Operations](#)
- Every setting and endpoint: [Nexusphere Ledger](#)
- The web UI: [Nexusphere Frontend](#)

Local Environment Setup

One-time settings on a developer machine for building Nexusphere and running mvn-devops.

Tools

Tool	Version	Check
Java (JDK)	21	<code>java -version</code>
Maven	3.9	<code>mvn -version</code>
Docker	any	<code>docker compose version</code>
Git	any	<code>git --version</code>
curl	any	<code>curl --version</code>
jq	any	<code>jq --version</code>

Install steps per operating system are in [mvn-devops/docs/prerequisites.md](#).

Git Line Endings

The Bash scripts must keep LF line endings. The repository enforces this with `.gitattributes`, but a working tree cloned with `core.autocrlf=true` before that may still hold CRLF files. On Windows, prefer:

```
git config --global core.autocrlf input
```

If `devops.sh doctor` reports files with CRLF line endings, write them again from Git at the project root:

```
rm -rf mvn-devops
git checkout HEAD -- mvn-devops
```

Windows Shell

mvn-devops runs in Bash. On Windows use Git Bash or Cygwin, not WSL and not PowerShell.

Git Bash

Works without extra settings. `mvn-devops\devops.bat` also starts the scripts with Git Bash.

Cygwin

Cygwin must use its own `bash`. When the Windows folders come first in `PATH`, `bash` resolves to `C:\Windows\System32\bash.exe`, which is WSL, and the scripts run inside WSL: `java.exe` and `jq.exe` are not found there, and CRLF files fail with `set: pipefail: invalid option name`.

Check it:

```
command -v bash
```

The answer must be `/usr/bin/bash`. If it is `/cygdrive/c/Windows/system32/bash`, put the Cygwin folders first in `~/.bashrc`. `~` is the Cygwin `$HOME`, which can be the Windows home such as `C:\Users\<user>`; `echo "$HOME"` shows it.

```
echo 'export PATH="/usr/local/bin:/usr/bin:$PATH"' >> ~/.bashrc
```

Open a new terminal and run `command -v bash` again.

The first build of mvn-devops 1.0.0 passed Cygwin paths such as `/cygdrive/c/...` to `docker.exe`, which reads them as `C:\cygdrive\c\...`, so `setup` failed with `couldn't find env file`. The release rebuilt on 2026-10-08 converts these paths with `cygpath`. If `setup` still fails with that message, run the Docker commands from Git Bash.

IntelliJ IDEA Terminal

Settings > Tools > Terminal > Shell path:

Shell	Shell path	Environment variables
Git Bash	"<git>\bin\bash.exe" --login -i	
Cygwin	<cygwin>\bin\bash.exe	
Cygwin	<cygwin>\bin\bash.exe --login -i	CHERE_INVOKING=1

`<git>` is the Git for Windows folder, for example `C:\Program Files\Git`, and `<cygwin>` is the Cygwin folder, for example `C:\cygwin64`. A Cygwin login shell moves to the home folder unless `CHERE_INVOKING=1` is set, so the terminal would not start in the project folder. Without `--login`, Cygwin needs the `PATH` line from [Cygwin](#).

Pipeline variables for IntelliJ IDEA run configurations, Checkstyle and coverage are described in [mvn-devops/docs/ide.md](#).

GitHub Access

Tokens and the SSH key used by the pipeline are described in [mvn-devops/docs/github-setup.md](#). With the `maven` orchestrator a classic token with `repo`, `write:packages` and `read:packages` is enough; the SSH key is needed only to publish the site.

Check

```
mvn-devops/devops.sh doctor
```

Every line must be ✓. The SSH key line may stay a warning when the site is not published from this machine.

PART 3

Nexusphere Ledger

The product: the evidence layer for AI agents, its formats, modules and conformance vectors.

Nexusphere Ledger

A self-hosted, tamper-evident evidence ledger for the actions of AI agents.

Purpose

Every action an agent takes for a person or an organization is recorded as an evidence entry: which agent acted, for which principal, what it did, on what target, whether it was allowed or denied and why, and how it ended. Entries form a SHA-256 hash chain, the database rejects any change or deletion, and the ledger signs its head with Ed25519 at a fixed interval. Anyone holding the published public key can check the chain and the checkpoints without trusting the server.

The ledger is an evidence layer, not another agent platform. Agent identity and per-call decisions are already offered by Amazon Bedrock AgentCore, Google Agent Gateway, Microsoft Entra Agent ID and open gateways such as agentgateway. The ledger keeps them and records what they decide: a gateway, a platform hook or the agent itself sends each decision and outcome to `POST /api/v1/evidence` (or `POST /api/v1/evidence/batch`) with its own agent key. Where nothing decides yet, the ledger's MCP and A2A gateways do it. What the ledger adds is the proof: a transparency log that witnesses cosign, SCITT statements and receipts, evidence packages with selective disclosure, mandates other organizations can check, and receipts both sides of an exchange hold.

Modules

Module	Responsibility
<code>ledger/chain</code>	Evidence format, hash chain, Merkle log, signed notes, COSE statements, receipts. No framework dependencies
<code>ledger/mandate</code>	SD-JWT VC mandates, status lists, JWK keys and the mandate verifier SDK for other organizations
<code>ledger/server</code>	Spring Boot service: evidence API, PostgreSQL storage, checkpoint scheduler, verification
<code>ledger/verifier</code>	Command-line verifier for evidence packages and mandates, with no server dependency
<code>ledger/demo-mcp</code>	Demo MCP server with file and mail tools, and the demo script
<code>ledger/ledger-e2e-tests</code>	End-to-end tests against the server and PostgreSQL
<code>ledger/sdk/python</code>	Python client and offline verifier for packages, statements, receipts and mandates
<code>ledger/sdk/typescript</code>	TypeScript client and offline verifier for Node and browsers, with no runtime dependencies
<code>ledger/otel-exporter</code>	OpenTelemetry Collector exporter that turns agent tool call spans into evidence, and a ready Collector
<code>ledger/adapters</code>	Adapters for LangGraph, the OpenAI Agents SDK, Amazon Bedrock AgentCore and agentgateway
<code>ledger/verifier-web</code>	Package verifier as one HTML file that runs in the browser with no network

Getting Started

Every command runs in the root of the repository. Each path below is a list of steps; do them in order and check the result of each step before the next one.

Prerequisites

Tool	Version	Check
Java	21	<code>java -version</code>
Maven	3.9	<code>mvn -version</code>
Docker	any	<code>docker compose version</code>
jq	any	<code>jq --version</code>

Dockerized

Step 1. Build the server, the verifier and the demo MCP server:

```
mvn -pl ledger/server,ledger/verifier,ledger/demo-mcp -am package -DskipTests=true
```

Step 2. Start every service:

```
docker compose --file ledger/compose.yaml --project-name ledger up -d --build
```

Step 3. Check that the ledger is up; the answer must be `{"status": "UP"}`:

```
curl -X GET http://localhost:8090/actuator/health
```

Service	URL	Description
ledger	http://localhost:8090	The ledger with the <code>dev</code> profile
demo-mcp	http://localhost:8091/mcp	Demo MCP server, reached through <code>/mcp/demo</code>
ledger-postgresql	localhost:5433	PostgreSQL, user, password and database <code>ledger</code>
ledger-adminer	http://localhost:8092	Adminer for <code>ledger-postgresql</code>
keycloak	http://localhost:8180	Keycloak, admin <code>admin / admin</code>
ledger-ui	http://localhost:5173	The web UI, built from <code>frontend</code>
otel-collector	localhost:4317, 4318	OpenTelemetry Collector, OTLP gRPC and HTTP
prometheus	http://localhost:9090	Prometheus with the ledger alerts
alertmanager	http://localhost:9093	Alertmanager, no channel until one is set

Keycloak imports the `nexusphere` realm from `ledger/keycloak/nexusphere-realm.json` with the public client `nexusphere-ledger` and the principals `alice` and `bob`, whose passwords are their names. The ledger in the compose file trusts this realm, so grants created by the operator wait for the principal's approval.

The web UI on `http://localhost:5173` opens with the operator key or a Keycloak sign-in; it is described in [Nexusphere Frontend](#). The MCP addresses have no web page. `/mcp/demo` on the ledger and `/mcp` on `demo-mcp` accept only MCP JSON-RPC `POST` requests, and the gateway also

needs an agent API key, so opening them in a browser shows nothing. Run the demo below to see calls go through the gateway.

Step 4. Run the demo; it must end with `Result: VALID` for the package and `Result: INVALID` for the changed copy:

```
ledger/demo-mcp/demo.sh
```

The script registers an agent, asks `alice` to grant it `read_file` and `send_email` on the demo server, signs `alice` in with Keycloak to approve the grant when the ledger asks for consent, calls both through the gateway, has `delete_file` denied, lists the recorded evidence, exports an evidence package, verifies it offline with the ledger's published key and shows that a changed entry fails verification. `LEDGER_URL`, `LEDGER_API_KEY`, `KEYCLOAK_URL`, `VERIFIER_JAR` and `WORK_DIR` override its defaults.

Step 5. Stop everything and remove the data:

```
docker compose --file ledger/compose.yaml --project-name ledger down
docker volume prune -f
```

Kubernetes

`ledger/kube-dev.yaml` runs the same services as the compose file in the `dev` namespace. The Keycloak realm comes from the `keycloak-realm` ConfigMap. Keycloak and the UI are reached on the same local ports as with compose, so the sign-in redirect and the token issuer stay `http://localhost:8180` and `http://localhost:5173`.

Step 1. Build the jars and the images:

```
mvn -pl ledger/server,ledger/verifier,ledger/demo-mcp -am package -DskipTests=true
docker compose --file ledger/compose.yaml --project-name ledger build
```

Step 2. Apply the manifest:

```
kubectl apply -f ledger/kube-dev.yaml
```

Step 3. Wait until every pod is `Running`:

```
kubectl get all -n dev
```

Step 4. Forward the ports, each in its own terminal:

```
kubectl port-forward service/ledger 8090:8090 -n dev
kubectl port-forward service/keycloak 8180:8080 -n dev
kubectl port-forward service/ledger-ui 5173:80 -n dev
kubectl port-forward service/ledger-adminer 8092:8080 -n dev
kubectl port-forward service/otel-collector 4318:4318 -n dev
```

Step 5. Check the ledger and run the demo as in [Dockerized](#):

```
curl -X GET http://localhost:8090/actuator/health
ledger/demo-mcp/demo.sh
```

Step 6. Remove everything:

```
kubectl delete -f ledger/kube-dev.yaml
```

Run

This path runs the server from Maven against the PostgreSQL of the compose file, for development.

Step 1. Start PostgreSQL only:

```
docker compose --file ledger/compose.yaml --project-name ledger up -d ledger-postgresql
```

Step 2. Build and start the server:

```
mvn install -DskipTests=true
LEDGER_PROFILES=postgresql,dev LEDGER_DATABASE_PORT=5433 mvn -pl ledger/server spring-boot:start
```

Step 3. Record an entry, sign a checkpoint and verify the chain; the last answer must hold

```
"valid": true :
```

```
curl -X POST http://localhost:8090/api/v1/evidence -H "Authorization: Bearer nexusphere-ledger-development-key-change-me" -H "Content-Type: application/json" -d '{"agentId":"invoice-agent","principalId":"alice","action":"tools/call","target":"send_email","decision":"ALLOW","outcome":"SUCCEEDED","attributes":{"server":"mail"}}'
curl -X POST http://localhost:8090/api/v1/checkpoints -H "Authorization: Bearer nexusphere-ledger-development-key-change-me"
curl -X GET http://localhost:8090/api/v1/verification -H "Authorization: Bearer nexusphere-ledger-development-key-change-me"
```

Step 4. Stop the server and PostgreSQL:

```
mvn -pl ledger/server spring-boot:stop
docker compose --file ledger/compose.yaml --project-name ledger down
```

Verify

Step 1. Run the unit, integration and end-to-end tests; Docker must be running for Testcontainers:

```
mvn -pl ledger/ledger-e2e-tests -am verify
```

Release

Ledger releases are tagged `ledger-v{version}` and described in the [Changelog](#). The release artifacts are the server jar (`ledger/server/target/server-*-exec.jar`), the verifier jar (`ledger/verifier/target/verifier-*-exec.jar`) and the two Docker images built by the compose file.

Environment Variables

```

LEDGER_HOST: 0.0.0.0
LEDGER_PORT: 8090
LEDGER_PROFILES: postgresql
LEDGER_DATABASE_HOST: localhost
LEDGER_DATABASE_PORT: 5432
LEDGER_DATABASE_DB: ledger
LEDGER_DATABASE_USERNAME: ledger
LEDGER_DATABASE_PASSWORD: ledger
LEDGER_API_KEY:
LEDGER_SIGNING_PRIVATE_KEY:
LEDGER_SIGNING_PUBLIC_KEY:
LEDGER_SIGNING_PREVIOUS_PRIVATE_KEY:
LEDGER_SIGNING_UNENDORSED_ROTATION: false
LEDGER_CHECKPOINT_INTERVAL: 1m
LEDGER_LOG_ORIGIN:
LEDGER_LOG_WITNESS_TIMEOUT: 10s
LEDGER_LOG_WITNESSES_{NAME}_URL:
LEDGER_LOG_WITNESSES_{NAME}_KEY:
LEDGER_LOG_WATCHED_{NAME}_KEY:
LEDGER_MCP_TIMEOUT: 60s
LEDGER_MCP_SERVERS_{NAME}_URL:
LEDGER_MCP_SERVERS_{NAME}_AUTHORIZATION:
LEDGER_MANDATE_ISSUER: http://localhost:8090
LEDGER_MANDATE_STATUS_LIST_TTL: 5m
LEDGER_A2A_TIMEOUT: 60s
LEDGER_A2A_REQUEST_MAX_AGE: 5m
LEDGER_A2A_STATUS_LIST_CACHE: 30s
LEDGER_A2A_TRUSTED_ISSUERS:
LEDGER_A2A_PEERS_{NAME}_URL:
LEDGER_A2A_PEERS_{NAME}_ISSUER:
LEDGER_A2A_AGENTS_{NAME}_URL:
LEDGER_A2A_AGENTS_{NAME}_AUTHORIZATION:
LEDGER_STREAM_TIMEOUT: 1h
LEDGER_OIDC_ISSUER:
LEDGER_OIDC_AUDIENCE:
LEDGER_OIDC_JWKS_URI:
LEDGER_OIDC_PRINCIPAL_CLAIM: sub
LEDGER_OIDC_CLIENT_ID:
LEDGER_SECRETS_DIR: /run/secrets/
LEDGER_MANAGEMENT_PORT:
LEDGER_RATE_LIMIT_PER_MINUTE: 1200
LEDGER_RATE_LIMIT_BURST: 200

```

`LEDGER_API_KEY` and the signing keys have no default outside the `dev` profile, and the ledger refuses to start with the published development secrets unless `dev` is active. The private key is a base64 PKCS#8 Ed25519 key and the public key is its base64 X.509 encoding; the ledger refuses to start when they do not match. The tables live in the `ledger` schema, so the ledger can share a database with other applications. Any setting can also come from a file of the same name in `LEDGER_SECRETS_DIR`, which is how a secret manager hands over keys; backups, keys and metrics are described in [Operations](#). Each caller, by key or by address when it has none, gets `LEDGER_RATE_LIMIT_PER_MINUTE` requests a minute with bursts up to `LEDGER_RATE_LIMIT_BURST`; over it the answer is 429 `RATE_LIMIT_EXCEEDED` with `Retry-After`, and `0` turns the limit off.

```
openssl genpkey -algorithm ed25519 -outform DER -out ledger-signing.der
export LEDGER_SIGNING_PRIVATE_KEY=$(base64 -w0 ledger-signing.der)
export LEDGER_SIGNING_PUBLIC_KEY=$(openssl pkey -inform DER -in ledger-signing.der -pubout -
outform DER | base64 -w0)
```

Evidence Format

Field	Description
format	nexusphere-ledger/evidence/v1
id	UUID assigned by the ledger
sequence	Position in the chain, starting at 1 without gaps
occurredAt	When the action happened, at most five minutes in the future
recordedAt	When the ledger recorded it
agentId	The acting agent (required)
principalId	The person or organization the agent acted for (required)
action	What was done, for example tools/call (required)
target	The tool, resource or counterparty
decision	ALLOW or DENY
reason	Why the decision was taken
delegationId	The grant or mandate the agent acted under
inputHash	Lowercase SHA-256 of the input
outputHash	Lowercase SHA-256 of the output
outcome	SUCCEEDED , FAILED , DENIED or PENDING (required); DENY requires DENIED , PENDING requires ALLOW
correlationId	Caller correlation, for example a conversation
attributes	Up to 32 string attributes
previousHash	Hash of the previous entry; 64 zeros for the first entry
hash	SHA-256 of the canonical JSON of format , sequence , previousHash and contentHash

The content hash is the SHA-256 of the canonical JSON of format and every field except sequence , previousHash and hash . The link (sequence , previousHash , contentHash , hash) can be checked without the content, so a package can keep entries private and still prove the chain. Canonical JSON has sorted keys, no whitespace and null for absent fields. Timestamps are UTC with microseconds, for example 2026-10-01T08:00:00.123456Z . A checkpoint signs the canonical JSON of format (nexusphere-ledger/checkpoint/v1), sequence , headHash , createdAt and keyId . The key ID is the first 16 hex characters of the SHA-256 of the X.509 public key.

Verification

GET /api/v1/verification recomputes every hash and link, checks the signature of every checkpoint against the entry it names and checks the head. The same checks run outside the server with ledger/chain : page through GET /api/v1/evidence , feed the entries to ChainVerifier ,

and verify each checkpoint with `SignedCheckpoint.verify` and the key it names from `GET /api/v1/keys`.

Transparency Log and Witnesses

Besides the hash chain, every entry is a leaf of an RFC 9162 Merkle tree: the leaf is the 32-byte entry hash, so leaf $n - 1$ is the entry with sequence n . Whenever the ledger signs a checkpoint it also signs a log checkpoint in the C2SP signed-note format (`origin`, tree size and base64 root hash, then an Ed25519 signature line). The origin is `LEDGER_LOG_ORIGIN`, or the mandate issuer without its scheme, for example `localhost:8090`. With the tree, anyone can prove that one entry is in the log with about $\log_2(n)$ hashes, and that a later checkpoint extends an earlier one without rewriting it.

A ledger alone can still show different histories to different people. Witnesses close that gap: each witness keeps the last checkpoint it saw for a log and cosigns a new one only with a valid consistency proof from it. Witnesses follow the C2SP `tlog-witness` protocol and sign `tlog-cosignature/v1` lines, so any compatible witness works, and every ledger is one: set `LEDGER_LOG_WATCHED_{NAME}_KEY` to the verifier key of each log it should witness, with `{NAME}` equal to the log's origin. To ask witnesses for cosignatures, set `LEDGER_LOG_WITNESSES_{NAME}_URL` to the witness's base URL (the ledger appends `/add-checkpoint`) and `LEDGER_LOG_WITNESSES_{NAME}_KEY` to its verifier key. The ledger asks every witness after each checkpoint and keeps each cosignature it gets; a witness that is down is asked again next time.

```
curl http://localhost:8090/public/v1/log/key
curl http://localhost:8090/public/v1/witness/key
curl http://localhost:8090/public/v1/log/checkpoint
curl "http://localhost:8090/api/v1/log/proofs/inclusion?sequence=7" -H "Authorization: Bearer nexusphere-ledger-development-key-change-me"
curl "http://localhost:8090/api/v1/log/proofs/consistency?firstSize=7&secondSize=42" -H "Authorization: Bearer nexusphere-ledger-development-key-change-me"
```

```
localhost:8090
42
0dpbGc4h7JcvSy6dXq0XpLM4pN0iA0p9cX0XQ4nHqEo=

- localhost:8090 b4Ex7xF0kZ0G1Xo9ZfPu0eQyD3l8Q3mJ2tV9...
- partner.example b8Gk2wAAAABnB10w0Tq9m3Q1...
```

`GET /api/v1/verification` also rebuilds the tree from the chain and checks every log checkpoint against it, so a log checkpoint that does not match the evidence is reported like a broken link.

SCITT Statements and Receipts

Every entry is also available as an IETF SCITT signed statement and a log receipt, both COSE_Sign1 (`application/cose`) signed with EdDSA by the ledger key. The statement is a COSE hash envelope: its payload is the 32-byte content hash (`payload_hash_alg` `SHA-256`, preimage content type `application/vnd.nexusphere.evidence+json`), its CWT claims name the mandate issuer as `iss` and `urn:uuid:{id}` as `sub`, and the protected headers `nexusphere-sequence` and `nexusphere-previous-hash` place it in the chain, so the Merkle leaf can be computed from the statement alone. The receipt follows RFC 9942 with the RFC 9162 tree algorithm: the inclusion

proof is in the unprotected header (396) and the signed payload is the tree root, which is left out of the message. A receipt is issued for a log checkpoint, the latest by default or the one named by `treeSize`, and the ledger signs a new checkpoint first when the entry is not in the latest one yet.

Only the content hash leaves the ledger in a statement, so a statement can be shared without the entry. Whoever has the entry checks it against the statement's payload. The formats are standard COSE and any COSE library can read them, but the evidence fields are Nexusphere's own; they are not an IETF agent-action profile. A2A receipts stay JWS and name the evidence sequence and hash of the exchange, whose statement and receipt the receiving ledger serves like any other.

```
curl http://localhost:8090/api/v1/evidence/{id}/statement -H "Authorization: Bearer nexusphere-
ledger-development-key-change-me" -o statement.cose
curl http://localhost:8090/api/v1/evidence/{id}/receipt -H "Authorization: Bearer nexusphere-
ledger-development-key-change-me" -o receipt.cose
java -jar ledger/verifier/target/verifier-1.0.0-SNAPSHOT-exec.jar statement --public-key "$(curl -
s http://localhost:8090/api/v1/keys -H 'Authorization: Bearer nexusphere-ledger-development-key-
change-me' | jq -r '.[ ] | select(.status == "ACTIVE") | .publicKey')" --receipt receipt.cose
statement.cose
```

```
Nexusphere Ledger evidence statement
Issuer      : http://localhost:8090
Subject     : urn:uuid:7c0e5a52-8f0b-4d1e-9a51-3c1f0b8e2d44
Sequence    : 7
Content hash: 5e1c...
Entry hash  : 9a40...
Signature   : valid, key 3f2a9c0d41b7e865
Receipt     : valid, log localhost:8090 at 42 entries, root
0dpbGc4h7JcvSy6dXq0XpLM4pN0iA0p9cX0XQ4nHqEo=
Result: VALID
```

Evidence Packages

`POST /api/v1/packages` exports an evidence package for an auditor, a court or a counterparty. The request selects evidence by `agentId`, `principalId`, `fromSequence` and `toSequence`. The package (`nexusphere-ledger/package/v1`) holds the public keys with their rotation records, an anchor checkpoint (the last checkpoint before the first selected entry, or genesis), an end checkpoint (the first checkpoint at or after the last selected entry, created at the head when none exists yet) and every link between them. Only selected links carry their full entry; all others carry only their link, so nothing outside the selection is disclosed. A package holds at most 200,000 links. Its `log` field carries the log checkpoint at the end checkpoint's sequence with every cosignature the ledger holds, and for each disclosed entry an inclusion proof, its SCITT statement and its receipt for that checkpoint.

The verifier needs no server and no database. It checks the signatures of both checkpoints, recomputes every link from the anchor to the end checkpoint, checks every disclosed entry against its content hash and the scope of the package, and exits with `0` for a valid package, `1` for an invalid one and `2` for a usage error. Pass the ledger's public key from `GET /api/v1/keys` (or a key published elsewhere) with `--public-key` so the verifier does not trust the key inside the package. When the package has a `log`, the verifier also checks that the log checkpoint is signed by the ledger, covers the end checkpoint and proves every disclosed entry, and that each statement and receipt is signed by a trusted key, describes its entry and proves it in the same log

checkpoint. Pass each witness's verifier key with `--witness` and the number of cosignatures needed with `--witnesses-required` (all given witnesses by default) to require that independent witnesses saw the same log.

```
curl -X POST http://localhost:8090/api/v1/packages -H "Authorization: Bearer nexusphere-ledger-development-key-change-me" -H "Content-Type: application/json" -d '{"agentId":"invoice-agent"}' -o package.json
mvn -pl ledger/verifier -am package -DskipTests=true
java -jar ledger/verifier/target/verifier-1.0.0-SNAPSHOT-exec.jar --public-key "$(curl -s http://localhost:8090/api/v1/keys -H 'Authorization: Bearer nexusphere-ledger-development-key-change-me' | jq -r '.[] | select(.status == "ACTIVE") | .publicKey')" package.json
```

```
Nexusphere Ledger evidence package
Signing key : 3f2a9c0d41b7e865 (pinned)
Checkpoint : sequence 42, signed at 2026-10-07T12:00:00.000123Z
Anchor      : checkpoint 40
Chain       : 2 links from sequence 41
Disclosed   : 1 entries, agent invoice-agent
Log         : localhost:8090, 42 entries, 1 disclosed entries proven
Receipts    : 1 SCITT statements with receipts
Witnesses   : partner.example
Result: VALID
```

Key Rotation

The ledger keeps every signing key it has used. Checkpoints, mandates, status lists and A2A receipts are signed with the active key, and everything signed with a retired key still verifies, because each signature names its key and the retired keys stay published in `GET /api/v1/keys`, `GET /public/v1/keys` and every evidence package.

To rotate, start the ledger with a new key pair in `LEDGER_SIGNING_PRIVATE_KEY` and `LEDGER_SIGNING_PUBLIC_KEY` and the old private key in `LEDGER_SIGNING_PREVIOUS_PRIVATE_KEY`. At startup the ledger retires the old key and stores a rotation record (`nexusphere-ledger/key-rotation/v1`) that both keys sign: the old key endorses the new one and the new key names the old one as its predecessor. A verifier that pinned either key reaches the other through this record. The rotation is recorded as a `key/rotate` evidence entry (the first key as `key/activate`), and the previous key can be removed from the settings after the restart. Every instance has to restart with the new key.

When the old private key is lost, set `LEDGER_SIGNING_UNENDORSED_ROTATION` to `true` instead. The rotation record then carries only the new key's signature, so a verifier that pinned the old key does not trust the new key, while one that pins the new key still trusts the old checkpoints. A retired key can never become active again, and the ledger refuses to start when the configured key changes without one of these two settings.

```
openssl genpkey -algorithm ed25519 -outform DER -out ledger-signing-2.der
export LEDGER_SIGNING_PREVIOUS_PRIVATE_KEY=${LEDGER_SIGNING_PRIVATE_KEY}
export LEDGER_SIGNING_PRIVATE_KEY=$(base64 -w0 ledger-signing-2.der)
export LEDGER_SIGNING_PUBLIC_KEY=$(openssl pkey -inform DER -in ledger-signing-2.der -pubout -outform DER | base64 -w0)
curl http://localhost:8090/api/v1/keys -H "Authorization: Bearer nexusphere-ledger-development-key-change-me"
```

Agents, Grants and Decisions

The operator holds `LEDGER_API_KEY`. Every agent is registered by the operator and gets its own API key, shown only once and stored as a SHA-256 hash. An agent can record and read only its own evidence, grants and decisions; registering agents, creating and revoking grants, creating checkpoints and running the full verification belong to the operator.

A grant lets one agent act for one principal: a list of actions and a list of targets, each exact or ending with `*`, valid until `expiresAt`, optionally from `notBefore` and for at most `maxUses` allowed decisions. Its terms are hashed as the canonical JSON of `nexusphere-ledger/grant/v1` (`GrantTerms` in `ledger/chain`), and the hash is the `inputHash` of the `grant/create` evidence, so anyone can check that a grant was not changed after it was recorded.

Before acting, an agent asks for a decision. The ledger picks the active grant that covers the action and target and expires first, counts one use, and records the decision as evidence with outcome `PENDING`, or `DENIED` with the reason. The agent then reports `SUCCEEDED` or `FAILED` once; this is a second evidence entry carrying the `decisionId` attribute, the grant as `delegationId` and the same `inputHash` and `correlationId`. Concurrent decisions never use a grant more than `maxUses` times. A reported `FAILED` keeps the use, because the ledger cannot know whether the action ran. The gateways return the use themselves when they see that the call never ran: the MCP or A2A server could not be reached, the MCP server rejected the request (HTTP 4xx, or JSON-RPC parse, invalid request, unknown method or invalid params), or the peer ledger refused the A2A request without a receipt. The outcome entry then carries `grant.useReturned`.

Reason code	Decision	Meaning
<code>ALLOWED_BY_GRANT</code>	ALLOW	An active grant covers the action and the target
<code>AGENT_NOT_ACTIVE</code>	DENY	The agent is unknown or disabled
<code>NO_ACTIVE_GRANT</code>	DENY	The principal has no active grant for this agent
<code>NOT_COVERED</code>	DENY	Active grants exist, but none covers the action and target
<code>NOT_YET_VALID</code>	DENY	The covering grant starts later
<code>USES_EXHAUSTED</code>	DENY	The covering grant has used all its uses

Agent, grant and decision changes are evidence too: `agent/register`, `agent/disable`, `agent/rotate-key`, `grant/create`, `grant/approve`, `grant/deny` and `grant/revoke`, with the owner or the granting principal as `principalId`.

```
curl -X POST http://localhost:8090/api/v1/agents -H "Authorization: Bearer nexusphere-ledger-development-key-change-me" -H "Content-Type: application/json" -d '{"agentId":"invoice-agent","name":"Invoice agent","ownerId":"acme"}'
curl -X POST http://localhost:8090/api/v1/grants -H "Authorization: Bearer nexusphere-ledger-development-key-change-me" -H "Content-Type: application/json" -d
'{"principalId":"alice","agentId":"invoice-agent","actions":["tools/call"],"targets":
["send_email","read_*"],"expiresAt":"2026-12-31T00:00:00Z","maxUses":100}'
curl -X POST http://localhost:8090/api/v1/decisions -H "Authorization: Bearer {agentApiKey}" -H
"Content-Type: application/json" -d
'{"principalId":"alice","action":"tools/call","target":"send_email"}'
curl -X POST http://localhost:8090/api/v1/decisions/{decisionId}/outcome -H "Authorization: Bearer
{agentApiKey}" -H "Content-Type: application/json" -d '{"outcome":"SUCCEEDED"}'
```

Principal Login and Consent

Without OIDC, the operator creates grants and the principal's consent is the operator system's job. When `LEDGER_OIDC_ISSUER` is set, principals sign in with that OpenID Connect provider (Keycloak, Entra ID, Auth0, Okta or any other) and decide on their own grants. The ledger accepts the provider's access or ID token as a bearer token on `/api/v1/principal/**` only. It checks the signature with the provider's keys (from the discovery document, or from `LEDGER_OIDC_JWKS_URI` when the ledger reaches the provider at another address than the issuer), the issuer, the expiry, and that `LEDGER_OIDC_AUDIENCE` is in `aud` or is the `azp` client. The principal ID is the claim named by `LEDGER_OIDC_PRINCIPAL_CLAIM`, for example `preferred_username` or `email`. `GET /public/v1/oidc` tells a sign-in page the issuer and the public client to use, `LEDGER_OIDC_CLIENT_ID` or else the audience.

With OIDC on, a grant created by the operator, or asked for by an agent for itself on `POST /api/v1/grants`, starts as `PENDING`. A pending grant allows nothing and gives no mandate. The principal lists it, then approves it, which makes it `ACTIVE` with `consent PRINCIPAL`, or denies it, which makes it `DENIED`. A principal can also create a grant on its own behalf, active at once, and revoke its own grants, which revokes their mandates. Each step is evidence (`grant/create`, `grant/approve`, `grant/deny`, `grant/revoke`) with `consent.issuer`, `consent.subject`, `consent.authTime` and `consent.token`, the SHA-256 of the token the principal used, so the consent can be traced to one sign-in at the provider without storing the token.

```
TOKEN=$(curl -s http://localhost:8180/realms/nexusphere/protocol/openid-connect/token -d
grant_type=password -d client_id=nexusphere-ledger -d username=alice -d password=alice | jq -r
.access_token)
curl -X GET "http://localhost:8090/api/v1/principal/grants?state=PENDING" -H "Authorization:
Bearer ${TOKEN}"
curl -X POST http://localhost:8090/api/v1/principal/grants/{id}/approve -H "Authorization: Bearer
${TOKEN}"
```

MCP Gateway

The ledger can stand in front of MCP servers that speak Streamable HTTP. Each server gets a name and a URL, for example `LEDGER_MCP_SERVERS_FILES_URL=http://files-mcp:3000/mcp`, and an optional `Authorization` header the ledger sends to it. The agent points its MCP client at `/mcp/{name}` with its own API key and names the principal it works for in the `X-Ledger-Principal` header.

Every `tools/call` is decided first, with action `tools/call` and target `{name}/{tool}`, so a grant on `files/*` or `files/read_file` covers it. A denied call never reaches the server and is answered with JSON-RPC error `-32003` and the decision ID and reason code in `error.data`. An allowed call is forwarded, and its outcome is reported from the answer: a JSON-RPC error, `isError: true`, a non-2xx status or an unreachable server (`-32002`, HTTP 502) count as `FAILED`. The input hash is the SHA-256 of the request body, the output hash is the SHA-256 of the answer, the MCP session ID is the correlation ID, and every answer to a decided call carries the `X-Ledger-Decision` header. Other messages, such as `initialize`, `tools/list` and notifications, pass through unchanged with the `Mcp-Session-Id` header.

When the client accepts `text/event-stream`, a streamed answer is relayed event by event as it arrives, including the requests and notifications the server sends inside it (for example `elicitation/create` or `progress`), and the outcome is recorded when the stream ends from the

event that answers the call, with `mcp.events` (the number of events) and `mcp.stream` (`COMPLETE` or `BROKEN`) as attributes. A client that accepts only JSON gets the answer as one JSON response. `GET /mcp/{name}` relays the server-to-client stream with `Mcp-Session-Id` and `Last-Event-ID` , and the client's answers to server requests pass through on `POST` . A stream stays open for at most `LEDGER_STREAM_TIMEOUT` . JSON-RPC batches are not supported, because MCP removed them in protocol version 2025-06-18; a batch is answered with `-32600` .

```
curl -X POST http://localhost:8090/mcp/files -H "Authorization: Bearer {agentApiKey}" -H "X-Ledger-Principal: alice" -H "Content-Type: application/json" -d
'{"jsonrpc":"2.0","id":1,"method":"tools/call","params":{"name":"read_file","arguments":{"path":"/tmp/report.txt"}}}'
```

Mandates

A grant lives inside one ledger. When an agent acts for its principal at another organization, it carries a mandate: a signed, self-contained copy of the grant that the other side can check without calling back. A mandate is an IETF SD-JWT VC signed with the ledger's Ed25519 key (`alg` `EdDSA` , `typ` `dc+sd-jwt` , `kid` the ledger key ID, `vct` `urn:nexusphere:vct:agent-mandate:1`): the issuer-signed JWT followed by its disclosures, each ending in `~` . The operator, or the agent the grant was given to, asks for one with `POST /api/v1/mandates` and the grant ID, an optional `audience` (the organization that will receive it) and an optional `expiresAt` that may not be later than the grant's own expiry. Only an active or not-yet-valid grant can be turned into a mandate.

Claim	Meaning
<code>iss</code>	The ledger, <code>LEDGER_MANDATE_ISSUER</code>
<code>vct</code>	<code>urn:nexusphere:vct:agent-mandate:1</code>
<code>sub</code>	The agent ID
<code>aud</code>	The receiving organization, when given
<code>jti</code>	The mandate ID
<code>iat</code> , <code>nbfi</code> , <code>exp</code>	Issued, valid from and expiry, in epoch seconds
<code>mandate.principal</code>	The person or organization the agent acts for (selectively disclosable)
<code>mandate.actions</code>	Allowed actions, exact or with a trailing <code>*</code>
<code>mandate.targets</code>	Allowed targets, exact or with a trailing <code>*</code>
<code>mandate.maxUses</code>	The grant's use limit, when it has one
<code>mandate.grant</code>	The grant ID (selectively disclosable)
<code>mandate.termsHash</code>	The grant terms' SHA-256, as in the grant's evidence (selectively disclosable)
<code>status.status_list</code>	<code>idx</code> and <code>uri</code> of the mandate's bit in the status list
<code>_sd_alg</code>	<code>sha-256</code> , the hash of the disclosures listed in <code>mandate._sd</code>

The principal, grant and terms hash are disclosures, so an agent can hand a mandate on without them: `SdJwt.parse(token).present(Set.of("grant"))` keeps only the grant, and the receiver still checks the signature, time, audience, coverage and revocation. Actions, targets, the use limit, time and status are always in the signed JWT and cannot be withheld. The A2A gateway always

sends the full mandate, because the receiving ledger records the principal. Agents hold API keys rather than key pairs, so mandates have no key binding; the audience and the A2A request proof take its place.

Revocation follows the IETF Token Status List draft. `GET /public/v1/mandates/status` returns a signed `statuslist+jwt` with one bit per mandate (1 is revoked), deflated with `zlib` and `base64url` encoded in `status_list.lst`, and an `exp` of `LEDGER_MANDATE_STATUS_LIST_TTL` after it was made. Revoking a mandate sets its bit, and revoking a grant revokes every mandate issued from it. The keys are published as a JWK set at `GET /public/v1/keys`. Both public endpoints need no API key. Issuing and revoking a mandate are recorded as evidence (`mandate/issue` with the SHA-256 of the token as the input hash, and `mandate/revoke`), with the grant as the delegation ID and the mandate ID as the target.

```
curl -X POST http://localhost:8090/api/v1/mandates -H "Authorization: Bearer {agentApiKey}" -H
"Content-Type: application/json" -d '{"grantId":
{grantId},"audience":"https://supplier.example"}'
curl http://localhost:8090/public/v1/keys
curl http://localhost:8090/public/v1/mandates/status
```

Verifying a Mandate

An organization that receives a mandate checks it with `MandateVerifier` from `ledger/mandate`, a plain Java library with no Spring dependency. It trusts only the issuers it is told to trust, reads their keys from `{issuer}/public/v1/keys`, checks the signature, `nbf` and `exp` with a clock skew (60 seconds by default), the audience, and, when an action and target are given, that the mandate covers them. It then reads the status list named in the mandate, which must be served by the issuer, checks its signature, issuer, address and expiry, and looks up the mandate's bit. Keys and status lists are cached (5 minutes by default, and never past the status list's `exp`). When the keys or the status list cannot be read the mandate is not valid, so a verifier fails closed. Each problem has a code: `MALFORMED`, `WRONG_TYPE`, `UNTRUSTED_ISSUER`, `UNKNOWN_KEY`, `BAD_SIGNATURE`, `NOT_YET_VALID`, `EXPIRED`, `WRONG_AUDIENCE`, `NOT_COVERED`, `REVOKED` and `STATUS_UNAVAILABLE`.

```
MandateVerifier verifier = MandateVerifier.builder()
    .trustIssuer("https://ledger.acme.example")
    .audience("https://supplier.example")
    .build();
MandateCheck check = verifier.verify(token, "a2a/send", "supplier/orders");
if (!check.valid()) {
    throw new AccessDeniedException(check.problems().toString());
}
String principal = check.claims().principalId();
```

The verifier jar checks a mandate from the command line. `--public-key` pins the issuer's key instead of reading it from the issuer, and `--skip-status` leaves out the revocation check for offline use. The exit codes are the same as for packages.

```
java -jar ledger/verifier/target/verifier-1.0.0-SNAPSHOT-exec.jar mandate --issuer
http://localhost:8090 --audience https://supplier.example --action a2a/send --target
supplier/orders mandate.jwt
```

AP2 and Verifiable Intent

AP2 and Mastercard's Verifiable Intent, now developed at the FIDO Alliance, carry payment mandates as SD-JWT credentials too. A Nexusphere mandate is not an AP2 mandate, but its parts line up, so a bridge can translate one into the other. AP2 signs with ES256 and binds mandates to the holder's key; Nexusphere signs with EdDSA and has no key binding.

AP2 / Verifiable Intent	Nexusphere
Intent mandate signed for the user	Mandate issued from a grant the principal approved with their own sign-in
User authorization	The <code>grant/approve</code> evidence of the principal's consent, named by <code>termsHash</code>
Merchants or payees the intent allows	<code>aud</code> and <code>mandate.targets</code>
What the agent may do	<code>mandate.actions</code>
Intent expiry	<code>exp</code> , never later than the grant
Spending or use limit	<code>mandate.maxUses</code> , counted by both ledgers
Credential status	<code>status.status_list</code> , the IETF Token Status List
Cart and payment mandates	No equivalent: Nexusphere does not handle payments
<code>merchant_authorization</code> from the merchant	The receipt the receiving ledger signs for an A2A exchange
Audit trail of the transaction	Evidence on both ledgers, with SCITT statements and receipts

A2A Gateway

Two organizations that each run a ledger can let their agents talk over A2A (JSON-RPC over HTTP) with evidence on both sides that matches entry for entry. The sending ledger names each peer with the URL of the peer's inbound endpoint and the peer ledger's issuer, for example

`LEDGER_A2A_PEERS_SUPPLIER_URL=https://ledger.supplier.example/a2a/in/sales` and

`LEDGER_A2A_PEERS_SUPPLIER_ISSUER=https://ledger.supplier.example` . The receiving ledger lists the issuers it trusts in `LEDGER_A2A_TRUSTED_ISSUERS` (comma separated) and names its own agents with their A2A URL and an optional `Authorization` header, for example

`LEDGER_A2A_AGENTS_SALES_URL=http://sales-agent:9000/a2a` .

The agent sends its JSON-RPC request to `/a2a/out/{peer}` with its own API key and the `X-Ledger-Principal` header. The ledger decides action `a2a/send` on target `{peer}/{method}` , so a grant on `supplier/*` covers it. A denied request never leaves and is answered with JSON-RPC error `-32003` . An allowed request carries two headers: `X-Nexusphere-Mandate` , a mandate for the grant with the peer's issuer as audience (reused while it is valid for at least another minute), and `X-Nexusphere-Request` , a request proof signed by the sending ledger (`typ` `nexusphere-a2a-request+jwt`) that binds the mandate, the method, the SHA-256 of the body and the decision ID as the request ID.

The receiving ledger answers on `/a2a/in/{agent}` without an API key. It verifies the mandate with `MandateVerifier` (trusted issuer, key from the issuer's JWKS, signature, time, its own issuer as audience, the `a2a/send` action and the revocation status) and the request proof (same issuer, key, mandate and agent, its own issuer as audience, the method, the body hash and an `iat` within `LEDGER_A2A_REQUEST_MAX_AGE`). A request ID is accepted once. A mandate or proof that cannot be trusted is answered with HTTP 401 and is not recorded. A trusted mandate that is revoked,

expired or for another audience is answered with HTTP 403 and recorded as an `a2a/receive` DENY entry. Otherwise the request goes to the local agent and one `a2a/receive` entry records it with the same input and output hashes as the sender, the mandate as delegation ID and the request ID as correlation ID. The answer carries `X-Nexusphere-Receipt`, a receipt signed by the receiving ledger (`typ nexusphere-a2a-receipt+jwt`) with the request and response hashes, the HTTP status, the outcome, and the sequence and hash of its evidence entry.

The sending ledger verifies the receipt with the peer's published key, records the outcome with `a2a.receipt` set to `VERIFIED`, `INVALID` or `MISSING` and the peer's evidence sequence and hash, and returns the answer with `X-Ledger-Decision`, `X-Ledger-Exchange`, `X-Ledger-Receipt` and the receipt. Both ledgers keep the mandate, the request proof and the receipt of every exchange at `GET /api/v1/a2a/exchanges/{id}`, so either side can later show what the other signed. The receiving ledger also counts the uses of every mandate on its own: once the requests it accepted under a mandate reach `mandate.maxUses`, it denies the next one with `USES_EXHAUSTED`, so a sender that ignores its own count gets nowhere. A request that could not reach the receiving agent does not count.

Streaming methods (`message/stream`, `tasks/resubscribe`) work the same way. When the agent answers with server-sent events, both ledgers relay every event as it arrives. The response hash of a stream is the SHA-256 of the `data` of every event, each followed by a newline, so both sides compute the same value. The exchange fails when an event is a JSON-RPC error, a task ends `failed` or `rejected`, or the stream breaks. When the stream ends, the receiving ledger records its entry and sends the receipt as a last event named `nexusphere-receipt`; the sending ledger takes that event out, verifies the receipt and records the outcome with `a2a.events` and `a2a.stream`, so the agent sees only the events of the task.

```
curl -X POST http://localhost:8090/a2a/out/supplier -H "Authorization: Bearer {agentApiKey}" -H
"X-Ledger-Principal: alice" -H "Content-Type: application/json" -d
'{"jsonrpc":"2.0","id":"1","method":"message/send","params":{"message":
{"role":"user","messageId":"1","parts":[{"kind":"text","text":"Order 40 pallets"}]}}'
```

Conformance Vectors

[Conformance Vectors](#) hold fixed inputs and outputs for every format the ledger produces, made with the published RFC 8032 test keys: canonical JSON, the evidence chain and checkpoints, Merkle roots and proofs, signed notes and cosignatures, SCITT statements and receipts, and an SD-JWT VC mandate. Another implementation checks itself against them, and

`ConformanceVectorsTest` in `ledger/verifier` fails when the ledger's own output changes.

API

All `/api/**`, `/mcp/**` and `/a2a/out/**` endpoints require `Authorization: Bearer {LEDGER_API_KEY}` or an agent API key, except `/api/v1/principal/**`, which requires a principal token from the OIDC provider and answers `403 FORBIDDEN` to everyone else. Operator-only endpoints answer `403 FORBIDDEN` to agents.

Method	Path	Description
POST	<code>/api/v1/evidence</code>	Record an evidence entry; returns <code>201</code> with <code>Location</code>

Method	Path	Description
POST	/api/v1/evidence/batch	Record up to 500 entries in order as one run, all or none; an item error names its <code>index</code>
GET	/api/v1/evidence/{id}	Get an evidence entry
GET	/api/v1/evidence/{id}/statement	The entry as a SCITT signed statement (<code>application/cose</code>)
GET	/api/v1/evidence/{id}/receipt	RFC 9942 receipt for the entry at <code>treeSize</code> , by default the latest log checkpoint
GET	/api/v1/evidence	List entries by <code>agentId</code> , <code>principalId</code> , <code>after</code> and <code>limit</code> (max 500)
GET	/api/v1/ledger/head	Current sequence and hash
POST	/api/v1/checkpoints	Sign the current head now; 409 <code>LEDGER_EMPTY</code> when nothing is recorded (operator)
GET	/api/v1/checkpoints/latest	Latest checkpoint
GET	/api/v1/checkpoints	List checkpoints by <code>after</code> and <code>limit</code>
GET	/api/v1/log/proofs/inclusion	Inclusion proof for a <code>sequence</code> in the log at <code>treeSize</code> , by default the latest log checkpoint
GET	/api/v1/log/proofs/consistency	Consistency proof from <code>firstSize</code> to <code>secondSize</code> , by default the latest log checkpoint
GET	/api/v1/keys	Signing keys with their status and rotation records
POST	/api/v1/packages	Export an evidence package by <code>agentId</code> , <code>principalId</code> , <code>fromSequence</code> and <code>toSequence</code> (operator)
POST	/api/v1/packages/verify	Verify a package with the verifier library, pinned to <code>publicKey</code> when given (operator)
GET	/api/v1/verification	Full verification report (operator)
POST	/api/v1/agents	Register an agent and return its API key once (operator)
GET	/api/v1/agents	List agents by <code>after</code> and <code>limit</code> (operator)
GET	/api/v1/agents/{agentId}	Get an agent
POST	/api/v1/agents/{agentId}/disable	Disable an agent and its key (operator)
POST	/api/v1/agents/{agentId}/key	Issue a new API key; the old one stops working (operator)
POST	/api/v1/grants	Create a grant (operator), or ask for one for itself (agent, only with OIDC)
GET	/api/v1/grants	List grants by <code>agentId</code> , <code>principalId</code> , <code>after</code> and <code>limit</code>
GET	/api/v1/grants/{id}	Get a grant with its status and uses
POST	/api/v1/grants/{id}/revoke	Revoke a grant with an optional <code>reason</code> (operator)
GET	/api/v1/principal/evidence	The evidence about the principal by <code>after</code> and <code>limit</code>
GET	/api/v1/principal	The signed-in principal
GET	/api/v1/principal/grants	List the principal's grants by <code>state</code> , <code>after</code> and <code>limit</code>
GET	/api/v1/principal/grants/{id}	Get one of the principal's grants
POST	/api/v1/principal/grants	Create a grant on the principal's own behalf; returns 201
POST	/api/v1/principal/grants/{id}/approve	Approve a pending grant

Method	Path	Description
POST	/api/v1/principal/grants/{id}/deny	Deny a pending grant with an optional <code>reason</code>
POST	/api/v1/principal/grants/{id}/revoke	Revoke one of the principal's grants with an optional <code>reason</code>
POST	/api/v1/decisions	Decide an action for an agent and record it; returns <code>201</code>
GET	/api/v1/decisions/{id}	Get a decision and its outcome
POST	/api/v1/decisions/{id}/outcome	Report <code>SUCCEEDED</code> or <code>FAILED</code> once for an allowed decision
POST	/api/v1/mandates	Issue a signed mandate from a grant; returns <code>201</code>
GET	/api/v1/mandates	List the mandates of a <code>grantId</code>
GET	/api/v1/mandates/{id}	Get a mandate, its token and status
POST	/api/v1/mandates/{id}/revoke	Revoke a mandate with an optional <code>reason</code> (operator)
GET	/public/v1/keys	Active and retired signing keys as a JWK set, no API key
GET	/public/v1/mandates/status	Signed mandate status list, no API key
GET	/public/v1/log/checkpoint	Latest log checkpoint as a signed note with its cosignatures, no API key
GET	/public/v1/log/key	Verifier key of the log, no API key
GET	/public/v1/witness/key	Verifier key this ledger cosigns with as a witness, no API key
POST	/public/v1/witness/add-checkpoint	C2SP witness endpoint: cosign a watched log's checkpoint, <code>404</code> when it watches none, no API key
GET	/public/v1/oidc	The OIDC issuer and client for principal sign-in, <code>404</code> without OIDC
POST	/mcp/{server}	MCP gateway: decide, forward and record a <code>tools/call</code> ; forward other messages
GET	/mcp/{server}	Relay the MCP server-to-client stream
DELETE	/mcp/{server}	Close an MCP session on the server
POST	/a2a/out/{peer}	A2A gateway: decide, attach a mandate and request proof, forward and record with the peer's receipt
POST	/a2a/in/{agent}	A2A inbound: verify the mandate and proof, forward to the agent, record and sign a receipt; no API key
GET	/api/v1/a2a/exchanges/{id}	Get an A2A exchange with its mandate, request proof and receipt
GET	/actuator/health	Health, open for probes

```
{
  "code": "INVALID_REQUEST",
  "message": "The evidence has invalid fields.",
  "details": {
    "fields": {
      "principalId": "must not be blank"
    }
  }
}
```

Nexusphere Ledger Chain

The evidence format and its cryptography, with no framework.

Purpose

Anyone who checks the ledger, including an auditor or a counterparty without the server, needs the exact rules for hashing and signing. This library is those rules in plain Java, shared by the server, the verifier and anyone who wants to verify evidence on their own.

Responsibilities

- Canonical JSON, so the same content always hashes to the same bytes
- `EvidenceEntry` with its content hash and link hash, and `EvidenceLink` for entries disclosed without their content
- `ChainVerifier`, which checks the hash chain from genesis or from an anchor
- Ed25519 keys and signatures (`SigningKeys`) and signed checkpoints (`Checkpoint` , `SignedCheckpoint`)
- Signed key rotations (`KeyRotation`) and the keys a pinned key reaches through them (`TrustedKeys`)
- `GrantTerms`, the hashed terms of a grant
- `MerkleTree`, the RFC 9162 tree hash with inclusion and consistency proofs and their verification
- `LogCheckpoint` and `NoteKey`, C2SP signed-note checkpoints, `tlog-cosignature/v1` cosignatures and verifier keys
- `Cbor` and `CoseSign1`, deterministic CBOR and COSE_Sign1 with EdDSA, and `EvidenceStatement` and `LogReceipt`, the SCITT signed statement of an entry and its RFC 9942 inclusion receipt

Dependencies

Depends on	Used by
none	<code>mandate</code> , <code>server</code> , <code>verifier</code>

Nexusphere Ledger Mandate

Mandates and the formats two organizations exchange, with a verifier SDK.

Purpose

A grant lives inside one ledger. When an agent acts at another organization, that organization needs a signed, self-contained proof of what the agent may do, and a way to check it without trusting the sender. This library is that proof, its revocation list and the verifier the receiving side uses.

Responsibilities

- Compact JWS signing and parsing with EdDSA (`Jws`) and Ed25519 keys as JWK (`Jwk`)
- `MandateClaims` and `Mandates` : the mandate issued from a grant as an SD-JWT VC, and `SdJwt` to parse it, check its disclosures and present it with fewer of them
- `StatusList` : the signed revocation list in the IETF Token Status List format
- `MandateVerifier` : trusted issuers, keys from the issuer's JWKS, signature, time, audience, coverage and revocation, failing closed
- `ExchangeRequest` and `ExchangeReceipt` : the signed request proof and receipt of an A2A exchange between two ledgers

Dependencies

Depends on	Used by
<code>chain</code>	<code>server</code> , <code>verifier</code>

Nexusphere Ledger Server

The ledger service: evidence, authorization and gateways for agents.

Purpose

This is the product that runs: a self-hosted service that records every agent action as tamper-evident evidence, decides whether an agent may act, and sits in front of MCP servers and A2A peers so nothing passes without a decision and a record.

Responsibilities

- Evidence API on an append-only SHA-256 hash chain in PostgreSQL, single or in batches, with signed checkpoints and full verification
- Agents with their own API keys, grants from principals to agents, and ALLOW or DENY decisions with outcomes
- MCP gateway that decides and records every tool call
- Evidence packages with selective disclosure for auditors
- Transparency log over the evidence with inclusion and consistency proofs, cosigned by witnesses, and a C2SP witness endpoint so the ledger can witness other logs
- SCITT signed statements and RFC 9942 receipts for every entry
- Signed mandates, the revocation status list and the public keys as JWKS
- Signing key history: rotation at startup with a cross-signed rotation record, retired keys kept for verification
- A2A gateway that sends and receives requests between two ledgers with evidence and signed receipts on both sides
- Principal sign-in with an OpenID Connect provider and principal consent on grants
- Health probes and Prometheus metrics for the chain head, checkpoints and recorded evidence

Dependencies

Depends on	Used by
chain , mandate , verifier	none

API

Path	Purpose
/api/v1/**	Evidence, checkpoints, log proofs, packages, agents, grants, decisions, mandates
/mcp/{server}	MCP gateway
/a2a/out/{peer} , /a2a/in/{agent}	A2A gateway

Path	Purpose
/public/v1/**	Public keys, status list, OIDC settings, log checkpoint and witness endpoint

Storage

Tables live in the `ledger` schema of PostgreSQL, created by the Flyway migrations in `src/main/resources/db/migration/ledger`.

Configuration, every endpoint and the formats are described in [Nexusphere Ledger](#).

Nexusphere Ledger Verifier

A command-line tool that checks ledger output without the ledger.

Purpose

Evidence is only worth something if someone else can check it. The verifier runs anywhere with Java and no server or database, so an auditor, a court or a counterparty can verify what a ledger handed them.

Responsibilities

- Verify an evidence package: checkpoint signatures, every hash link, and that each disclosed entry belongs to the chain
- Check the package's log checkpoint and the inclusion proof of every disclosed entry, and require witness cosignatures with `--witness` and `--witnesses-required`
- Check the SCITT statement and receipt of every disclosed entry, and a single statement and receipt with the `statement` command
- Verify a mandate with the `mandate` command: issuer, key, signature, time, audience, coverage and revocation
- Pin the ledger's public key with `--public-key`, print JSON with `--json`, and exit with 0 (valid), 1 (invalid) or 2 (usage error)
- Accept checkpoints signed by other keys of the ledger only when signed key rotations lead to them from the pinned key
- Run the [Conformance Vectors](#) in `ConformanceVectorsTest`

Dependencies

Depends on	Used by
<code>chain</code> , <code>mandate</code>	<code>server</code>

Usage is in [Evidence Packages](#), [SCITT Statements and Receipts](#) and [Verifying a Mandate](#).

Nexusphere Ledger Demo MCP

A small MCP server and a script that show the ledger end to end.

Purpose

The ledger is easiest to understand by watching it stop and record a real tool call. This module is a toy MCP server with harmless tools and the script that walks through the whole flow against the compose stack.

Responsibilities

- Serve the MCP tools `read_file`, `send_email` and `delete_file` over Streamable HTTP on port 8091
- `demo.sh` : register an agent, ask `alice` for a grant and approve it with a Keycloak sign-in, call tools through the gateway, see an allowed and a denied call, export a package and verify it, then show that a tampered package fails

Dependencies

Depends on	Used by
none	none

How to run it is in [Demo](#).

Nexusphere Ledger Conformance Vectors

Fixed inputs and outputs for every format the ledger produces.

Purpose

A verifier written in another language, or another ledger, can prove it reads and writes the same formats by running these vectors. The ledger runs them too: `ConformanceVectorsTest` in `ledger/verifier` rebuilds every vector and fails when its output differs, so a format never changes by accident.

Keys

The vectors are signed with the published RFC 8032 Ed25519 test keys: test 1 is the ledger key and test 2 the witness key. Ed25519 signatures are deterministic, so the same input always gives the same bytes. The keys are public; never use them for real evidence.

Vectors

File	Contents
<code>keys.json</code>	Seeds, PKCS#8 and X.509 keys, the key ID and the C2SP verifier keys of the log and the witness
<code>canonical-json.json</code>	An object and its canonical JSON with sorted keys, no whitespace and escaped text, and its SHA-256
<code>evidence-chain.json</code>	Three entries with canonical content, content hash and link hash, and a signed checkpoint of the head
<code>merkle-tree.json</code>	RFC 9162 roots, inclusion proofs and consistency proofs for every size up to 8
<code>signed-note.json</code>	The C2SP signed-note log checkpoint over the three entries and its <code>tlog-cosignature/v1</code> cosignature
<code>scitt.json</code>	COSE_Sign1 SCITT statements and RFC 9942 receipts for the three entries, in hex
<code>mandate.json</code>	An SD-JWT VC mandate, the same mandate presented with only the grant disclosed, and its disclosed claims

SD-JWT disclosures carry random salts, so `mandate.json` is checked rather than rebuilt: its signature, digests and claims must verify.

Checked Elsewhere

The vectors were also checked with other implementations: the Merkle roots, canonical JSON, content hashes and checkpoint signature with Python's `hashlib` and `cryptography`; the signed note with Go's `golang.org/x/mod/sumdb/note`; the cosignature, COSE signatures and SD-JWT digests with Python's `cryptography` and `cbor2`. The [Python SDK](#) and the [TypeScript SDK](#) rebuild every vector except the mandate in their own test suites and check the mandate.

Regenerate

Change a vector only together with a deliberate format change, and record it in the changelog.

```
mvn -pl ledger/verifier -am test -Dtest=ConformanceVectorsTest -Dconformance.write=true -  
Dsurefire.failIfNoSpecifiedTests=false
```

Nexusphere Ledger Changelog

1.0.0 (not released yet)

First release of the Nexusphere Ledger, a self-hosted evidence layer for AI agent actions that sits beside existing agent platforms and gateways.

- Evidence entries in a SHA-256 hash chain with append-only PostgreSQL storage and Ed25519-signed checkpoints
- Agents with their own API keys, grants from principals to agents, and ALLOW/DENY decisions with reason codes and outcome reports, all recorded as evidence
- MCP gateway that decides every `tools/call`, forwards only allowed calls and records their outcome
- Evidence packages with selective disclosure and the offline verifier command-line tool
- Signed mandates (EdDSA JWS) issued from grants, a signed revocation status list and the ledger keys as a JWK set on public endpoints, with issue and revoke recorded as evidence
- `MandateVerifier` SDK that checks a mandate's issuer, key, signature, time, audience, coverage and revocation, and the `mandate` command of the verifier jar
- A2A gateway between two ledgers: outbound requests are decided and carry a mandate and a signed request proof, inbound requests are verified, recorded and answered with a signed receipt, so both sides hold matching evidence
- Signing key rotation at startup with a rotation record signed by the old and the new key; retired keys stay published, so everything they signed still verifies, and the offline verifier follows rotations from a pinned key
- Grant uses are returned when a gateway sees that the call never ran, and a receiving ledger counts the uses of each mandate on its own
- Streaming in both gateways: MCP answers, server requests and the `GET` stream, and A2A `message/stream` and `tasks/resubscribe` are relayed live, with the A2A receipt sent as the last event and covering every event
- Principal login with any OpenID Connect provider: principals approve, deny, create and revoke their own grants, grants from the operator or an agent wait for that consent, and every consent is evidence tied to the sign-in
- Web UI (React and TypeScript) for operators and principals: evidence, agents, grants, mandates, packages, A2A exchanges, ledger verification, and principal sign-in with PKCE to approve, deny and revoke grants
- Evidence batches: up to 500 entries recorded in order in one call, all or none
- Python SDK (`nexusphere-ledger`): a client for evidence, decisions, grants, mandates and packages, and an offline verifier for packages, statements, receipts and mandates that passes every conformance vector, with the `nexusphere-ledger-verify` command
- TypeScript SDK (`@nexusphere/ledger`): the same client and offline verifier for Node 20 and browsers on Web Crypto, with no runtime dependencies and the same `nexusphere-ledger-verify` command
- OpenTelemetry Collector exporter `nexusphere`: agent tool call and agent invocation spans that follow the GenAI semantic conventions become evidence in batches, with retries and a

sending queue, plus a ready Collector image; MCP spans with a JSON-RPC error are recorded as `FAILED`

- Adapters that record or decide tool calls: a LangGraph callback handler and `govern` wrapper, OpenAI Agents SDK run hooks with a tool input guardrail, an Amazon Bedrock AgentCore Gateway interceptor for Lambda, and an agentgateway tracing configuration with an end-to-end check
- Browser verifier: one self-contained HTML page in React and TypeScript that verifies a package with the TypeScript SDK, pinned to the ledger key and optional witnesses, with a Content Security Policy that blocks every connection
- Secrets and signing keys from files written by a secret manager, Prometheus metrics with alert rules, and a tested backup and restore procedure
- Rate limit per caller with 429 and `Retry-After`, and alert delivery by email, Slack, Telegram or webhook, each turned on by setting its address
- Transparency log: every entry is a leaf of an RFC 9162 Merkle tree, log checkpoints are C2SP signed notes, and inclusion and consistency proofs are served by the API and carried in evidence packages
- Witness cosigning with the C2SP `tllog-witness` protocol: the ledger asks configured witnesses to cosign each log checkpoint, any ledger can witness other logs, and the verifier can require cosignatures from given witnesses
- IETF SCITT signed statements (COSE hash envelopes) for every entry and RFC 9942 receipts from the transparency log, served by the API, carried in evidence packages and checked by the verifier's `statement` command
- Mandates are IETF SD-JWT VCs (`dc+sd-jwt`): the principal, grant and terms hash are selectively disclosable, the limits are not, and the documentation maps them to AP2 and Verifiable Intent
- Conformance vectors for every format, signed with the RFC 8032 test keys and checked by a test
- Dockerfiles, compose file with Adminer, Keycloak, Prometheus, Alertmanager and the web UI, Kubernetes manifest, demo MCP server and demo script

PART 4

SDKs and Integrations

Client libraries and offline verifiers for other languages, checked against the same conformance vectors, the OpenTelemetry Collector exporter, the adapters for agent frameworks and gateways, and the browser verifier.

Nexusphere Ledger Python SDK

Record evidence from a Python agent and verify everything the ledger produces, offline.

Purpose

`nexusphere-ledger` lets a Python agent, a gateway hook or a platform record evidence in the Nexusphere Ledger, and lets an auditor or a counterparty check what a ledger handed them without Java and without the server. The verifier is a second implementation of the formats, not a wrapper: it passes every [Conformance Vector](#) and rebuilds the signed checkpoints, notes, cosignatures, COSE statements and receipts byte for byte. Its only dependency is `cryptography`.

Install

Step 1. Check Python; the version must be 3.10 or later:

```
python3 --version
```

Step 2. Install the SDK from the repository:

```
python3 -m pip install ledger/sdk/python
```

Step 3. Check the install; it must print `1.0.0`:

```
python3 -c "import nexusphere_ledger; print(nexusphere_ledger.__version__)"
```

Record Evidence

These steps need a running ledger, as in the [Quick Start](#), and an agent API key from `POST /api/v1/agents`. `input` and `output` are hashed on the client with SHA-256; only the hash reaches the ledger.

Step 1. Register an agent with the operator key and keep its API key:

```
from nexusphere_ledger import LedgerClient

operator = LedgerClient("http://localhost:8090", "nexusphere-ledger-development-key-change-me")
agent_key = operator.register_agent("invoice-agent", "Invoice agent", "acme")["apiKey"]
```

Step 2. Record what the agent did; the answer carries the `sequence` and `hash` of the new entry:

```
agent = LedgerClient("http://localhost:8090", agent_key)
entry = agent.record("invoice-agent", "acme", "tools/call", "SUCCEEDED", target="read_invoice",
                    decision="ALLOW", input={"invoice": 7}, output="invoice text")
print(entry["sequence"], entry["hash"])
```

Step 3. Record many entries in one call, all or none:

```
agent.record_batch([
    {"agentId": "invoice-agent", "principalId": "acme", "action": "tools/call", "outcome":
    "SUCCEEDED"},
    {"agentId": "invoice-agent", "principalId": "acme", "action": "tools/call", "outcome":
    "FAILED"},
])
```

Decide and Report

When the ledger decides, `act` asks for a decision, raises `Denied` when no grant covers the action, and reports the outcome when the block ends: `SUCCEEDED`, or `FAILED` with the exception name.

Step 1. Grant the agent the tools, as the operator:

```
from datetime import datetime, timedelta, timezone

operator.create_grant("alice", "invoice-agent", ["tools/call"], ["read_*"],
                    datetime.now(timezone.utc) + timedelta(days=30), max_uses=100)
```

Step 2. Act under the grant:

```
from nexusphere_ledger import Denied

try:
    with agent.act("alice", "tools/call", "read_invoice", input={"invoice": 7}) as action:
        text = "invoice text"
        action.output(text)
except Denied as denied:
    print(denied.decision["reasonCode"])
```

Step 3. Check the outcome; it must print `SUCCEEDED` :

```
print(action.reported["outcome"])
```

Verify a Package

Step 1. Export a package and read the ledger's public key, as the operator:

```
package = operator.export_package(agent_id="invoice-agent")
public_key = operator.active_public_key()
```

Step 2. Verify it pinned to that key; `valid` must be `True` :

```
from nexusphere_ledger import verify_package

report = verify_package(package, public_key)
print(report.valid, report.disclosed_entries, report.problems)
```

Pass witness verifier keys as the third argument and the number of cosignatures needed as the fourth, as with the Java verifier. A key in the package is trusted only when it is the pinned key or signed key rotations lead to it from the pinned key. `report.to_dict()` has the same fields as the Java `PackageReport`.

Verify a Statement and Receipt

```
from nexusphere_ledger import EvidenceStatement, LogReceipt, PublicKey

key = PublicKey.from_base64(public_key)
statement = EvidenceStatement.parse(agent.statement(entry["id"]))
receipt = LogReceipt.parse(agent.receipt(entry["id"]))
print(statement.verify(key), receipt.verify(statement.leaf_hash, key))
```

Verify a Mandate

`MandateVerifier` reads the issuer's keys from `/public/v1/keys` and its revocation status list, and caches both for five minutes. Problems carry the same codes as the Java SDK: `MALFORMED`, `WRONG_TYPE`, `UNTRUSTED_ISSUER`, `UNKNOWN_KEY`, `BAD_SIGNATURE`, `NOT_YET_VALID`, `EXPIRED`, `WRONG_AUDIENCE`, `NOT_COVERED`, `REVOKED` and `STATUS_UNAVAILABLE`.

```
from nexusphere_ledger import MandateVerifier

verifier = MandateVerifier("http://localhost:8090", audience="https://supplier.example")
check = verifier.verify_action(token, "a2a/send", "supplier/sales")
print(check.valid, check.claims.agent_id, [p.code for p in check.problems])
```

Command Line

The install adds `nexusphere-ledger-verify` with the same arguments, output and exit codes (0 valid, 1 invalid, 2 usage error) as the Java verifier jar.

Step 1. Verify a package pinned to the ledger's key; the last line must be `Result: VALID`:

```
nexusphere-ledger-verify --public-key "$(curl -s http://localhost:8090/api/v1/keys -H
'Authorization: Bearer nexusphere-ledger-development-key-change-me' | jq -r '.[0] | select(.status
== "ACTIVE") | .publicKey')" package.json
```

Step 2. Verify a statement with its receipt:

```
nexusphere-ledger-verify statement --public-key "{publicKey}" --receipt receipt.cose
statement.cose
```

Step 3. Verify a mandate for an action:

```
nexusphere-ledger-verify mandate --issuer http://localhost:8090 --action a2a/send --target
supplier/sales "{token}"
```

Reference

Module	Contents
<code>client</code>	<code>LedgerClient</code> for evidence, batches, decisions, <code>act</code> , grants, mandates, packages, keys, proofs
<code>package</code>	<code>verify_package</code> and <code>PackageReport</code>
<code>evidence</code>	Content and link hashes, <code>ChainVerifier</code> , <code>Checkpoint</code>

Module	Contents
merkle	RFC 9162 roots, inclusion and consistency proofs
note	C2SP signed notes, note keys and tlog-cosignature/v1 cosignatures
cose	CoseSign1 , SCITT EvidenceStatement and RFC 9942 LogReceipt
mandate	SD-JWT VC mandates, MandateVerifier , status lists and JWKs
keys	Ed25519 keys, key IDs, key rotations and trusted keys from a pinned key
canonical	Canonical JSON
cbor	Deterministic CBOR

Test

Step 1. Run the unit tests and the conformance vectors; every test must pass:

```
cd ledger/sdk/python
PYTHONPATH=src python3 -m unittest discover -s tests -t .
```

Step 2. Run the same tests against a running ledger, as in the [Quick Start](#); the three LiveLedger tests must pass instead of being skipped:

```
NEXUSPHERE_LEDGER_URL=http://localhost:8090 PYTHONPATH=src python3 -m unittest discover -s tests -t . -v
```

Nexusphere Ledger TypeScript SDK

Record evidence from a Node or browser agent and verify everything the ledger produces, offline.

Purpose

`@nexusphere/ledger` is the TypeScript counterpart of the [Python SDK](#): a client for agents, gateways and platforms that record evidence, and an offline verifier for packages, SCITT statements, receipts and mandates. It has no runtime dependencies. Signatures and hashes use Web Crypto and status lists use `DecompressionStream`, so the same code runs in Node 20 or later and in current browsers. It passes every [Conformance Vector](#) byte for byte. Every call that hashes or checks a signature returns a promise.

Install

Step 1. Check Node; the version must be 20 or later:

```
node --version
```

Step 2. Build the SDK and add it to your project:

```
cd ledger/sdk/typescript
npm install
npm run build
cd -
npm install ../ledger/sdk/typescript
```

Step 3. Check the install; it must print `1.0.0`:

```
node --input-type=module -e "import { VERSION } from '@nexusphere/ledger'; console.log(VERSION)"
```

Record Evidence

These steps need a running ledger, as in the [Quick Start](#). `input` and `output` are hashed on the client with SHA-256; only the hash reaches the ledger.

Step 1. Register an agent with the operator key and keep its API key:

```
import { LedgerClient } from "@nexusphere/ledger";

const operator = new LedgerClient("http://localhost:8090", { apiKey: "nexusphere-ledger-development-key-change-me" });
const { apiKey } = await operator.registerAgent("invoice-agent", "Invoice agent", "acme");
```

Step 2. Record what the agent did; the answer carries the `sequence` and `hash` of the new entry:

```
const agent = new LedgerClient("http://localhost:8090", { apiKey: apiKey as string });
const entry = await agent.record({
  agentId: "invoice-agent", principalId: "acme", action: "tools/call", outcome: "SUCCEEDED",
  target: "read_invoice", decision: "ALLOW", input: { invoice: 7 }, output: "invoice text",
});
console.log(entry.sequence, entry.hash);
```

Step 3. Record many entries in one call, all or none:

```
await agent.recordBatch([
  { agentId: "invoice-agent", principalId: "acme", action: "tools/call", outcome: "SUCCEEDED" },
  { agentId: "invoice-agent", principalId: "acme", action: "tools/call", outcome: "FAILED" },
]);
```

Decide and Report

`act` asks for a decision, throws `Denied` when no grant covers the action, runs the work, and reports `SUCCEEDED`, or `FAILED` with the error name when the work throws.

Step 1. Grant the agent the tools, as the operator:

```
await operator.createGrant({
  principalId: "alice", agentId: "invoice-agent", actions: ["tools/call"], targets: ["read_*"],
  expiresAt: new Date(Date.now() + 30 * 86_400_000), maxUses: 100,
});
```

Step 2. Act under the grant; it must print `SUCCEEDED` :

```
import { Denied } from "@nexusphere/ledger";

try {
  await agent.act({ principalId: "alice", action: "tools/call", target: "read_invoice", input: {
    invoice: 7 } },
    async (action) => {
      await action.output("invoice text");
      console.log("running");
    });
} catch (e) {
  if (e instanceof Denied) console.log(e.decision.reasonCode);
}
```

Verify a Package

Step 1. Export a package and read the ledger's public key, as the operator:

```
const pkg = await operator.exportPackage({ agentId: "invoice-agent" });
const publicKey = await operator.activePublicKey();
```

Step 2. Verify it pinned to that key; `valid` must be `true` :

```
import { verifyPackage } from "@nexusphere/ledger";

const report = await verifyPackage(pkg, publicKey);
console.log(report.valid, report.disclosedEntries, report.problems);
```

Pass `{ witnesses: [verifierKey], requiredWitnesses: 1 }` as the third argument to require witness cosignatures. The report has the same fields as the Java `PackageReport`.

Verify a Statement and Receipt

```
import { EvidenceStatement, LogReceipt, PublicKey } from "@nexusphere/ledger";

const key = await PublicKey.fromBase64(publicKey);
const statement = EvidenceStatement.parse(await agent.statement(entry.id as string));
const receipt = LogReceipt.parse(await agent.receipt(entry.id as string));
console.log(await statement.verify(key), await receipt.verify(await statement.leafHash(), key));
```

Verify a Mandate

`MandateVerifier` reads the issuer's keys from `/public/v1/keys` and its revocation status list, caches both for five minutes, and reports the same problem codes as the Java and Python SDKs.

```
import { MandateVerifier } from "@nexusphere/ledger";

const verifier = new MandateVerifier("http://localhost:8090", { audience:
  "https://supplier.example" });
const check = await verifier.verifyAction(token, "a2a/send", "supplier/sales");
console.log(check.valid, check.claims?.agentId, check.problems.map((p) => p.code));
```

Command Line

The package adds `nexusphere-ledger-verify` with the same arguments, output and exit codes (0 valid, 1 invalid, 2 usage error) as the Java verifier jar and the Python SDK.

Step 1. Verify a package pinned to the ledger's key; the last line must be `Result: VALID`:

```
npx nexusphere-ledger-verify --public-key "$(curl -s http://localhost:8090/api/v1/keys -H
'Authorization: Bearer nexusphere-ledger-development-key-change-me' | jq -r '.[0] | select(.status
== "ACTIVE") | .publicKey')" package.json
```

Step 2. Verify a statement with its receipt, and a mandate for an action:

```
npx nexusphere-ledger-verify statement --public-key "{publicKey}" --receipt receipt.cose
statement.cose
npx nexusphere-ledger-verify mandate --issuer http://localhost:8090 --action a2a/send --target
supplier/sales "{token}"
```

Reference

Module	Contents
client	LedgerClient for evidence, batches, decisions, act, grants, mandates, packages, keys, proofs
package	verifyPackage and PackageReport
evidence	Content and link hashes, ChainVerifier, Checkpoint
merkle	RFC 9162 roots, inclusion and consistency proofs, exported as merkle

Module	Contents
note	C2SP signed notes, note keys and tlog-cosignature/v1 cosignatures
cose	CoseSign1, SCITT EvidenceStatement and RFC 9942 LogReceipt
mandate	SD-JWT VC mandates, MandateVerifier, status lists and JWKs
keys	Ed25519 keys, key IDs, key rotations and trusted keys from a pinned key
canonical	Canonical JSON and SHA-256
cbor	Deterministic CBOR, exported as cbor

Test

Step 1. Install the tools, type-check and run the unit tests and the conformance vectors; every test must pass:

```
cd ledger/sdk/typescript
npm install
npm run lint
npm test
```

Step 2. Run the same tests against a running ledger, as in the [Quick Start](#); the three tests of a running ledger must pass instead of being skipped:

```
NEXUSPHERE_LEDGER_URL=http://localhost:8090 npm test
```

Nexusphere Ledger OpenTelemetry Exporter

Turn the GenAI spans agents already emit into ledger evidence, through the OpenTelemetry Collector.

Purpose

Most agent frameworks and gateways already emit OpenTelemetry spans that follow the GenAI and MCP semantic conventions. The `nexusphere` exporter is a Collector component that picks the spans of tool calls and agent invocations and records each one as evidence through `POST /api/v1/evidence/batch`. An agent needs no code change and no SDK: point its OTLP exporter at a Collector that has this exporter in its traces pipeline. The resulting entries get the same hash chain, transparency log, witnesses, statements and packages as any other evidence. A ready configuration for agentgateway is in [Adapters](#).

Tool arguments and results never reach the ledger. When a span carries `gen_ai.tool.call.arguments` or `gen_ai.tool.call.result`, the exporter sends only their SHA-256 as `inputHash` and `outputHash`, so the ledger can later prove which input and output belonged to the call without holding them.

Mapping

Evidence field	Taken from
selected spans	<code>gen_ai.operation.name</code> or <code>mcp.method.name</code> listed in <code>operations</code> (default <code>execute_tool</code> , <code>invoke_agent</code> , <code>tools/call</code>)
agentId	The first of <code>agent_attributes</code> (default <code>gen_ai.agent.id</code> , <code>gen_ai.agent.name</code>), else <code>default_agent</code>
principalId	The first of <code>principal_attributes</code> (default <code>nexusphere.principal.id</code> , <code>user.id</code> , <code>enduser.id</code>), else <code>default_principal</code>
action	<code>mcp.method.name</code> ; <code>tools/call</code> for <code>execute_tool</code> ; <code>agents/invoke</code> for <code>invoke_agent</code> ; else <code>gen_ai/{operation}</code>
target	<code>gen_ai.tool.name</code> ; the invoked <code>gen_ai.agent.name</code> for <code>invoke_agent</code> ; else the span name
occurredAt	The span start time
decision	<code>nexusphere.decision</code> (<code>ALLOW</code> or <code>DENY</code>) when the gateway sets it
outcome	<code>DENIED</code> after a <code>DENY</code> , <code>FAILED</code> when the span status is an error or <code>mcp.error.code</code> is set, else <code>SUCCEEDED</code>
reason	<code>nexusphere.reason</code> , else <code>error.type</code> or the status message of a failed span
delegationId	<code>nexusphere.delegation.id</code>
inputHash	<code>nexusphere.input.hash</code> , else the SHA-256 of <code>gen_ai.tool.call.arguments</code>
outputHash	<code>nexusphere.output.hash</code> , else the SHA-256 of <code>gen_ai.tool.call.result</code>
correlationId	<code>gen_ai.conversation.id</code> , else the trace ID

Evidence field	Taken from
attributes	otel.trace_id , otel.span_id , otel.span_name , otel.duration_ms , the model, provider, tool call ID, mcp.session.id , mcp.target and service.name , and the keys in attributes

Span attributes win over resource attributes. A selected span without an agent or a principal is skipped and logged at debug level. Text is cut to the ledger's limits and control characters are replaced, so one odd span never makes the ledger refuse a whole batch. A batch the ledger still refuses (HTTP 4xx) is dropped and logged; network errors, 429 and 5xx are retried with the Collector's retry and queue settings. A batch that reached the ledger but whose answer was lost is sent again, so the same span can be recorded twice; `otel.span_id` tells such entries apart.

Configuration

Setting	Default	Meaning
endpoint	none, required	Base URL of the ledger, such as <code>http://localhost:8090</code>
api_key	none, required	An agent API key, which records only that agent, or the operator key
operations	execute_tool , invoke_agent , tools/call	Values of <code>gen_ai.operation.name</code> or <code>mcp.method.name</code> that become evidence
agent_attributes	gen_ai.agent.id , gen_ai.agent.name	Attributes that name the agent, first match wins
default_agent	empty	Agent for spans without one
principal_attributes	nexusphere.principal.id , user.id , enduser.id	Attributes that name the principal, first match wins
default_principal	empty	Principal for spans without one
attributes	empty	More span or resource attributes to copy into the evidence attributes
hash_content	true	Hash tool arguments and results into <code>inputHash</code> and <code>outputHash</code>
batch_size	500	Entries per request, at most 500
timeout , headers , tls , sending_queue , retry_on_failure	Collector defaults, 30 s timeout	The standard Collector HTTP client, queue and retry settings

```
exporters:
  nexusphere:
    endpoint: http://localhost:8090
    api_key: ${env:NEXUSPHERE_LEDGER_API_KEY}
    default_principal: acme

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [batch]
      exporters: [nexusphere]
```

Run with Docker

The compose file of the ledger starts the Collector as `otel-collector` with the operator key of the `dev` profile, and `ledger/kube-dev.yaml` runs it in the `dev` namespace. Agents send OTLP to it on `4317` (gRPC) or `4318` (HTTP).

Step 1. Start the ledger as in the [Quick Start](#); `otel-collector` starts with it.

Step 2. Send one tool call span over OTLP/HTTP; the answer must be `{"partialSuccess":{}}` :

```
curl -X POST http://localhost:4318/v1/traces -H "Content-Type: application/json" -d
'{"resourceSpans":[{"resource":{"attributes":[{"key":"service.name","value":
{"stringValue":"invoice-service"}}]}],"scopeSpans":[{"spans":
[{"traceId":"5b8efff798038103d269b633813fc60c","spanId":"eee19b7ec3c1b174","name":"execute_tool
read_invoice","kind":1,"startTimeUnixNano":"'$(date +%s)000000000'", "endTimeUnixNano":"'$(date
+%s)1000000000'", "attributes":[{"key":"gen_ai.operation.name","value":
{"stringValue":"execute_tool"}},{ "key":"gen_ai.tool.name","value":{"stringValue":"read_invoice"}},
{"key":"gen_ai.agent.id","value":{"stringValue":"otel-agent"}},{ "key":"user.id","value":
{"stringValue":"alice"}}]}]}]}'
```

Step 3. Check the evidence; one entry with `tools/call` on `read_invoice` for `alice` must appear:

```
curl -s "http://localhost:8090/api/v1/evidence?agentId=otel-agent" -H "Authorization: Bearer
nexusphere-ledger-development-key-change-me" | jq '.items[] | {action, target, principalId,
outcome}'
```

Build the Collector

`collector/builder-config.yaml` describes a Collector distribution, `nexusphere-otelcol`, with the OTLP receiver, the batch processor, the debug exporter and this exporter, built with the OpenTelemetry Collector Builder.

Step 1. Check Go; the version must be 1.26 or later:

```
go version
```

Step 2. Install the builder:

```
go install go.opentelemetry.io/collector/cmd/builder@v0.162.0
```

Step 3. Build the distribution; the last line names `./_build/nexusphere-otelcol` :

```
cd ledger/otel-exporter/collector
$(go env GOPATH)/bin/builder --config builder-config.yaml
```

Step 4. Run it against a ledger with an agent key; it must log `Everything is ready` :

```
NEXUSPHERE_LEDGER_URL=http://localhost:8090 NEXUSPHERE_LEDGER_API_KEY={agentApiKey}
NEXUSPHERE_DEFAULT_PRINCIPAL=acme ./_build/nexusphere-otelcol --config config.yaml
```

Add It to Your Collector

A Collector built with the builder takes the exporter as one more entry under `exporters` in its own builder configuration; the module path is `github.com/oss-community/nexusphere/ledger/otel-exporter` and its package name is `nexusphereexporter`.

```
exporters:  
- gomod: github.com/oss-community/nexusphere/ledger/otel-exporter v1.0.0  
  name: nexusphereexporter
```

Test

Step 1. Run the unit tests; they must end with `ok`:

```
cd ledger/otel-exporter  
go test ./...
```

Step 2. Run them against a running ledger; `TestLiveLedger` must pass instead of being skipped:

```
NEXUSPHERE_LEDGER_URL=http://localhost:8090 go test -count=1 -v -run TestLiveLedger ./...
```

Nexusphere Ledger Adapters

Ready-made connections that put the tool calls of existing agent frameworks and gateways into the ledger.

Purpose

An adapter sits beside an agent framework or a gateway that already runs the tool calls and turns each call into an evidence entry, so nothing about the agent has to change but one line of setup. Every adapter records the same shape of entry as the ledger's own MCP gateway: action `tools/call`, the tool name as the target, the agent, the principal, SHA-256 hashes of the arguments and the result, and the outcome.

Adapters

Adapter	Connects	How
LangGraph	LangGraph and LangChain agents	A callback handler, or tools wrapped with <code>govern</code>
OpenAI Agents SDK	OpenAI Agents SDK agents	Run hooks, and a tool input guardrail to decide
AgentCore	Amazon Bedrock AgentCore Gateway	A Lambda function as REQUEST and RESPONSE interceptor
agentgateway	agentgateway	Its OpenTelemetry traces through the Collector exporter

Record or Decide

Each adapter can only record what already happened, which leaves the decision with the framework or gateway, or ask the ledger before the tool runs, which makes the ledger's grants the authority. agentgateway only records; its own authorization policies decide.

Mode	Before the tool runs	After the tool runs	A denied call
Record	Nothing	One entry with the outcome	Not possible
Decide	POST <code>/api/v1/decisions</code> against the grants	POST <code>/api/v1/decisions/{id}/outcome</code>	Never runs; entry <code>DENIED</code>

What an Entry Holds

Field	LangGraph	OpenAI Agents SDK	AgentCore	agentgateway
<code>agentId</code>	Set in the adapter	Set in the adapter	JWT claim or a fixed id	Request header or JWT claim
<code>principalId</code>	Run metadata or a default	Run context or a default	JWT claim or a default	Request header or JWT claim

Field	LangGraph	OpenAI Agents SDK	AgentCore	agentgateway
target	Tool name	Tool name	Tool name with its target	gen_ai.tool.name
correlationId	LangGraph thread_id	Agents SDK trace id	Mcp-Session-Id	Trace id
inputHash	Tool arguments	Tool arguments	Tool arguments	Not available in the span
outputHash	Tool result	Tool result	MCP result	Not available in the span

Test

Step 1. Start the ledger as in the [Quick Start](#).

Step 2. Run the tests of each adapter as its README describes; every Python adapter has a live test that runs when `NEXUSPHERE_LEDGER_URL` is set, and agentgateway has `e2e.sh` . Each must end with `OK` or `Result: PASSED` .

Nexusphere Ledger for LangGraph

Every tool call of a LangGraph or LangChain agent as evidence in the Nexusphere Ledger.

Purpose

`nexusphere-langgraph` connects the ledger to LangGraph without changing the graph.

`NexusphereCallbackHandler` records each tool call after it ran, with its outcome. `govern` wraps the tools so the ledger decides each call against the principal's grants first, and a denied call never runs: the model gets the denial as the tool's error message. Use one of the two for a tool, not both, or the call is recorded twice. Both work with `ToolNode`, `create_react_agent` and any LangChain tool, synchronous or asynchronous.

Install

Step 1. Install the SDK and the adapter from the repository; Python 3.10 or later is needed:

```
python3 -m pip install ledger/sdk/python ledger/adapters/langgraph
```

Step 2. Check the install; it must print `1.0.0`:

```
python3 -c "import nexusphere_langgraph; print(nexusphere_langgraph.__version__)"
```

Record Tool Calls

These steps need a running ledger, as in the [Quick Start](#), and an agent registered with `POST /api/v1/agents`, as in the [Python SDK](#).

Step 1. Create the handler with the agent's own key:

```
from nexusphere_ledger import LedgerClient
from nexusphere_langgraph import NexusphereCallbackHandler

ledger = LedgerClient("http://localhost:8090", agent_key)
handler = NexusphereCallbackHandler(ledger, "invoice-agent")
```

Step 2. Pass it with every run, and name the principal in the run metadata:

```
graph.invoke({"messages": [{"user", "Read invoice 7"}]},
             {"callbacks": [handler], "metadata": {"nexusphere_principal": "alice"},
              "configurable": {"thread_id": "conversation-42"}})
```

Step 3. Check the evidence; one `tools/call` entry per tool call must appear, with `SUCCEEDED` or `FAILED`:

```
curl -s "http://localhost:8090/api/v1/evidence?agentId=invoice-agent" -H "Authorization: Bearer nexusphere-ledger-development-key-change-me" | jq '.items[] | {action, target, principalId, outcome, correlationId}'
```

A failure to reach the ledger is logged and does not stop the agent. With `strict=True` the tool call fails instead, so no call runs without its evidence.

Let the Ledger Decide

Step 1. Create a grant for the agent, as in the [Ledger Guide](#).

Step 2. Wrap the tools before building the graph:

```
from langgraph.prebuilt import ToolNode
from nexusphere_langgraph import govern

tools = govern([read_invoice, send_email], ledger, principal_key="nexusphere_principal")
tool_node = ToolNode(tools)
```

Step 3. Run the graph with the principal in the metadata or the `configurable` part of the config:

```
graph.invoke(inputs, {"configurable": {"nexusphere_principal": "alice", "thread_id": "conversation-42"}})
```

Step 4. Check the evidence; an allowed call shows `ALLOW` with its outcome, and a call outside the grant shows `DENY` with `DENIED`, while the model reads `Denied by the Nexusphere Ledger: NOT_COVERED`.

Reference

Name	Purpose
<code>NexusphereCallbackHandler(client, agent_id, principal_id=None, principal_key="nexusphere_principal", attributes=None, strict=False)</code>	Records each tool call after it ran
<code>govern(tools, client, agent_id=None, principal_id=None, principal_key="nexusphere_principal", attributes=None)</code>	Returns tools whose calls the ledger decides first
<code>principal_id</code>	The principal when the run names none
<code>principal_key</code>	The key in the run metadata or <code>configurable</code> that names the principal
<code>attributes</code>	Extra evidence attributes; <code>framework</code> is always <code>langgraph</code>

The input hash is the SHA-256 of the tool arguments as canonical JSON, the output hash that of the result text, and the correlation id the LangGraph `thread_id`. A `ToolMessage` with status `error` is recorded as `FAILED`.

Test

Step 1. Install the adapter with LangGraph:

```
python3 -m pip install ./ledger/sdk/python "./ledger/adapters/langgraph[test]"
```

Step 2. Run the tests; they must end with `OK` :

```
cd ledger/adapters/langgraph && python3 -m unittest -v
```

Step 3. Run them against a running ledger; the two `LiveLedger` tests must pass instead of being skipped:

```
cd ledger/adapters/langgraph && NEXUSPHERE_LEDGER_URL=http://localhost:8090 python3 -m unittest -v tests.test_live_ledger
```

Nexusphere Ledger for the OpenAI Agents SDK

Every tool call of an OpenAI Agents SDK agent as evidence in the Nexusphere Ledger.

Purpose

`nexusphere-openai-agents` connects the ledger to the OpenAI Agents SDK through its own extension points. `NexusphereHooks` is a set of run hooks that records each function tool call with its outcome. `hooks.govern(tools)` adds a tool input guardrail, so the ledger decides each call against the principal's grants before it runs; a denied call never runs and the model reads the denial instead of a result. The same hooks then report the outcome of the allowed calls.

Install

Step 1. Install the SDK and the adapter from the repository; Python 3.10 or later is needed:

```
python3 -m pip install ledger/sdk/python ledger/adapters/openai-agents
```

Step 2. Check the install; it must print `1.0.0` :

```
python3 -c "import nexusphere_openai_agents; print(nexusphere_openai_agents.__version__)"
```

Record Tool Calls

These steps need a running ledger, as in the [Quick Start](#), and an agent registered with `POST /api/v1/agents`, as in the [Python SDK](#).

Step 1. Create the hooks with the agent's own key, and let them see tool failures:

```
from agents import Agent, Runner
from nexusphere_ledger import LedgerClient
from nexusphere_openai_agents import NexusphereHooks

hooks = NexusphereHooks(LedgerClient("http://localhost:8090", agent_key), "invoice-agent")
agent = Agent(name="Invoices", instructions="Handle invoices.",
              tools=hooks.govern([read_invoice, send_email], decide=False))
```

Step 2. Run the agent with the hooks and the principal in the run context:

```
result = await Runner.run(agent, "Read invoice 7", context={"principal_id": "alice"}, hooks=hooks)
```

Step 3. Check the evidence; one `tools/call` entry per tool call must appear, with `SUCCEEDED` or `FAILED` :

```
curl -s "http://localhost:8090/api/v1/evidence?agentId=invoice-agent" -H "Authorization: Bearer nexusphere-ledger-development-key-change-me" | jq '.items[] | {action, target, principalId, outcome, correlationId}'
```

Without `govern` the hooks still record every call, but a tool that raised is recorded as `SUCCEEDED`, because the SDK hands the hooks the error message as an ordinary result. A failure to reach the ledger is logged and does not stop the agent; with `strict=True` the run fails instead.

Let the Ledger Decide

Step 1. Create a grant for the agent, as in the [Ledger Guide](#).

Step 2. Govern the tools; `decide` is on by default:

```
agent = Agent(name="Invoices", instructions="Handle invoices.", tools=hooks.govern([read_invoice, send_email]))
```

Step 3. Run the agent with the same hooks, as in [Record Tool Calls](#) Step 2.

Step 4. Check the evidence; an allowed call shows `ALLOW` with its outcome, and a call outside the grant shows `DENY` with `DENIED`, while the model reads `Denied by the Nexusphere Ledger: NOT_COVERED`.

Reference

Name	Purpose
<code>NexusphereHooks(client, agent_id=None, principal_id=None, principal=None, attributes=None, strict=False)</code>	Run hooks that record or report each tool call
<code>hooks.govern(tools, decide=True)</code>	Adds the ledger guardrail and failure tracking to function tools
<code>hooks.guardrail</code>	The tool input guardrail, for tools built by hand
<code>principal</code>	A function from the run context to the principal
<code>principal_id</code>	The principal when the context names none

Without `principal`, the principal is the `principal_id` key of a dictionary context or the `principal_id` attribute of any other context. The input hash is the SHA-256 of the tool arguments as canonical JSON, the output hash that of the result, and the correlation id the Agents SDK trace id.

Test

Step 1. Run the tests; they must end with `OK`:

```
cd ledger/adapters/openai-agents && python3 -m unittest -v
```

Step 2. Run them against a running ledger; the two `LiveLedger` tests must pass instead of being skipped:

```
cd ledger/adapters/openai-agents && NEXUSPHERE_LEDGER_URL=http://localhost:8090 python3 -m  
unittest -v tests.test_live_ledger
```

The tests run real agents on the SDK's own `ScriptedModel`, so no model and no OpenAI key are needed.

Nexusphere Ledger for Amazon Bedrock AgentCore

An AgentCore Gateway interceptor that records or decides every MCP tool call in the Nexusphere Ledger.

Purpose

`nexusphere-agentcore` is an AWS Lambda function for the REQUEST and RESPONSE interceptors of an Amazon Bedrock AgentCore Gateway with MCP targets. In `record` mode it lets every request through and records each `tools/call` with its outcome when the response passes. In `decide` mode the REQUEST interceptor asks the ledger first: a denied call is answered at once with a JSON-RPC error `-32003`, the same answer the ledger's own MCP gateway gives, and never reaches the target; an allowed call carries its decision id in `params._meta`, and the RESPONSE interceptor reports its outcome. Other MCP methods pass unchanged.

Build the Function

Step 1. Install the dependencies for the Lambda runtime into a build folder; the runtime here is Python 3.13 on x86_64:

```
python3 -m pip install --target build/agentcore --platform manylinux2014_x86_64 --python-version 3.13 --only-binary=:all: "cryptography>=42"
python3 -m pip install --target build/agentcore --no-deps ledger/sdk/python
ledger/adapters/agentcore
```

Step 2. Pack the folder:

```
cd build/agentcore && zip -qr ../nexusphere-agentcore.zip . && cd ../..
```

Step 3. Check the package; the list must contain `nexusphere_agentcore/interceptor.py`:

```
unzip -l build/nexusphere-agentcore.zip | grep nexusphere_agentcore/
```

Deploy

The ledger must be reachable from the Lambda function, for example through a VPC or a public HTTPS address. The commands use the AWS CLI; `{account}`, `{region}` and the role names are your own.

Step 1. Create the function with the handler `nexusphere_agentcore.lambda_handler`:

```
aws lambda create-function --function-name nexusphere-interceptor --runtime python3.13 --
architectures x86_64 --handler nexusphere_agentcore.lambda_handler --zip-file
fileb://build/nexusphere-agentcore.zip --role arn:aws:iam::{account}:role/nexusphere-interceptor-
role --timeout 10 --environment "Variables=
{NEXUSPHERE_LEDGER_URL=https://ledger.example.com,NEXUSPHERE_LEDGER_API_KEY=
{key},NEXUSPHERE_MODE=record}"
```

Step 2. Allow the gateway service role to invoke the function, as in the AgentCore guide on interceptor permissions.

Step 3. Add the function to the gateway as both interceptors, with the request headers, so the interceptor sees the caller's token:

```
aws bedrock-agentcore-control update-gateway --gateway-identifier {gatewayId} --name {gatewayName}
--role-arn arn:aws:iam::{account}:role/{gatewayRole} --protocol-type MCP --authorizer-type
CUSTOM_JWT --authorizer-configuration file://authorizer.json --interceptor-configurations
' [{"interceptor":{"lambda":{"arn":"arn:aws:lambda:{region}:{account}:function:nexusphere-
interceptor"}}, "interceptionPoints":["REQUEST","RESPONSE"],"inputConfiguration":
{"passRequestHeaders":true}} ]'
```

Step 4. Call a tool through the gateway, then check the evidence; one `tools/call` entry with the tool name as target, such as `invoices__read_invoice`, must appear:

```
curl -s "https://ledger.example.com/api/v1/evidence?agentId={agentId}" -H "Authorization: Bearer
{operatorKey}" | jq '.items[] | {action, target, principalId, outcome, correlationId}'
```

Settings

Variable	Default	Meaning
NEXUSPHERE_LEDGER_URL	(required)	Address of the ledger
NEXUSPHERE_LEDGER_API_KEY	(required)	The operator key for many agents, or one agent's key with NEXUSPHERE_AGENT_ID
NEXUSPHERE_MODE	record	record , or decide to let the ledger allow or deny each call
NEXUSPHERE_AGENT_ID	(none)	A fixed agent id for every call
NEXUSPHERE_AGENT_CLAIM	client_id	The token claim that names the agent when no fixed id is set
NEXUSPHERE_PRINCIPAL_CLAIM	sub	The token claim that names the principal
NEXUSPHERE_DEFAULT_PRINCIPAL	(none)	The principal when the token has none

Keep the key out of plain environment variables in production: encrypt it with a customer managed KMS key, or read it from AWS Secrets Manager through the Lambda extension and set the variable at cold start.

Identity

The gateway has already validated the caller's JWT before the interceptor runs, so the interceptor reads its claims without checking the signature again. A call without an agent or a principal is not recorded in `record` mode and is denied with `UNKNOWN_IDENTITY` in `decide` mode. When the ledger cannot be reached, `record` mode logs the error and lets the call through, while `decide` mode denies it with `LEDGER_UNAVAILABLE`.

Evidence field	Source
agentId	NEXUSPHERE_AGENT_ID , or the agent claim of the token
principalId	The principal claim of the token, or the default principal
target	params.name of the tools/call
inputHash	SHA-256 of params.arguments as canonical JSON
outputHash	SHA-256 of the JSON-RPC result as canonical JSON
outcome	FAILED for a JSON-RPC error, isError or an HTTP error
correlationId	The Mcp-Session-Id header

Test

The tests feed the function the interceptor events in the shapes the AgentCore documentation specifies; they do not call AWS.

Step 1. Run the tests; they must end with OK :

```
cd ledger/adapters/agentcore && python3 -m unittest -v
```

Step 2. Run them against a running ledger; the two LiveLedger tests must pass instead of being skipped:

```
cd ledger/adapters/agentcore && NEXUSPHERE_LEDGER_URL=http://localhost:8090 python3 -m unittest -v tests.test_live_ledger
```

Nexusphere Ledger for agentgateway

Every MCP tool call that passes agentgateway as evidence in the Nexusphere Ledger.

Purpose

agentgateway already traces every MCP request with OpenTelemetry, including `mcp.method.name`, `gen_ai.tool.name` and the JSON-RPC error of a failed call. `config.yaml` sends those traces to the Collector of the [OpenTelemetry Exporter](#), which turns each `tools/call` span into an evidence entry. Two trace fields name the agent and the principal; here they come from request headers. agentgateway keeps deciding with its own authorization policies; the ledger records.

Run

Step 1. Start the ledger and the Collector as in the [Quick Start](#); the compose file runs the Collector as `otel-collector` on 4317 and the demo MCP server on 8091.

Step 2. Install agentgateway as its [documentation](#) describes, and check it; the command must print the version:

```
agentgateway --version
```

Step 3. Start agentgateway with the configuration; the log must end with `started bind :`

```
agentgateway -f ledger/adapters/agentgateway/config.yaml
```

Step 4. Run the end-to-end check in another terminal; it must end with `Result: PASSED :`

```
ledger/adapters/agentgateway/e2e.sh
```

To proxy your own MCP servers, replace the `demo` target in `config.yaml`; the `config.tracing` part stays as it is. On a host without IPv6, start agentgateway with `IPV6_ENABLED=false`.

Identity from a Token

With `mcpAuthentication` on the route, agentgateway validates the caller's JWT, and the trace fields can name the agent and the principal from its claims instead of headers:

```
config:
  tracing:
    otlpEndpoint: http://localhost:4317
    randomSampling: true
  fields:
    add:
      gen_ai.agent.id: jwt.azp
      nexusphere.principal.id: jwt.sub
```

What Is Recorded

Evidence field	From the agentgateway span
agentId	gen_ai.agent.id , set by the trace field
principalId	nexusphere.principal.id , set by the trace field
target	gen_ai.tool.name
outcome	FAILED with JSON-RPC {code} {message} when mcp.error.code is set
correlationId	The trace id
attributes	mcp.target , mcp.session.id , service.name and the span ids

agentgateway does not put tool arguments or results into its spans, so the entries carry no input or output hash, and a tool result with `isError` is recorded as `SUCCEEDED` . Where those matter, use the ledger's own MCP gateway or one of the framework adapters.

Test

`e2e.sh` opens an MCP session through agentgateway as a new agent for `alice` , calls `read_file` and a tool that does not exist, waits for the Collector, and checks that the ledger holds a `SUCCEEDED` and a `FAILED` entry. It reads `AGENTGATEWAY_URL` , `NEXUSPHERE_LEDGER_URL` and `NEXUSPHERE_LEDGER_API_KEY` , and defaults to the local setup above.

Nexusphere Package Verifier for the Browser

One HTML file that verifies an evidence package in the browser, with no server and no network.

Purpose

An auditor or a counterparty who receives an evidence package should not need Java, a command line or the ledger to check it. The verifier is a React and TypeScript page that runs the [TypeScript SDK](#) verifier on Web Crypto. The build makes one self-contained `index.html` with the script and styles inline. Its Content Security Policy forbids every connection (`connect-src 'none'`), so the browser itself guarantees that the package and the keys never leave the page. The file can be mailed, put on a USB stick or opened from disk.

Build

Step 1. Check Node; the version must be 20 or later:

```
node --version
```

Step 2. Install the dependencies:

```
cd ledger/verifier-web && npm ci
```

Step 3. Build the page:

```
npm run build
```

Step 4. Check the result; `dist` must hold only `index.html` :

```
ls dist
```

Verify a Package

Step 1. Export a package and read the ledger's public key, as the operator:

```
curl -s -X POST http://localhost:8090/api/v1/packages -H "Authorization: Bearer nexusphere-ledger-development-key-change-me" -H "Content-Type: application/json" -d '{"agentId":"invoice-agent"}' > package.json
curl -s http://localhost:8090/api/v1/keys -H "Authorization: Bearer nexusphere-ledger-development-key-change-me" | jq -r '[] | select(.status == "ACTIVE") | .publicKey'
```

Step 2. Open `ledger/verifier-web/dist/index.html` in a browser, from disk or any web server.

Step 3. Choose or drop `package.json` , paste the public key, and add witness keys if the log must carry their cosignatures.

Step 4. Press Verify; the page must show **VALID** with the signing key, the checkpoint, the log, the proofs and the disclosed entries. Change one character of an entry in the file and verify again; the page must show **INVALID** and name the entry.

Without a public key, the page checks the package only against the keys it carries and says so: anyone can build such a package, so pin the ledger's key whenever the result matters. The page needs a browser with Ed25519 in Web Crypto: Chrome or Edge 137, Firefox 129, Safari 17 or later.

What It Checks

Check	Shown as
Package format and the signature of the checkpoint	Signing key and checkpoint
The pinned key, or a key the pinned key endorsed by rotation	Pinned key
Every link of the hash chain from the anchor to the checkpoint	Checked links
Disclosed entries against their content hash and the scope	Disclosed entries and scope
Merkle inclusion proofs and SCITT receipts of disclosed entries	Inclusion proofs and receipts
Witness cosignatures on the log checkpoint	Witnesses

The checks are the same as those of the [Offline Verifier](#), so a package that is valid on the command line is valid here.

Develop

Step 1. Start the development server; the page opens on <http://localhost:5174>:

```
cd ledger/verifier-web && npm run dev
```

Test

Step 1. Run the unit tests; they must all pass:

```
cd ledger/verifier-web && npm test
```

Step 2. Build the page and run the browser test, which opens the built file from disk, verifies a package and checks that no request leaves the page; it must report **1 passed** :

```
npm run build && npx playwright install chromium && npm run e2e
```

When Chromium is already installed elsewhere, skip `playwright install` and set `CHROMIUM_PATH` to its executable.

PART 5

Web UI

The React and TypeScript interface for operators and principals.

Nexusphere Frontend

The web UI of Nexusphere, built with React and TypeScript.

Purpose

The ledger's API is enough for agents and scripts, but operators and principals need to see evidence, grants and consent without curl. This module is a single-page application that talks to the Nexusphere Ledger API through the same origin, so it needs no CORS on the ledger.

Responsibilities

- Operator pages: the ledger head, checkpoints and signing keys, full verification, evidence with filters and entry details, agents, grants with their mandates, evidence package export and verification, and A2A exchanges
- Principal pages: sign-in with the ledger's OpenID Connect provider (authorization code with PKCE), the grants waiting for approval, approve, deny, revoke and create grants, and the evidence about the principal

Dependencies

Depends on	Used by
the Nexusphere Ledger API (<code>/api</code> , <code>/public</code>)	none

Getting Started

Prerequisites

- [Node.js 22](#)

Development

Step 1. Start the ledger, for example with the [Quick Start](#), and stop its `ledger-ui` container so port 5173 is free:

```
docker stop ledger-ui
```

Step 2. Install the dependencies:

```
cd frontend
npm install
```

Step 3. Start the development server:

```
npm run dev
```

Step 4. Open `http://localhost:5173` and sign in with the operator key `nexusphere-ledger-development-key-change-me`.

The development server forwards `/api` and `/public` to the ledger at `http://localhost:8090`, or to `LEDGER_URL` when it is set.

Test

```
npm test
```

Build

```
npm run build
```

Dockerized

The image builds the application and serves it with nginx on port 80, forwarding `/api` and `/public` to `LEDGER_URL` (default `http://ledger:8090`). The ledger compose file runs it as `ledger-ui` on `http://localhost:5173`.

```
docker build -t samanalishiri/nexusphere-frontend:latest frontend
docker run -p 5173:80 -e LEDGER_URL=http://host.docker.internal:8090 samanalishiri/nexusphere-frontend:latest
```

Sign-in

The operator signs in with the ledger API key; the key stays in the browser's session storage until sign-out or the tab closes, and an agent key is refused. A principal signs in with the provider the ledger trusts: the UI reads the issuer and client ID from `/public/v1/oidc` and uses the authorization code flow with PKCE, so the provider needs a public client that allows `http://localhost:5173/*` as redirect URI. The Keycloak realm in the ledger compose file has one, `nexusphere-ledger`, with the principals `alice` and `bob`.

PART 6

Operations

Running Nexusphere in production and through the delivery pipeline.

Operations

Secrets, signing keys, backup and restore, monitoring, alerts and rate limits for the ledger.

Secrets

The ledger reads secrets from environment variables or from files. Every file in `LEDGER_SECRETS_DIR`, `/run/secrets/` by default, becomes a setting named like the file, so a file `LEDGER_API_KEY` holding the key works exactly like the environment variable. A missing directory is ignored. This is how a secret manager hands secrets over without putting them in the environment: Docker secrets, Kubernetes secrets mounted as a volume, the Vault agent, the External Secrets Operator or the Secrets Store CSI driver all write files.

Secret file	Holds
<code>LEDGER_API_KEY</code>	The operator key
<code>LEDGER_SIGNING_PRIVATE_KEY</code>	The base64 PKCS#8 Ed25519 signing key
<code>LEDGER_SIGNING_PUBLIC_KEY</code>	The base64 X.509 public key
<code>LEDGER_DATABASE_PASSWORD</code>	The PostgreSQL password

Docker Compose

```
services:
  ledger:
    secrets:
      - LEDGER_API_KEY
      - LEDGER_SIGNING_PRIVATE_KEY
      - LEDGER_SIGNING_PUBLIC_KEY
secrets:
  LEDGER_API_KEY:
    file: ./secrets/ledger-api-key
  LEDGER_SIGNING_PRIVATE_KEY:
    file: ./secrets/ledger-signing-private-key
  LEDGER_SIGNING_PUBLIC_KEY:
    file: ./secrets/ledger-signing-public-key
```

Kubernetes

```
apiVersion: v1
kind: Secret
metadata:
  name: ledger-secrets
  namespace: production
type: Opaque
stringData:
  LEDGER_API_KEY: "{operator key}"
  LEDGER_SIGNING_PRIVATE_KEY: "{base64 PKCS#8 Ed25519 key}"
  LEDGER_SIGNING_PUBLIC_KEY: "{base64 X.509 public key}"
```

```

env:
  - name: LEDGER_SECRETS_DIR
    value: /run/secrets/ledger/
volumeMounts:
  - name: ledger-secrets
    mountPath: /run/secrets/ledger
    readOnly: true
volumes:
  - name: ledger-secrets
    secret:
      secretName: ledger-secrets

```

Mount the secrets in their own directory, not on `/run/secrets` itself, so the service account token Kubernetes mounts there stays in place. With Vault or a cloud secret manager, let the External Secrets Operator create this Secret, or let the CSI driver mount the same files; the application side does not change.

Signing Keys

The signing key is the ledger's identity: it signs checkpoints, mandates and receipts. The private key is never stored in the database, only the public keys and their rotation history are. Generate it once, keep it in the secret manager, and keep a second copy offline; a backup of the database without the key cannot sign anything new.

Step 1. Generate an Ed25519 key:

```
openssl genpkey -algorithm ed25519 -outform DER -out ledger-signing.der
```

Step 2. Write the private and the public key in the form the ledger reads:

```
base64 -w0 ledger-signing.der > ledger-signing-private-key
openssl pkey -inform DER -in ledger-signing.der -pubout -outform DER | base64 -w0 > ledger-
signing-public-key
```

Step 3. Put both files in the secret manager as `LEDGER_SIGNING_PRIVATE_KEY` and `LEDGER_SIGNING_PUBLIC_KEY`, as in [Secrets](#), keep one copy offline, and delete `ledger-signing.der`.

Step 4. Start the ledger and check that `/api/v1/keys` shows the new key as `ACTIVE`.

Situation	What to do
Planned rotation	Put the new key in <code>LEDGER_SIGNING_PRIVATE_KEY</code> / <code>PUBLIC_KEY</code> and the old one in <code>LEDGER_SIGNING_PREVIOUS_PRIVATE_KEY</code> , then restart
Lost or leaked key	Start with a new key and <code>LEDGER_SIGNING_UNENDORSED_ROTATION=true</code> ; tell verifiers to pin the new public key
After rotation	Old evidence and checkpoints still verify: retired public keys stay published in <code>/api/v1/keys</code> and in packages

The details of the rotation record are in [Key Rotation](#).

Backup and Restore

What to Back Up

Data	Where	How
Ledger evidence, checkpoints, grants, mandates, agents, key history	PostgreSQL schema <code>ledger</code>	Database backup
Ledger signing private key	Secret manager	The secret manager's own backup, plus one offline copy
Signed checkpoints	Outside the ledger, with auditors	Copy <code>/api/v1/checkpoints/latest</code> regularly
Log checkpoints	With independent witnesses	<code>LEDGER_LOG_WITNESSES_{NAME}_URL</code> and <code>_KEY</code>

Logical Backup

Step 1. Dump the `ledger` schema:

```
docker exec ledger-postgresql pg_dump -U ledger -d ledger --schema=ledger --format=custom > ledger-$(date +%F).dump
```

Restore

Step 1. Restore the dump into a new database:

```
docker exec ledger-postgresql createdb -U ledger ledger_restored
docker exec -i ledger-postgresql pg_restore -U ledger -d ledger_restored --no-owner < ledger-2026-10-09.dump
```

Step 2. Start the ledger on the restored database (`LEDGER_DATABASE_DB=ledger_restored`) with the same signing key. The restore keeps the triggers that make evidence append-only.

Step 3. Check the whole chain:

```
curl -s http://localhost:8090/api/v1/verification -H "Authorization: Bearer {operator key}"
```

The answer must be `"valid": true` with the `headSequence` of the backup. This was checked end to end: evidence and a signed checkpoint were recorded, dumped, restored into a new PostgreSQL, and the restored ledger verified the whole chain and checkpoint and still refused changes to evidence.

No Lost Evidence

A nightly dump loses what was recorded after it. For evidence, run PostgreSQL with continuous WAL archiving and point-in-time recovery (for example `pgBackRest`, or a managed PostgreSQL with PITR) and keep the dump as a second line. A restore can never hide a loss: a signed checkpoint held outside the ledger with a higher sequence than the restored head shows that entries are missing, and any entry that differs fails verification against that checkpoint. Witnesses

give the same guarantee without anyone copying checkpoints by hand: each keeps the last log checkpoint it cosigned and refuses a smaller or forked one, so a restored ledger that lost entries cannot get new cosignatures until it is back on the history the witnesses saw.

Monitoring

Endpoints

Health	Metrics
<code>/actuator/health</code> , <code>/actuator/health/liveness</code> , <code>/actuator/health/readiness</code>	<code>/actuator/prometheus</code>

By default these share the application port. Set `LEDGER_MANAGEMENT_PORT` to serve them on a separate port that only the cluster can reach, and point probes and Prometheus at it.

Metrics

Metric	Meaning
<code>ledger_head_sequence</code>	Sequence of the last evidence entry
<code>ledger_checkpoint_sequence</code>	Sequence covered by the latest signed checkpoint
<code>ledger_checkpoint_lag</code>	Entries not yet covered by a signed checkpoint
<code>ledger_checkpoint_age_seconds</code>	Seconds since the latest signed checkpoint
<code>ledger_evidence_recorded_total</code>	Entries appended, by <code>decision</code> and <code>outcome</code>
<code>http_server_requests_seconds</code>	Requests, latency and status
<code>jvm_*</code> , <code>hikaricp_*</code>	Memory, threads and database connection pools

Prometheus

The ledger compose file runs Prometheus on `http://localhost:9090`. It scrapes the ledger with the configuration in `ledger/prometheus/prometheus.yml`. In Kubernetes the ledger pod carries the `prometheus.io/scrape` , `prometheus.io/path` and `prometheus.io/port` annotations.

Alerts

Alert	Fires when
<code>LedgerDown</code>	The ledger has not answered for 1 minute
<code>LedgerCheckpointStale</code>	New evidence has waited more than 10 minutes for a checkpoint
<code>LedgerHighErrorRate</code>	The ledger answers more than one 5xx every ten seconds

The rules are in `ledger/prometheus/alerts.yml`. Prometheus evaluates them and sends them to Alertmanager, which delivers them on the channels below. Run a full chain verification (`/api/v1/verification`) on a schedule as well; it reads the whole chain, so run it at a quiet hour rather than on every scrape.

Alert Channels

The ledger compose file runs Alertmanager on `http://localhost:9093`. No channel is on by default: alerts are visible there and in Prometheus but go nowhere until a channel's settings are given. Any number of channels can be on at once; each alert goes to all of them, again when it is resolved, and every 4 hours while it keeps firing.

Channel	Turned on by	Optional
Email	<code>ALERT_EMAIL_TO</code> and <code>ALERT_SMTP_HOST</code>	<code>ALERT_SMTP_PORT</code> (587), <code>ALERT_SMTP_FROM</code> , <code>ALERT_SMTP_USERNAME</code> , <code>ALERT_SMTP_PASSWORD</code> , <code>ALERT_SMTP_TLS</code> (true)
Slack	<code>ALERT_SLACK_WEBHOOK_URL</code>	<code>ALERT_SLACK_CHANNEL</code>
Telegram	<code>ALERT_TELEGRAM_BOT_TOKEN</code> and <code>ALERT_TELEGRAM_CHAT_ID</code>	
Webhook	<code>ALERT_WEBHOOK_URL</code>	

```
ALERT_EMAIL_TO=ops@example.com ALERT_SMTP_HOST=smtp.example.com ALERT_SMTP_USERNAME=alerts
ALERT_SMTP_PASSWORD=secret \
  docker compose --file ledger/compose.yaml --project-name ledger up -d alertmanager
  docker logs alertmanager 2>&1 | head -1
```

The first log line names the channels that are on, such as `Alert channels: email`. The settings can also live in a `ledger/.env` file, and each one can come from a file of the same name in `/run/secrets`, so passwords and tokens stay out of the environment. A Slack webhook URL comes from an incoming webhook of a Slack app. A Telegram bot token comes from `@BotFather`, and the chat id is the user's or group's id after they send the bot a message. The webhook channel posts the Alertmanager JSON to any URL, such as a ticketing system or an on-call service.

`ALERT_GROUP_WAIT` (30s), `ALERT_GROUP_INTERVAL` (5m) and `ALERT_REPEAT_INTERVAL` (4h) tune how often messages are sent.

Delivery was checked end to end: with email and webhook on, Prometheus fired an alert, and the message reached a test mail server and the webhook. In Kubernetes, give Alertmanager the same endpoint `ledger/alertmanager/endpoint.sh` with these variables, or use the Alertmanager configuration of the cluster's own monitoring stack.

Rate Limits

The ledger limits requests per caller. A caller is its token or API key, or its address when it sends none. Each caller has a bucket that holds the burst and refills at the per-minute rate; a request over it is answered `429 RATE_LIMIT_EXCEEDED` with `Retry-After` in seconds. Health and metrics under `/actuator/` are never limited. The limit is per instance and kept in memory, so with several replicas a caller can reach the limit once on each.

Requests a minute	Burst
<code>LEDGER_RATE_LIMIT_PER_MINUTE</code> (1200)	<code>LEDGER_RATE_LIMIT_BURST</code> (200)

`0` turns the limit off. Behind a reverse proxy the address comes from `X-Forwarded-For`, so anonymous callers are told apart; let only the proxy set that header. Keep a limit at the proxy as well when the ledger is exposed to the internet, since it can also stop floods before they reach

Java.

Production Checklist

- Profiles without `dev` : the ledger refuses to start with the published development secrets
- The operator key, the database password and the signing key from the secret manager, as files
- A signing key generated for this ledger, with an offline copy
- TLS at a reverse proxy or ingress in front of the ledger; it speaks plain HTTP
- Management port reachable only inside the cluster
- WAL archiving with point-in-time recovery, plus a regular logical dump, and a restore tested once
- Signed checkpoints copied outside the ledger, and at least one independent witness cosigning the log
- Prometheus scraping the ledger, and at least one alert channel on
- Rate limits sized for the expected traffic, plus a limit at the reverse proxy for internet-facing deployments
- `LEDGER_OIDC_*` set so principals approve grants with their own sign-in

DevOps Step by Step

Running the Nexusphere pipeline with the mvn-devops ready-made pipelines.

Before You Start

Every step runs in the root of the repository with Docker running. On Windows use Git Bash or Cygwin and `mvn-devops\devops.bat` instead of `mvn-devops/devops.sh`; machine settings are in [Local Environment Setup](#).

Step 1. Create a GitHub personal access token for `oss-community/nexusphere` with the `repo` scope, and `write:packages` if GitHub Packages will be used. The pipeline clones the repository with it, so a private repository needs it in every pipeline. See [mvn-devops/docs/github-setup.md](#).

Step 2. Check the prerequisites (Bash 4, Docker with compose, curl, jq, git, ssh-keygen, Java 21 and Maven):

```
mvn-devops/devops.sh doctor
```

Step 3. List the ready-made pipelines:

```
mvn-devops/devops.sh pipelines
```

Maven with SonarQube and Nexus

The simplest pipeline. Maven runs on this machine; SonarQube and Nexus run in Docker. The pipeline builds and tests the project, sends it to SonarQube and publishes the artifacts to Nexus. Nothing is deployed.

Step 1. Set it up:

```
mvn-devops/devops.sh setup --pipeline maven-sonarqube-nexus
```

Question	Answer
GitHub username	your GitHub user (asked only if git lacks it)
GitHub personal access token	the token from step 1
Accept the Nexus Community Edition EULA?	yes
Any other question	Enter, to keep the default

Step 2. Run the pipeline:

```
mvn-devops/devops.sh run
```

Step 3. Open SonarQube and Nexus; the URLs and admin users are printed, passwords are in `.devops/`:

```
mvn-devops/devops.sh urls
```

Step 4. Commit the settings so a teammate gets the same tools (`.devops/` stays out of git):

```
git add devops.conf
git commit -m "Add the maven-sonarqube-nexus pipeline"
```

Jenkins Complete

The heaviest pipeline: Jenkins with SonarQube, Nexus, a Docker registry, Trivy, Syft, Cosign, Argo CD on Kubernetes, Vault, PostgreSQL, Prometheus, Loki and k6. The image goes through `dev`, `test` and `staging` on every run, is load-tested in `staging`, and goes to `prod` only when approved. Every tool starts at once, so give Docker plenty of memory (16 GB or more) and disk. Jenkins checks the project out of GitHub, so push your commits first.

Step 1. Push the branch the pipeline will build:

```
git push
```

Step 2. Set it up:

```
mvn-devops/devops.sh setup --pipeline jenkins-complete
```

Question	Answer
GitHub username	your GitHub user (asked only if git lacks it)
GitHub personal access token	the token from step 1 of Before You Start
Accept the Nexus Community Edition EULA?	<code>yes</code>
Any other question	Enter, to keep the default

Step 3. Run ci, cd, dev, test and staging:

```
mvn-devops/devops.sh run
```

Step 4. Approve production when staging looks right:

```
mvn-devops/devops.sh run --phase prod
```

Step 5. Open Jenkins, SonarQube, Nexus, Argo CD, Vault, Grafana and the other tools:

```
mvn-devops/devops.sh urls
```

Step 6. Put the previous image back if a release goes wrong:

```
mvn-devops/devops.sh rollback prod
```

Step 7. Commit the settings:

```
git add devops.conf
git commit -m "Add the jenkins-complete pipeline"
```

Notes for This Repository

`devops.sh stages` shows what a pipeline will run before anything starts. For `jenkins-complete` two stages assume a single-module project and have not been run against this multi-module one yet: `migrate` points Flyway at `src/main/resources/db/migration` in the root, while the migrations of this repository live in `ledger/server/src/main/resources/db/migration/ledger`, and `image` runs Jib from the root, but the executable jar is built by `ledger/server`. Expect to adjust these two stages on the first run.

Every Day

Command	Does
<code>mvn-devops/devops.sh status</code>	Shows the selected tools and whether they are running
<code>mvn-devops/devops.sh stages</code>	Prints the stages and the exact commands they run
<code>mvn-devops/devops.sh run</code>	Runs the pipeline
<code>mvn-devops/devops.sh urls</code>	Prints the tool URLs and users
<code>mvn-devops/devops.sh secrets --reconfigure</code>	Answers every question again

Stop and Clean Up

```
mvn-devops/devops.sh down
mvn-devops/devops.sh destroy
```

`down` stops the tools and keeps their data; `destroy` asks before removing their containers and volumes, and again before deleting the stored values in `.devops/`.

Upgrade

```
mvn-devops/devops.sh upgrade --check
mvn-devops/devops.sh upgrade
```

`upgrade` replaces the `mvn-devops` folder with the latest release after checking it against the release's `SHA256SUMS`. Commit the folder afterwards.

PART 7

Testing

The end-to-end test suite and the scenarios it proves.

Nexusphere Ledger End-to-End Tests

Getting Started

Prerequisites

- [Java 21](#)
- [Maven 3](#)
- [Docker](#)

Run Only This Module

```
mvn -pl ledger/ledger-e2e-tests -am verify
```

Run One Test Class

```
mvn -pl ledger/ledger-e2e-tests -am verify -Dit.test=VerificationE2ETest -Dfailsafe.failIfNoSpecifiedTests=false -Dsurefire.failIfNoSpecifiedTests=false
```

Test Harness

`LedgerE2ETestBase` starts the ledger on a free port, which is also its mandate issuer address, with the `postgresql` and `dev` profiles against a PostgreSQL 18 container. `LedgerClient` talks to it over HTTP with the development API key or a registered agent's key, like any outside caller, and rebuilds evidence entries from the JSON so they can be checked with `ledger/chain` without trusting the server. `FakeMcpServer` is a small MCP server on a random port, configured as the `files` server of the gateway; it answers `initialize`, `tools/list` and `tools/call`, and streams the answer of tools ending in `_sse`. `A2aE2ETest` starts a second ledger, `SupplierLedger`, with its own PostgreSQL container and signing key, which trusts the first one and forwards to `FakeA2aAgent`, which streams `message/stream` and `tasks/resubscribe`, so mandates, request proofs and receipts cross between two ledgers over HTTP. `PrincipalConsentE2ETest` starts a `StandaloneLedger` that trusts `FakeOidcProvider`, a small OpenID Connect provider on a random port that signs RS256 tokens for any principal. `KeyRotationE2ETest` restarts a `StandaloneLedger` against one PostgreSQL container with a new signing key, so the rotation happens at startup as it does in production. `WitnessE2ETest` starts two `StandaloneLedger`s with their own PostgreSQL containers: one is the log and asks the other, which watches it, to cosign its log checkpoints over the C2SP witness protocol.

Test Classes

Class	Scenario
<code>EvidenceE2ETest</code>	Recorded evidence is sealed and linked to the previous entry
	Evidence is read back exactly as recorded, with microsecond timestamps

Class	Scenario
	Evidence is listed per agent and per principal in pages
	A denied action is recorded with its reason
	Invalid evidence is rejected with field errors
	Malformed requests, unknown evidence and bad paging are reported clearly
VerificationE2ETest	The ledger verifies its own chain and checkpoints
	A checkpoint is verified offline with the published key
	An outsider rebuilds and verifies the whole chain from the API
PackageE2ETest	A package discloses only the requested evidence and verifies offline
	A package starts at the last checkpoint before the evidence
	Any change to a package is detected
	Packages are for the operator and need matching evidence
	The operator can have the ledger verify a package
ApiKeyE2ETest	The API requires the ledger key
	Health is open for probes
RateLimitE2ETest	A caller over its rate limit is told when to retry
AgentE2ETest	A registered agent gets its own key once
	Registering the same agent twice is a conflict
	A disabled agent or an old key is rejected
	Agent lifecycle is recorded as evidence
	An agent cannot do what only the operator may
	An agent records and reads only its own evidence
GrantE2ETest	A grant is created and its terms can be recomputed from the evidence
	A revoked grant is recorded and shown as revoked
	Invalid grants are rejected with field errors
	Grants need an active agent
	An agent sees only its own grants
DecisionE2ETest	A covered action is allowed and its outcome is recorded
	Every denial is recorded with its reason
	The operator can ask for a decision on behalf of an agent
	Outcomes are reported once by the deciding agent for allowed actions only
	An agent cannot ask for another agent's decision and the chain stays valid
McpGatewayE2ETest	An allowed tool call is forwarded and recorded with its outcome
	A denied tool call never reaches the server
	Tool errors are recorded as failed outcomes
	Streamed responses are returned as JSON

Class	Scenario
	A streamed answer is relayed live with the server requests inside
	The server stream is relayed on GET
	Other messages pass through with the session
	Bad requests are answered as JSON-RPC errors
	An unreachable server is recorded as a failed call
MandateE2ETest	An agent obtains a mandate that anyone can verify with the published key
	Revoking a mandate sets its bit in the signed status list
	Revoking the grant revokes every mandate issued from it
	An agent cannot obtain or revoke mandates it does not own
	A mandate cannot outlive its grant
	The keys and status list are public and signed
MandateVerifierE2ETest	Another organization verifies a mandate from the public endpoints only
	An agent can present a mandate without revealing its principal
	The verifier sees a revocation as soon as the status list is refetched
	The verifier rejects what the mandate does not allow
A2aE2ETest	An allowed message carries a mandate and both ledgers hold matching evidence
	A denied message never leaves the ledger
	A failed task is recorded as failed on both sides with a receipt
	The receiver rejects a revoked mandate and records the denial
	Replayed, tampered or forged requests are rejected without reaching the agent
	The receiver counts the uses of a mandate on its own
	An unreachable peer is answered without delivery
	A streamed task is relayed live and the receipt covers every event
	A stream that ends in a failed task is recorded as failed on both sides
PrincipalConsentE2ETest	A principal signs in with an OIDC token that only opens the principal API
	An operator grant waits for the principal's consent
	An agent asks for a grant and the principal denies it
	A principal grants and revokes on its own
	A principal cannot see or decide another principal's grants
KeyRotationE2ETest	An endorsed rotation keeps earlier checkpoints and mandates valid
	A new key without the previous key is refused unless the operator allows it
	A retired key cannot sign again
TransparencyLogE2ETest	The log checkpoint is a signed note over the Merkle root
	An entry is proven in the signed tree
	A later tree is proven to extend an earlier one

Class	Scenario
	A package proves its entries in the log
	Verification checks the Merkle tree and log checkpoints
	This ledger witnesses no log unless configured
WitnessE2ETest	A witness cosigns every checkpoint that extends what it saw
	A witness refuses a forked or stale checkpoint
ScittE2ETest	Every entry has a signed statement and a receipt from the log
	A receipt can be asked for at an earlier log checkpoint
	Statements and receipts belong to the evidence owner
	A package carries a statement and receipt for every disclosed entry

PART 8

Build and Delivery Toolkit

mvn-devops, the DevOps framework the project builds and ships with.

mvn-devops

A modular DevOps framework for Maven projects, written in plain Bash.

You pick the tools from a menu, mvn-devops starts them in Docker, configures them (admin passwords, tokens, repositories) and turns the stages they contribute into a pipeline for the orchestrator you chose: plain `mvn` on your machine, `mvn` in a container on a Docker machine, Jenkins or Concourse.

```
$ mvn-devops/devops.sh init

Pipeline orchestrator (choose one)
 1) concourse      Concourse CI in Docker: a ci job on every push, a manual cd job
 2) jenkins        Jenkins in Docker, configured as code (no setup wizard)
 3) maven          Run the stages with mvn directly on this machine
 4) maven-container Run the stages with mvn in a container on the Docker machine, from a fresh
checkout of GitHub
> [1]: 2

Code quality (choose any, comma separated, 0 for none)
 1) sonarqube      Static code analysis (SonarQube Community + PostgreSQL)
> [0]: 1

Artifact repositories (choose any, comma separated, 0 for none)
 1) github-packages Deploy to the Maven registry of the GitHub repository
 2) jfrog          JFrog Artifactory OSS + PostgreSQL
 3) nexus          Sonatype Nexus 3 for Maven releases and snapshots
> [0]: 1,3
...
```

- **No changes to your pom.** Plugins are called by their coordinates and credentials come from the framework's own settings file.
- **Nothing to click.** Admin passwords, tokens, repositories, the Jenkins job and the Concourse pipeline are created for you.
- **Shared by the team.** The tools and answers are kept in a committed `devops.conf`; passwords and tokens stay on each machine.
- **Anywhere.** Each tool runs in Docker on your machine or a VM, or is an existing server with its own URL.

Prerequisites

Bash 4+ (Linux, macOS, or Git Bash on Windows), Docker with the compose plugin, `curl`, `jq`, `git` and `ssh-keygen`, plus Java and Maven when the pipeline runs on your machine. `devops.sh doctor` checks them; install commands per system are in [docs/prerequisites.md](#).

Install

mvn-devops is meant to live inside the project, like the Maven wrapper. From the project root:

```
curl -fsSL0 https://github.com/oss-community/mvn-devops/releases/download/v1.0.0/mvn-devops-1.0.0.zip
unzip -q mvn-devops-1.0.0.zip && mv mvn-devops-1.0.0 mvn-devops && rm mvn-devops-1.0.0.zip
git add mvn-devops && git commit -m "Add mvn-devops 1.0.0"
```

Later, `mvn-devops/devops.sh upgrade` replaces it with the latest release. Windows steps, the deb and rpm packages, and notes on line endings are in [docs/installation.md](#).

Quick start

Step-by-step guides, one command per step, are in [docs/getting-started.md](#).

Create the GitHub token first ([docs/github-setup.md](#)), then from the project root:

```
mvn-devops/devops.sh setup      # menu, questions, containers, tokens, pipeline
mvn-devops/devops.sh run       # run the pipeline
mvn-devops/devops.sh urls      # web consoles and how to log in
```

On Windows use `mvn-devops\devops.bat` instead. Commit the `devops.conf` that `setup` writes: a teammate who clones the project runs the same `setup` and is only asked for their own passwords, tokens and user names. Passwords, tokens and generated files go to `.devops/`, which keeps itself out of git.

`setup` is these five steps; each can also be run on its own:

Step	Command	What happens
1	<code>init</code>	Menu per category. Saves the choice in <code>devops.conf</code> .
2	<code>secrets</code>	Each selected module asks for the values it needs. Passwords default to random ones.
3	<code>up</code>	Modules prepare (config files, keys), then <code>docker compose up</code> with the selected tools.
4	<code>configure</code>	Modules finish their tool: replace default admin passwords, create tokens and repositories.
5	<code>publish</code>	The orchestrator renders the pipeline and installs it (Jenkins job, Concourse pipeline, local script).

Or start from a ready-made pipeline, a tested combination of tools, and answer only for your GitHub user and token ([docs/pipelines.md](#)):

```
mvn-devops/devops.sh pipelines
mvn-devops/devops.sh setup --pipeline jenkins-sonarqube-nexus
```

Without questions, for scripts and CI:

```
mvn-devops/devops.sh -y init --orchestrator jenkins --with sonarqube,nexus,github-pages
mvn-devops/devops.sh -y setup
```

[examples/hello-maven](#) is a small project to try it on.

Commands

Command	
<code>init [--orchestrator X --with a,b \] --pipeline name]</code>	choose modules, or a ready-made pipeline
<code>pipelines</code>	list the ready-made pipelines
<code>setup [--pipeline name]</code>	init (if needed) + secrets + up + configure + publish
<code>secrets [--reconfigure]</code>	ask for values; <code>--reconfigure</code> asks again
<code>up, configure, publish</code>	single setup steps
<code>stages</code>	the ordered stages contributed by the selected modules
<code>render</code>	write pipeline files into <code>.devops/generated</code>
<code>run</code>	run the pipeline. maven, maven-container: <code>--dry-run</code> , <code>--from <stage></code> , <code>--only <stage></code> , <code>--phase ci\ cd</code> . concourse: <code>--phase ci\ cd</code> . Every orchestrator: <code>--phase <environment></code> approves an environment
<code>rollback [environment] [--to TAG]</code>	put the previous image back, in the last environment by default
<code>release [--version X] [--next Y] [--dry-run] [--no-push]</code>	release the project (docs/releasing.md)
<code>status, urls, logs [service], compose ...</code>	operations
<code>down</code>	stop containers, keep data
<code>destroy</code>	remove containers and volumes, the deploy key and webhook on GitHub, optionally the stored values
<code>export-compose [dir]</code>	write one <code>docker-compose.yml</code> and its <code>.env</code> for the selected tools
<code>env [--show\ --windows]</code>	regenerate env files; <code>--show</code> masks secrets; <code>--windows</code> writes a <code>setx</code> script for IDEs
<code>get <KEY></code>	print one stored value, e.g. <code>get SONAR_ADMIN_PASSWORD</code>
<code>modules, doctor [--fix]</code>	list modules, check prerequisites; <code>--fix</code> repairs CRLF line endings
<code>upgrade [--version X] [--check]</code>	replace the copy of mvn-devops inside the project with a release

Documentation

Getting started	step-by-step guides
Installation	packages, the zip inside a project, upgrades, line endings
Prerequisites	install commands for Windows, Linux and macOS
GitHub setup	tokens and their scopes, SSH key
Ready-made pipelines	tested combinations of tools, and your own
Modules and stages	every tool and the stages it adds
Project requirements	what the Maven project needs (almost nothing)

Orchestrators	maven, maven-container, Jenkins, Concourse, and the Docker images
Where the tools run	existing servers, a VM, the exported compose file
Project site	the Maven site on GitHub Pages
Deployment	environments, container images, registries and deploying the application
Releasing your project	<code>devops.sh release</code>
Troubleshooting	common errors and their fixes
IDE, ngrok	IntelliJ settings, exposing a local Jenkins to GitHub
Design	how the framework is built
Writing a module	adding a tool
Development	tests and releasing mvn-devops
Changelog	what changed in each version

License

Apache License 2.0

Installation

Every release on the [releases page](#) has:

File	For
mvn-devops_<version>_all.deb	Debian, Ubuntu: <code>sudo apt install ./mvn-devops_<version>_all.deb</code>
mvn-devops-<version>-1.noarch.rpm	Fedora, RHEL, Rocky: <code>sudo dnf install ./mvn-devops-<version>-1.noarch.rpm</code>
mvn-devops-<version>.zip	Windows: unzip, add the folder to <code>PATH</code> , run <code>devops.bat</code> (needs Git for Windows)
mvn-devops-<version>.tar.gz	macOS or any Linux: unpack and add the folder to <code>PATH</code>
SHA256SUMS	checksums of the files above

The Linux packages install into `/usr/share/mvn-devops` and add the command `mvn-devops`. With the zip or tar.gz the command is `devops.bat` or `devops.sh` in the unpacked folder. A `git clone` of this repository works the same way. `mvn-devops --version` prints the installed version.

Shipping it with the project (zip)

mvn-devops can live inside the project and be committed with it, so everyone who clones the project gets the same DevOps setup, like the Maven wrapper.

1. Download and extract `mvn-devops-<version>.zip` from the [releases page](#) into the project root, and rename the folder to `mvn-devops` so paths stay the same across upgrades.

Linux, macOS, Git Bash:

```
cd my-maven-project
curl -fsSL0 https://github.com/oss-community/mvn-devops/releases/download/v1.0.0/mvn-devops-1.0.0.zip
unzip -q mvn-devops-1.0.0.zip && mv mvn-devops-1.0.0 mvn-devops && rm mvn-devops-1.0.0.zip
```

Windows PowerShell (or right-click the zip > Extract All, then rename the folder):

```
cd my-maven-project
Invoke-WebRequest https://github.com/oss-community/mvn-devops/releases/download/v1.0.0/mvn-devops-1.0.0.zip -OutFile mvn-devops.zip
Expand-Archive mvn-devops.zip -DestinationPath . ; Rename-Item mvn-devops-1.0.0 mvn-devops ;
Remove-Item mvn-devops.zip
```

The project then looks like this:

```
my-maven-project/
  pom.xml
  mvn-devops/      committed: devops.sh, devops.bat, lib/, modules/, ...
  .devops/         created by setup, never committed (passwords, tokens)
```

2. Commit it:

```
git add mvn-devops && git commit -m "Add mvn-devops 1.0.0"
```

3. Use it from the project root:

```
mvn-devops/devops.sh setup          # Windows: mvn-devops\devops.bat setup
mvn-devops/devops.sh run
```

Commit `devops.conf` too. Everyone else clones the project and runs `mvn-devops/devops.sh setup` : the tools and answers come from `devops.conf` , so they are only asked for their own passwords, tokens and user names.

Upgrade with `mvn-devops/devops.sh upgrade` : it downloads the latest release, checks it against `SHA256SUMS` and replaces the folder; review the change with `git status` and commit it. `upgrade --check` only compares versions, `upgrade --version X` picks a release, and `--version` shows the version in use.

Notes:

- The folder has no `pom.xml` , so Maven, Sonar and the pipeline ignore it.
- Use the zip or tar.gz, not a `git clone` , which would put a repository inside the project's repository.
- On Linux and macOS keep the executable bit when committing (`unzip` and git keep it). If `devops.sh` lost it, run `git update-index --chmod=+x mvn-devops/devops.sh` .
- Line endings: Bash cannot run scripts with Windows (CRLF) line endings, and git on Windows (`core.autocrlf=true`) converts files to CRLF on checkout. The zip contains `mvn-devops/.gitattributes` , which keeps every file LF (and `devops.bat` CRLF) whatever `core.autocrlf` is, so commit it together with the folder. `devops.sh doctor` reports files that are already CRLF and prints the command that fixes them. A project that committed mvn-devops before this file existed adds it from the new zip, then fixes its checkout once with:

```
bash git add --renormalize mvn-devops && git commit -m "Normalize mvn-devops line endings" rm -rf mvn-devops && git checkout -- mvn-devops
```

Installing the prerequisites

Tool	Needed for
Bash 4+, curl, jq, git, ssh-keygen	always
Docker with the compose plugin	tools that run in Docker
Java 21 and Maven 3.9	the <code>maven</code> orchestrator and <code>release</code> (Jenkins and Concourse bring their own)

`devops.sh doctor` (or `mvn-devops doctor`) checks all of them.

Debian, Ubuntu

```
sudo apt-get update
sudo apt-get install -y bash curl jq git openssh-client openjdk-21-jdk maven
## Docker Engine with the compose plugin: https://docs.docker.com/engine/install/ubuntu/
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker "$USER"      # log out and in again
```

Fedora, RHEL, Rocky

```
sudo dnf install -y bash curl jq git openssh-clients java-21-openjdk-devel maven
## Docker Engine: https://docs.docker.com/engine/install/fedora/ (or /rhel/)
sudo dnf -y install dnf-plugins-core
sudo dnf config-manager --add-repo https://download.docker.com/linux/fedora/docker-ce.repo
sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
sudo systemctl enable --now docker && sudo usermod -aG docker "$USER"
```

The Maven packages of Debian, Ubuntu and RHEL are often older than 3.9; check with `mvn -version` and install Maven by hand as described below if needed.

macOS

```
brew install bash jq git openjdk@21 maven
brew install --cask docker      # Docker Desktop, start it once
```

macOS ships Bash 3.2; Homebrew's Bash 4+ is used through `#!/usr/bin/env bash` when `/opt/homebrew/bin` (or `/usr/local/bin`) comes first in `PATH`.

Windows

1. **Git for Windows**: brings Git Bash, curl and ssh-keygen. `devops.bat` finds it.
2. **jq**: `winget install jqlang.jq`, or download `jq-windows-amd64.exe` from jqlang.github.io/jq/, rename it to `jq.exe` and put it in a folder on `PATH`.
3. **Docker Desktop** with the WSL 2 backend.
4. **Java 21**: `winget install EclipseAdoptium.Temurin.21.JDK`.

5. Maven: download the binary zip from maven.apache.org, unpack it to e.g. `C:\sdk\maven`, then in a console run as administrator: `bat setx /M MAVEN_HOME C:\sdk\maven setx /M PATH "%PATH%;C:\sdk\maven\bin"`

Open a new console afterwards so `PATH` is reloaded.

Manual Maven install (any Linux, macOS)

```
curl -fsSL0 https://archive.apache.org/dist/maven/maven-3/3.9.11/binaries/apache-maven-3.9.11-bin.tar.gz
sudo mkdir -p /opt/maven && sudo tar -xzf apache-maven-3.9.11-bin.tar.gz -C /opt/maven --strip-components=1
echo 'export PATH=/opt/maven/bin:$PATH' >> ~/.bashrc
```

Check

```
java -version && mvn -version && git --version && docker compose version && jq --version
```

What your Maven project needs

A `pom.xml` that builds with `mvn package`. Nothing else.

The stages call every plugin by its full coordinates and configure it with `-D` properties, so the project needs no profiles, no `distributionManagement`, no plugin declarations and no settings file of its own:

```
org.apache.maven.plugins:maven-checkstyle-plugin:3.6.0:check -
Dcheckstyle.config.location=google_checks.xml
org.sonarsource.scanner.maven:sonar-maven-plugin:5.8.0.7211:sonar -Dsonar.host.url=$SONAR_URL -
Dsonar.token=$SONAR_TOKEN
package ... org.apache.maven.plugins:maven-deploy-plugin:3.2.0:deploy -
DaltSnapshotDeploymentRepository=nexus-snapshots::$NEXUS_ARTIFACTORY_SNAPSHOT_URL ...
```

`./devops.sh stages` prints the exact commands for the selected modules.

Why not a profiles.xml?

Maven 2 read profiles from a `profiles.xml` next to the `pom`; Maven 3 removed it. Profiles in `settings.xml` can only hold properties, repositories and activation rules, not plugins or `distributionManagement`. Calling the plugins directly is the way to keep all of that out of the project.

Settings and credentials

The framework passes `templates/settings.xml` as global settings (`-gs`). It declares one `<server>` per deploy target, with credentials read from environment variables the modules export:

Server id	Credentials
github	GITHUB_USERNAME , GITHUB_PACKAGE_TOKEN
nexus-snapshots , nexus-releases	NEXUS_ARTIFACTORY_USERNAME , NEXUS_ARTIFACTORY_PASSWORD
jfrog-snapshots , jfrog-releases	JFROG_ARTIFACTORY_USERNAME , JFROG_ARTIFACTORY_ENCRYPTED_PASSWORD

Jenkins and Concourse write the same file into the build container before the first stage.

Optional knobs

All of them are questions asked by `secrets` :

Value	Default	
MAVEN_SETTINGS	empty	the project's own settings file, added with <code>-s</code>
MAVEN_PROFILES	empty	project profiles to activate in every stage
MAVEN_ATTACH_SOURCES	yes	deploy sources and javadoc jars too
MAVEN_CHECKSTYLE	yes	run the checkstyle stage

Value	Default	
MAVEN_CHECKSTYLE_CONFIG	google_checks.xml	sun_checks.xml , a URL, or a file in the project such as code-style/checkstyle.xml
SITE_BRANCH	site	branch the site is published to

Plugin versions live in [lib/maven.sh](#). Override one for a project by storing a value, for example `MVN_JAVADOC_VERSION`.

If the pom already configures one of these plugins (checkstyle rules, Sonar exclusions, site reports in `<reporting>`), that configuration still applies; the command line only fills in what the pom leaves open.

Site

The site stage builds the site with the reports in the pom's `<reporting>` section, or the default project information pages when there is none. `maven-site-plugin`'s own `stage` goal requires `<distributionManagement><site>` in the pom, so a short shell step (`stage-site`) assembles `target/staging` instead: the root site plus each first-level module's site in a sub directory. `publish-site` pushes it with `maven-scm-publish-plugin` to `git@github.com:<owner>/<repo>.git`, branch `site`. Pushing uses SSH: your own key when the `maven` orchestrator runs on your machine, and a deploy key (generated and registered on the repository by `configure`) when Jenkins or Concourse runs the build.

Create the `site` branch once and point GitHub Pages at it, as described in [Project site on GitHub Pages](#).

Tokens

See [github-setup.md](#) for the GitHub tokens, their scopes and the SSH key.

GitHub tokens and SSH keys

Do this once before `setup . secrets` asks for the tokens; the SSH key is only needed when the `maven` orchestrator publishes the site from your machine.

Tokens

Create classic tokens under GitHub > Settings > Developer settings > Personal access tokens > Tokens (classic) > Generate new token ([direct link](#)). On GitHub Enterprise it is the same menu on your server. Copy each token right away; GitHub shows it only once.

Token	Asked as	Scopes	Used for
Repository token	GITHUB_TOKEN	repo	Jenkins and Concourse clone the repository; <code>configure</code> checks the site branch and registers the deploy key
		admin:repo_hook	only with <code>JENKINS_TRIGGER=webhook</code> : <code>configure</code> registers the push webhook
Packages token (optional)	GITHUB_PACKAGE_TOKEN	write:packages , read:packages	the <code>deploy-github</code> stage. Empty: the repository token is used, so give it these scopes too
		delete:packages	only if you want to delete package versions later
		write:packages on GITHUB_TOKEN	the <code>image</code> stage of the <code>github-container</code> module pushes to ghcr.io with the repository token

With a public repository and no private dependencies, `public_repo` is enough instead of `repo`.

Fine-grained tokens work for the repository token: select the repository and give it *Contents: Read and write*, *Administration: Read and write* (deploy keys) and, for the webhook, *Webhooks: Read and write*. GitHub Packages does not accept fine-grained tokens yet, so the packages token must be a classic one.

Change a token later with `devops.sh secrets --reconfigure`, then `devops.sh publish` so the pipeline gets the new value.

SSH key (site publishing with the `maven` orchestrator)

`publish-site` pushes the site with `git@github.com:<owner>/<repo>.git`. With Jenkins or Concourse a deploy key is generated and registered for you. With the `maven` orchestrator your own key is used, so set it up once:

```
ssh-keygen -t ed25519 -C "my-laptop"           # accept the default file
eval "$(ssh-agent -s)" && ssh-add ~/.ssh/id_ed25519 # Windows: run in Git Bash
ssh-keyscan github.com >> ~/.ssh/known_hosts
cat ~/.ssh/id_ed25519.pub                     # add it at github.com/settings/ssh/new
ssh -T git@github.com                         # "Hi <user>! You've successfully
authenticated"
```

Or add it with the GitHub CLI: `gh ssh-key add ~/.ssh/id_ed25519.pub`. `devops.sh doctor` runs the `ssh -T` check for the project's GitHub host.

Getting started, step by step

Every guide starts in the root of your Maven project, with Docker running. On Windows, use `mvn-devops\devops.bat` instead of `mvn-devops/devops.sh`.

- [Guide 1: a ready-made pipeline](#)
- [Guide 2: your own choice of tools](#)
- [Guide 3: Maven in a container on another machine](#)
- [Guide 4: a ready-made pipeline file of your own](#)
- [Guide 5: a teammate joins](#)

Before every guide

Step 1. Create a GitHub token with the `repo` scope (and `write:packages`). See [github-setup.md](#).

Step 2. Add mvn-devops to the project:

```
curl -fsSL0 https://github.com/oss-community/mvn-devops/releases/download/v1.0.0/mvn-devops-1.0.0.zip
unzip -q mvn-devops-1.0.0.zip && mv mvn-devops-1.0.0 mvn-devops && rm mvn-devops-1.0.0.zip
```

Step 3. Check the prerequisites:

```
mvn-devops/devops.sh doctor
```

Guide 1: a ready-made pipeline

Step 1. List the ready-made pipelines:

```
mvn-devops/devops.sh pipelines
```

Step 2. Set one up:

```
mvn-devops/devops.sh setup --pipeline jenkins-sonarqube-nexus-argocd
```

Questions:

Question	Answer
GitHub username	your user (asked only when git does not know it)
GitHub token	the token from <i>Before every guide</i>
Accept the Nexus Community Edition EULA?	yes (only with Nexus)

Step 3. Run the pipeline:

```
mvn-devops/devops.sh run
```

Step 4. Approve production when you want it deployed:

```
mvn-devops/devops.sh run --phase prod
```

Step 5. Open the tools:

```
mvn-devops/devops.sh urls
```

Step 6. Commit the settings:

```
git add devops.conf mvn-devops && git commit -m "Add mvn-devops"
```

Guide 2: your own choice of tools

Step 1. Choose the tools from the menu:

```
mvn-devops/devops.sh init
```

Questions (type the number, or several separated by commas):

Menu	Example answer
Pipeline orchestrator	jenkins
Code quality	sonarqube
Artifact repositories	nexus
Container image	docker-registry
Deployment	kubernetes
The other menus	0 for none

Step 2. Answer the questions:

```
mvn-devops/devops.sh secrets
```

Press Enter to keep each default. Type only:

Question	Answer
GitHub token	the token from <i>Before every guide</i>
Deployment environments, in the order the image goes through them	Enter for staging production , or e.g. dev test staging prod
Approval before deploying to <environment>	Enter (only the last one needs it)
Accept the Nexus Community Edition EULA?	yes (only with Nexus)

Step 3. Start, configure and publish:

```
mvn-devops/devops.sh up
mvn-devops/devops.sh configure
mvn-devops/devops.sh publish
```

Step 4. Run the pipeline:

```
mvn-devops/devops.sh run
```

Step 5. Commit the settings:

```
git add devops.conf mvn-devops && git commit -m "Add mvn-devops"
```

Guide 3: Maven in a container on another machine

The pipeline runs in a container with Java and Maven, which checks the project out of GitHub. Nothing runs on your machine but `devops.sh`.

Step 1. Point Docker at the other machine (skip this step to use this one):

```
export DOCKER_HOST=ssh://user@build-vm
```

Step 2. Check that Docker answers:

```
mvn-devops/devops.sh doctor
```

Step 3. Push your commits; the container builds what is on GitHub:

```
git push
```

Step 4. Set it up:

```
mvn-devops/devops.sh setup --pipeline maven-container-kubernetes
```

Questions:

Question	Answer
GitHub username	your user (asked only when git does not know it)
GitHub token	the token from <i>Before every guide</i>

Step 5. Run the pipeline:

```
mvn-devops/devops.sh run
```

Step 6. Approve production:

```
mvn-devops/devops.sh run --phase prod
```

To choose the tools yourself instead of step 4, follow guide 2 and choose `maven-container` as the orchestrator.

Guide 4: a ready-made pipeline file of your own

Step 1. Write the file, for example `../team/jenkins-k8s.conf`:

```
## Jenkins and SonarQube with three environments.
ORCHESTRATOR=jenkins
TOOLS=sonarqube,docker-registry,kubernetes
ENVIRONMENTS=dev test prod
ENV_TEST_APPROVAL=yes
ENV_PROD_APPROVAL=yes
```

Step 2. Set it up:

```
mvn-devops/devops.sh setup --pipeline ../team/jenkins-k8s.conf
```

Questions: the GitHub token, and the Nexus EULA when Nexus is in `TOOLS`.

Step 3. Run the pipeline:

```
mvn-devops/devops.sh run
```

Step 4. Approve each environment that needs it, in order:

```
mvn-devops/devops.sh run --phase test
mvn-devops/devops.sh run --phase prod
```

Guide 5: a teammate joins

Step 1. Clone the project:

```
git clone https://github.com/acme/app.git && cd app
```

Step 2. Set it up; the tools come from the committed `devops.conf`:

```
mvn-devops/devops.sh setup
```

Questions: the GitHub username and token. Passwords are generated.

Step 3. Run the pipeline:

```
mvn-devops/devops.sh run
```

When something goes wrong

Problem	Command
A stage failed	<code>mvn-devops/devops.sh run --from <stage></code>
See the stages	<code>mvn-devops/devops.sh stages</code>
Logs of a tool	<code>mvn-devops/devops.sh logs <service></code>
Answer the questions again	<code>mvn-devops/devops.sh secrets --reconfigure</code>
Stop everything	<code>mvn-devops/devops.sh down</code>
Remove everything	<code>mvn-devops/devops.sh destroy</code>

More in [troubleshooting.md](#).

Ready-made pipelines

A ready-made pipeline is a tested combination of tools with its settings, in the [pipelines](#) folder. Choose one instead of going through the menu and the questions:

```
mvn-devops/devops.sh pipelines # what there is
mvn-devops/devops.sh setup --pipeline jenkins-sonarqube-nexus
```

`setup` then asks only what cannot be chosen for you: your GitHub user and token, and whether you accept the Nexus licence when Nexus is part of it. Every password, token and key is generated for your project, so two projects set up from the same pipeline share no secrets. The images of the tools are downloaded when they start, as with any other setup.

Pipeline	Orchestrator	Tools	Environments
maven-sonarqube-nexus	maven	SonarQube, Nexus	
jenkins-sonarqube-nexus	Jenkins	SonarQube, Nexus	
concourse-sonarqube-nexus	Concourse	SonarQube, Nexus	
maven-docker-host	maven	registry, Docker Compose over SSH	dev test staging prod
jenkins-kubernetes	Jenkins	SonarQube, Nexus, registry, Trivy, Syft, Cosign, Kubernetes	dev test staging prod
jenkins-observability	Jenkins	registry, Docker Compose over SSH, PostgreSQL, Prometheus, Loki, k6	dev test staging prod
concourse-argocd	Concourse	registry, Argo CD, Vault, Sealed Secrets	dev test staging prod
jenkins-sonarqube-nexus-argocd	Jenkins	SonarQube, Nexus, registry, Argo CD	dev test staging prod
maven-container-sonarqube-nexus	maven-container	SonarQube, Nexus	
maven-container-kubernetes	maven-container	registry, Trivy, Kubernetes, PostgreSQL	dev staging prod
maven-kubernetes	maven	registry, Kubernetes	dev test staging prod
concourse-kubernetes-vault	Concourse	SonarQube, registry, Kubernetes, Vault, PostgreSQL	dev test staging prod
jenkins-argocd-observability	Jenkins	registry, Argo CD, Prometheus, Loki, k6	dev test staging prod
concourse-docker-host-secure	Concourse	registry, Trivy, Syft, Cosign, Docker Compose over SSH, PostgreSQL	dev staging prod
maven-sonarqube-github	maven	SonarQube, GitHub Packages, GitHub Pages	
jenkins-complete	Jenkins	SonarQube, Nexus, registry, Trivy, Syft, Cosign, Argo CD, Vault, PostgreSQL, Prometheus, Loki, k6	dev test staging prod

Only `prod` waits for approval, except in `concourse-kubernetes-vault`, where staging does too; see [Environments](#). `concourse-argocd` releases `prod` as a canary. The end-to-end test runs every combination on GitHub's runners except `maven-sonarqube-github`, which needs a real GitHub repository. `jenkins-complete` is the heaviest: it starts every tool at once.

Step by step: [getting-started.md](#).

What it writes

`init --pipeline` selects the pipeline's tools and writes its settings to the project's `devops.conf`, together with `PIPELINE=<name>`. From then on the project is like any other: commit `devops.conf`, and a teammate's `setup` gets the same tools and settings. Change a setting in `devops.conf`, or answer everything again with `devops.sh secrets --reconfigure`.

A pipeline of your own

A ready-made pipeline is a `devops.conf` without the project's name: the orchestrator, the tools as `--with` takes them, and any setting that is neither secret nor personal. The first comment line describes it.

```
## Jenkins and SonarQube with three environments.
ORCHESTRATOR=jenkins
TOOLS=sonarqube,docker-registry,kubernetes
ENVIRONMENTS=dev test prod
ENV_TEST_APPROVAL=yes
ENV_PROD_APPROVAL=yes
KUBERNETES_REPLICAS=1
```

Keep the file anywhere, for example in a repository your team shares, and pass its path:

```
mvn-devops/devops.sh setup --pipeline ../team/jenkins-k8s.conf
```

A file added to the `pipelines` folder of `mvn-devops` is listed by `devops.sh pipelines`.

Modules and stages

Category	Mode	Modules
Source control	required	github
Build	required	maven (validate, package, test, checkstyle, install)
Pipeline orchestrator	one	maven , maven-container , jenkins , concourse
Code quality	any	sonarqube
Artifact repositories	any	jfrog , nexus , github-packages
Project site	any	github-pages
Container image	one or none	docker-registry , github-container
Image security	any	trivy , syft , cosign
Deployment	one or none	docker-host , kubernetes
GitOps	one or none	argocd (adds kubernetes)
Database	one or none	postgresql
Secrets	any	vault , sealed-secrets (adds kubernetes)
Monitoring	any	prometheus (with Grafana), loki (adds prometheus)
Load test	one or none	k6

Artifactory OSS does not allow creating repositories through its API. After `configure` , open Artifactory, choose Quick Setup > Maven and enter the repository prefix you gave in `secrets` (`JFROG_ARTIFACTORY_REPOSITORY_PREFIX`), so `<prefix>-libs-release-local` and `<prefix>-libs-snapshot-local` exist. `configure` creates them itself on the Pro editions.

Default stages with every module selected (plugin coordinates shortened):

ORDER	PHASE	STAGE	MAVEN ARGUMENTS
10	ci	validate	validate
20	ci	build	clean package -DskipTests=true
30	ci	test	test
35	ci	migrate	flyway-maven-plugin:12.4.0:clean ...:migrate ...:validate - Dflyway.url=\$DATABASE_CI_URL ...
40	ci	checkstyle	maven-checkstyle-plugin:3.6.0:check - Dcheckstyle.config.location=google_checks.xml
45	ci	sonar	sonar-maven-plugin:5.8.0.7211:sonar -Dsonar.host.url=\$SONAR_URL - Dsonar.token=\$SONAR_TOKEN
50	ci	install	install -DskipTests=true
60	cd	site	maven-site-plugin:3.22.0:site
62	cd	stage-site	(shell) copy the root and module sites into target/staging
65	cd	publish-site	-N maven-scm-publish-plugin:3.3.0:publish-scm - Dscmpublish.pubScmUrl=... -Dscmpublish.scmBranch=site
70	cd	deploy-jfrog	package source:jar-no-fork javadoc:jar maven-deploy- plugin:3.2.0:deploy -DaltSnapshotDeploymentRepository=jfrog- snapshots::\$JFROG_ARTIFACTORY_SNAPSHOT_URL ...
71	cd	deploy-github	... -DaltSnapshotDeploymentRepository=github::\$GITHUB_PACKAGES_URL ...
72	cd	deploy-nexus	... -DaltSnapshotDeploymentRepository=nexus- snapshots::\$NEXUS_ARTIFACTORY_SNAPSHOT_URL ...
75	cd	image	package -DskipTests=true jib-maven-plugin:3.5.2:build - Djib.to.image=\$IMAGE_REPOSITORY:(git rev-parse --short=12 HEAD) ...
76	cd	sbom	(shell) sh "\$DEVOPS_SCRIPTS/sbom.sh"
77	cd	scan-image	(shell) sh "\$DEVOPS_SCRIPTS/scan-image.sh"
78	cd	sign-image	(shell) sh "\$DEVOPS_SCRIPTS/sign-image.sh"
80	staging	deploy-staging	(shell) sh "\$DEVOPS_SCRIPTS/deploy-compose.sh" staging
85	staging	load-test	(shell) sh "\$DEVOPS_SCRIPTS/load-test.sh" staging
-	production	approval	waits until someone approves production
79	production	verify-image-production	(shell) sh "\$DEVOPS_SCRIPTS/verify-image.sh"
80	production	deploy-production	(shell) sh "\$DEVOPS_SCRIPTS/deploy-compose.sh"
		production	

The project's pom.xml needs no profiles, no distributionManagement and no settings file. Every plugin is called by its coordinates and configured with `-D` properties, and credentials come from the framework's own [settings.xml](#), passed as global settings (`-gs`). Details and optional knobs (extra profiles, checkstyle rules, plugin versions) are in [docs/project-requirements.md](#).

Container images and deployments are described in [docs/deployment.md](#).

Adding a tool is one directory; see [docs/module-guide.md](#).

Orchestrators

maven runs each stage with `mvn` on your machine, with the pipeline variables exported only for that process. `render` also writes `.devops/generated/pipeline.sh` for IDE run configurations.

maven-container runs the same stages with `mvn` in a container from the official image `maven:<MAVEN_VERSION>-eclipse-temurin-<JAVA_VERSION>` (`MAVEN_IMAGE`), started by `run` on the Docker machine: this one, or the one `DOCKER_HOST` points to ([where-tools-run.md](#)). The container checks the branch (`GIT_BRANCH`) out of GitHub, so push your commits before `run` ; it needs neither Java, Maven nor the checkout of this machine. It joins the network of the tools and reaches them by their service names, as Jenkins and Concourse do, and a volume keeps the Maven repository and the downloaded tools between runs. `run` takes the same options as with maven; `run --phase <environment>` deploys the commit that passed the part of the pipeline before it, not the newest one. `destroy` removes the volume.

jenkins builds an image from `jenkins/jenkins:lts-jdk21` with Maven 3.9 (copied from the official Maven image) and the needed plugins, skips the setup wizard and configures everything with Configuration as Code: the admin user, one secret-text credential per secret and a pipeline job generated from the stages. `run` triggers the job and streams its console. There is no login to the UI and no API token to create by hand: `devops.sh` talks to its own Jenkins with the admin password over the REST API.

Builds also start on every push, chosen with `JENKINS_TRIGGER` in `secrets` : `poll` (default) checks the repository every two minutes, `webhook` registers a GitHub webhook to `JENKINS_PUBLIC_URL/github-webhook/` (Jenkins must be reachable from GitHub, see [docs/ngrok.md](#), and the token needs `admin:repo_hook`), `none` builds only on `run` . Either trigger starts working after the first build, which records the repository.

concourse runs `concourse quickstart` (web and worker in one container). The pipeline has a `ci` job, triggered by every push, a `cd` job that runs the ci and cd stages and deploys to the environments before the first approval, started by hand after `ci` passed, and a job per environment that needs approval (see below). Tasks run in `maven:3.9-eclipse-temurin-21` . `fly` is downloaded from the server.

Approvals

A deployment environment that needs approval (by default the last one, see [Environments](#)) runs its stages, and those of the environments after it up to the next approval, only after someone approves it, in the way each orchestrator offers:

Orchestrator	Waiting	Approve with
maven, maven-container	<code>run</code> stops before the environment	<code>devops.sh run --phase <environment></code>
jenkins	the build stops at the <code>approve-<environment></code> stage (an <code>input</code> step, kept for 7 days)	<code>devops.sh run --phase <environment></code> , or <code>Deploy</code> in the build
concourse	the environment's job, after the job before it passed	<code>devops.sh run --phase <environment></code> , or the job's <code>+</code> button

`prod` stands for the last environment. An approved environment gets the commit that passed the environments before it; it is not built again.

Stage scripts (`templates/scripts`) and the Helm chart (`templates/helm`) reach maven-container, Jenkins and Concourse as a compressed copy in the generated pipeline, unpacked into `.devops/` of the checkout before the first stage.

All orchestrators use the same Java and Maven, set with `JAVA_VERSION` (21) and `MAVEN_VERSION` (3.9) in `secrets` . Java 21 also builds projects whose pom targets an older release such as 17.

Docker images

Every tool runs from its official image, unchanged:

Tool	Image
SonarQube	<code>sonarqube:26.9.0.129388-community</code> (with <code>postgres:18</code>)
Nexus	<code>sonatype/nexus3</code>
Artifactory OSS	<code>releases-docker.jfrog.io/jfrog/artifactory-oss</code> (with <code>postgres:18</code>)
Concourse	<code>concourse/concourse</code> (with <code>postgres:18</code>)
Concourse build tasks, maven-container	<code>maven:<MAVEN_VERSION>-eclipse-temurin-<JAVA_VERSION></code>
Docker registry	<code>registry:3</code>
Simulated deploy machine	built from <code>docker:cli</code> with <code>openssh-server</code> and <code>curl</code>
Kubernetes	<code>rancher/k3s:v1.37.1-k3s1</code> (<code>K3S_IMAGE_TAG</code>)
Argo CD, Argo Rollouts	the projects' release manifests (3.5.4, 1.10.0), applied to the cluster

Jenkins is the one exception. The official `jenkins/jenkins` image has Java but no Maven, and the pipeline stages are `mvn` commands run inside Jenkins. So

`modules/orchestrator/jenkins/docker/Dockerfile` adds a thin layer built only from official images: it starts from `jenkins/jenkins:lts-jdk<JAVA_VERSION>` , copies Maven from `maven:<MAVEN_VERSION>-eclipse-temurin-<JAVA_VERSION>` , and adds `git`, `ssh`, `curl`, `jq` and the plugins in `plugins.txt` . `up` builds it locally; no image is published by `mvn-devops`. Java and Maven versions come from `JAVA_VERSION` (default 21) and `MAVEN_VERSION` (default 3.9), asked by the `maven` build module, so Jenkins, Concourse and your machine use the same versions.

Only stable releases are used. SonarQube Community Build has no long-term release and a new version each month, so it is pinned (`SONAR_IMAGE_TAG` overrides it); the other tools follow their stable `latest` or `lts` tag. SonarQube refuses to start when the disk holding Docker is more than 90% full.

Where the tools run

No tool has to run on `localhost`. Every tool with a server can be one of:

	How	Asked in <code>secrets</code>
started in Docker by <code>devops.sh</code>	on this machine, or on the machine <code>DOCKER_HOST</code> points to	<code>DEVOPS_HOST</code> : address of the Docker machine
an existing server	anywhere, with its own URL; nothing is started for it	<code><TOOL>_SERVER_URL</code> , plus the credentials

Each tool is decided on its own, so any mix works: SonarQube on `https://sonar.acme.com`, Nexus in Docker on a VM, Jenkins on `https://jenkins.acme.com`, and the repository on GitHub Enterprise.

Tool	Existing server	What <code>devops.sh</code> asks for it
GitHub	<code>GITHUB_URL</code> , e.g. <code>https://github.acme.com</code> (detected from <code>origin</code>)	token as for github.com
SonarQube	<code>SONAR_SERVER_URL</code>	an analysis token (My Account > Security)
Nexus	<code>NEXUS_SERVER_URL</code>	a user that may deploy, its password, repository names
Artifactory	<code>JFROG_SERVER_URL</code> , ending in <code>/artifactory</code>	a user, its password, API key or identity token, repository names
GitHub Packages	<code>GITHUB_PACKAGES_REGISTRY</code> (<code>https://maven.<host></code> on Enterprise)	
Jenkins	<code>JENKINS_SERVER_URL</code>	a user and its API token
Concourse	<code>CONCOURSE_SERVER_URL</code>	team, user and password

For an existing server, `configure` only checks the credentials and the repositories; it does not change passwords or create anything there. For Jenkins, `publish` creates or updates the job and its credentials through the REST API (credential ids get the prefix `<project>-`, because the server is shared). The agents need git, ssh, Java 21 and Maven 3.9, and the server the plugins workflow-aggregator, git, credentials-binding, plain-credentials and timestampers. For Concourse, `publish` downloads `fly` from the server and sets the pipeline in your team.

Leave a `*_SERVER_URL` empty to run that tool in Docker. Change your answers later with `devops.sh secrets --reconfigure`.

Tools in Docker on another machine

Clone `mvn-devops` and the project on the VM and run `devops.sh` there, or point Docker at the VM and run everything from your machine:

```
export DOCKER_HOST=ssh://user@build-vm      # or a docker context
./mvn-devops/devops.sh doctor              # docker daemon reachable on build-vm
./mvn-devops/devops.sh setup                # containers start on the VM
./mvn-devops/devops.sh run                  # mvn runs here and talks to build-vm:<port>
```

With the `maven-container` orchestrator `mvn` runs on the VM as well, in a container next to the tools, and your machine needs only Docker's client and `devops.sh` ([getting-started.md](#)).

`secrets` takes the default for `DEVOPS_HOST` from `DOCKER_HOST` or the docker context. The ports must be reachable from where the pipeline runs (firewall, security group). Jenkins in Docker mounts its configuration from `.devops/`, which a remote Docker daemon cannot see, so run `devops.sh` on the VM for it.

When an existing Jenkins or Concourse server runs the pipeline and some tools run in Docker, set `DEVOPS_HOST` to an address that server can reach; `publish` warns when it is `localhost`.

The compose file of your tools

`up` also writes the selected tools as one plain compose project, so you can read it, keep it and run it without `devops.sh`:

```
../mvn-devops/devops.sh export-compose          # .devops/compose/docker-compose.yml + .env
../mvn-devops/devops.sh export-compose ./docker # or into a folder of the project
cd docker && docker compose up -d
```

`docker-compose.yml` merges the compose fragments of the selected modules; paths are written out and every other `${VAR}` (ports, passwords) is read from the `.env` next to it. `.env` holds the passwords, so it is `chmod 600` and, outside `.devops`, added to a `.gitignore` in that folder. The compose project name is `devops-<project>`, the same one `devops.sh` uses, so both manage the same containers. Tools on an existing server are not in the file.

Container images and deployment

These modules turn the build into something that runs: an image in a registry, and later a running application. They are capabilities for your project; select the ones you want in `init`.

The [hello-api example](#) is a small web application with a health endpoint (`/actuator/health`) that the end-to-end tests deploy; [hello-data](#) adds a PostgreSQL database with Flyway migrations.

Environments

The image is built once and the same image goes through the deployment environments one after the other. `ENVIRONMENTS` in `devops.conf` lists them in that order; a project that does not set it has `staging production`. The names commonly used for four environments are:

```
ENVIRONMENTS=dev test staging prod
```

An environment either follows the one before it on its own, or waits until someone approves it (`ENV_<NAME>_APPROVAL=yes` ; by default only the last one waits). `secrets` asks for each. The environments after an approved one follow it up to the next one that needs approval, so the pipeline falls into parts:

ENVIRONMENTS	Approval	What runs
staging production	production	<code>run : ci, cd, staging. run --phase production : production</code>
dev test staging prod	prod	<code>run : ci, cd, dev, test, staging. run --phase prod : prod</code>
dev test staging prod	staging, prod	<code>run : ci, cd, dev, test. run --phase staging : staging. run --phase prod :</code> prod

`prod` also stands for the last environment, whatever it is called, in `run --phase prod` and `rollback prod`. `devops.sh stages` shows each environment's stages and where an approval comes. How each orchestrator asks for approval is in [orchestrators.md](#).

Everything that belongs to an environment is per environment: its machine or namespace, its port, its secrets in Vault, its database and password, its metrics and logs. Values follow the pattern `<PREFIX>_<NAME>_<SETTING>`, for example `DEPLOY_DEV_PORT` or `DATABASE_PROD_PASSWORD`. Default ports count up from the last environment: with `dev test staging prod` the simulated machine serves prod on 8180, staging on 8181, test on 8182 and dev on 8183 (k3s: 8280 and up). Names are lowercase letters and digits, starting with a letter.

Container image

Choose one module of the *Container image* category:

Module	Registry
docker-registry	The Distribution registry in Docker (port 5000), or an existing registry given as <code>REGISTRY_SERVER_URL : https://docker.io</code> , Harbor, a Docker repository of Nexus or Artifactory

Module	Registry
github-container	The GitHub Container Registry (<code>ghcr.io/<owner>/<image></code>), with the GitHub user and token. The token needs <code>write:packages</code> (github-setup.md)

The `image` stage (cd phase, order 75) builds the application and pushes it with two tags:

- the commit, `git rev-parse --short=12 HEAD`. It never changes, and it is what deployments use, so a rollback is just the previous tag;
- `latest`, for people.

Question	Default	Meaning
<code>IMAGE_NAME</code>	the project name	repository name in the registry; add the namespace where the registry needs one (<code>acme/app</code> on Docker Hub)
<code>IMAGE_MODULE</code>	empty	Maven module that is the application, in a multi-module project
<code>IMAGE_BUILDER</code>	<code>jib</code>	<code>jib</code> , or <code>dockerfile</code> to build the project's own Dockerfile
<code>IMAGE_BASE</code>	<code>eclipse-temurin: <java>-jre</code>	base image (jib)
<code>IMAGE_PORT</code>	<code>8080</code>	port the application listens on (jib)

jib (the default) builds the image from the Maven build without Docker and without a Dockerfile, so it works in every orchestrator. **A Dockerfile** is used by default when the project has one in its root and the pipeline runs on this machine (the `maven` orchestrator); Jenkins and Concourse run without a Docker daemon and need jib.

The pipeline gets these variables:

Variable	Meaning
<code>IMAGE_REPOSITORY</code>	where the pipeline pushes, as it reaches the registry
<code>IMAGE_DEPLOY_REPOSITORY</code>	the same repository as the machines that run the image pull it
<code>IMAGE_REGISTRY_USERNAME</code> , <code>IMAGE_REGISTRY_PASSWORD</code>	credentials, empty for the local registry

The local registry has no TLS and no login. Docker accepts it without configuration only as `localhost:<port>`, which is why the deploy repository uses `localhost`. To pull from it on another machine, add it to `insecure-registries` in that machine's `/etc/docker/daemon.json`, or use a registry with TLS.

Check the result:

```
curl -s http://localhost:5000/v2/hello-api/tags/list
docker run --rm -p 8080:8080 localhost:5000/hello-api:latest
```

Image security

The *Image security* modules work on the pushed image, between the image stage and the deployment:

Module	Stage	What it does
syft	sbom (76, cd)	writes the software bill of materials of the image to <code>target/sbom.spdx.json</code> (SPDX) and <code>target/sbom.cdx.json</code> (CycloneDX)
trivy	scan-image (77, cd)	scans the operating system packages and the jars in the image, writes <code>target/trivy-report.json</code> and prints the findings of <code>TRIVY_SEVERITY</code> (HIGH,CRITICAL). Fails when there are vulnerabilities of <code>TRIVY_FAIL_ON</code> (CRITICAL) that have a fix; empty means report only
cosign	sign-image (78, cd)	signs the image by digest with the project's key and, with <code>syft</code> , attaches the SBOM as a signed attestation
cosign	verify-image-<environment> (79)	with a deployment module, in each environment that needs approval: it only gets an image signed with that key

The tools are not installed by hand: `templates/scripts/tool.sh` downloads the pinned release where the pipeline runs (your machine, the Jenkins container or the Concourse task), checks it against the release's checksum file and caches it in `~/.cache/mvn-devops/tools` (`DEVOPS_TOOLS`). Trivy's vulnerability database is cached there too. `TRIVY_VERSION`, `SYFT_VERSION` and `COSIGN_VERSION` override a version.

The Cosign key is generated by `setup` (`.devops/keys/cosign.key` and `cosign.pub`, protected by `COSIGN_PASSWORD`), or taken from `COSIGN_KEY_FILE`. The pipeline gets it as a secret. Keep a copy of the key: images signed with it can only be verified with its public key. Signatures go to the image's registry (OCI referrers); nothing is sent to the public Sigstore transparency log. Check a signature yourself with:

```
cosign verify --key .devops/keys/cosign.pub --insecure-ignore-tlog=true --allow-http-registry localhost:5000/hello-api:latest
```

Deploying to a machine

The `docker-host` module (*Deployment* category) runs the image with Docker Compose on a machine it reaches over SSH:

Stage	Phase	What happens
deploy-<environment> (80)	the environment	deploys the image of the commit to the environment's machine, after its approval when it needs one

Each deployment:

1. writes `compose.yml` and the tag to `~/mvn-devops/<image name>-<environment>/` on the machine and runs `docker compose up`;
2. calls the health endpoint (`DEPLOY_HEALTH_PATH`, default `/actuator/health`) for up to two minutes;
3. when it does not answer, puts back the image that ran before and fails the stage.

The container gets `DEPLOY_ENVIRONMENT` (the environment's name), the environment's secrets from Vault (see [Secrets](#)), written to `secrets.yml` in that directory, and the settings in `app.env` there, which you keep on the machine; it is never overwritten.

The machines

Question	Default	Meaning
<code>DEPLOY_SERVER_URL</code>	empty	machine of the first environment, e.g. <code>ssh://deploy@staging.example.com:22</code> ; empty: a simulated machine in Docker for all of them
<code>DEPLOY_<NAME>_SERVER_URL</code>	the first machine	machine of each other environment, e.g. <code>DEPLOY_PRODUCTION_SERVER_URL</code>
<code>DEPLOY_SSH_KEY_FILE</code>	empty	private key that may log in; empty: <code>setup</code> generates <code>.devops/keys/deploy</code>
<code>DEPLOY_<NAME>_PORT</code>	8180 for the last, counting up	port of the application on the machine, e.g. <code>DEPLOY_STAGING_PORT</code> 8181
<code>DEPLOY_HEALTH_PATH</code>	<code>/actuator/health</code>	health check

A real machine needs Docker with the compose plugin, curl or wget, and a user in the `docker` group whose `~/.ssh/authorized_keys` holds the public key (`devops.sh get DEPLOY_SSH_PUBLIC_KEY`). `configure` records the machines' SSH host keys, which the pipeline then checks, and tells you when the key cannot log in yet. The machine pulls `IMAGE_DEPLOY_REPOSITORY`, logging in with the registry user when there is one.

The simulated machine is a container with an SSH server and the Docker CLI, using the Docker daemon of your machine. The applications it starts are ordinary containers on your machine:

```
curl http://localhost:8181/hello    # staging
curl http://localhost:8180/hello    # production
```

Rollback

```
devops.sh rollback                # the last environment: the image that ran before
devops.sh rollback staging
devops.sh rollback prod --to 3f2a9c1d4e5b
```

Rolling back twice returns to where you started. Every image stays in the registry under its commit tag, so `--to` can go further back.

Each deployment replaces the container, so the application is down for the few seconds it takes to start. Zero-downtime updates come with Kubernetes.

Deploying to Kubernetes

The `kubernetes` module (*Deployment* category) installs the application with Helm, one release per environment:

Stage	Phase	What happens
<code>deploy-<environment></code> (80)	the environment	<code>helm upgrade --install</code> in the namespace <code><app>-<environment></code> , after its approval when it needs one

Helm waits until the new pods pass their readiness probe (the health path). The update is rolling: a new pod must be ready before an old one stops, so the application stays up. When the new version does not become ready within `KUBERNETES_TIMEOUT` (5m), Helm rolls the release back and

the stage fails.

Question	Default	Meaning
KUBERNETES_SERVER_URL	empty	an existing cluster; empty: k3s in Docker
KUBERNETES_KUBECONFIG	~/.kube/config	kubeconfig of the existing cluster, passed to the pipeline as a secret
KUBERNETES_<NAME>_PORT	8280 for the last, counting up	k3s: the application on this machine, e.g. KUBERNETES_STAGING_PORT 8281
KUBERNETES_<NAME>_NODE_PORT	30080 for the last, counting up, with k3s	node port of the service; empty: a ClusterIP service for your ingress
KUBERNETES_CHART	empty	the project's own chart, e.g. deploy/chart ; empty: the generic chart
KUBERNETES_REPLICAS	2	Pods per environment
DEPLOY_HEALTH_PATH	/actuator/health	readiness and liveness probe

The generic chart (`templates/helm/app`) has a Deployment with readiness and liveness probes, resource requests, a rolling update strategy and a Service. The container gets `DEPLOY_ENVIRONMENT`, and every key of the Secret `<app>-env` in its namespace. With Vault (see [Secrets](#)) the release creates that Secret from the environment's secrets (`secretEnv`) and the pods wait for it; without, it is used when it exists:

```
kubectl --namespace hello-api-production create secret generic hello-api-env --from-literal=GREETING=hello
```

A private registry gets an image pull secret from the registry credentials. **A chart of your own** receives the same values: `image.repository`, `image.tag`, `registryAuth`, `environment`, `replicas`, `containerPort`, `healthPath`, `service.nodePort`, and `secretEnv` OR `sealedSecretEnv`.

k3s in Docker is a one-node cluster on your machine. It pulls from the local registry through its service name, so the image reference stays `localhost:5000/<image>`. Use it from your machine with:

```
export KUBECONFIG=.devops/k3s/kubeconfig-host.yaml
kubectl get pods --namespace hello-api-staging
curl http://localhost:8281/hello
```

`devops.sh rollback` goes back to the previous Helm revision (`helm rollback`); `--to TAG` deploys an earlier image again.

GitOps with Argo CD

With the `argocd` module (*GitOps* category, it adds `kubernetes`) the pipeline does not deploy to the cluster itself. It writes the desired state to git, and Argo CD, running in the cluster, makes the cluster match it:

Stage	Phase	What happens
deploy- <environment> (80)	the environment	commits <code>environments/<environment>/</code> to the GitOps branch and waits until Argo CD reports the application synced and healthy

Each environment directory holds the chart and `environment.yaml` with the image tag and the environment's values, so the branch's history is the history of every release, and a pull request against it is a release proposal. When a release does not become healthy within `GITOPS_TIMEOUT` (600 seconds), the script commits the previous release again and the stage fails. `devops.sh rollback` is such a commit too.

Question	Default	Meaning
<code>GITOPS_BRANCH</code>	<code>gitops</code>	branch of the project's repository with the desired state; created by the first release
<code>GITOPS_CANARY</code>	<code>yes</code>	the last environment as a canary with Argo Rollouts
<code>ARGOCD_HOST_PORT</code>	<code>8443</code>	k3s: Argo CD's web console on this machine (<code>admin</code> , <code>devops.sh get ARGOCD_ADMIN_PASSWORD</code>)

`configure` installs Argo CD and Argo Rollouts from their release manifests, adds the repository with the GitHub token, and creates one Application `<app>-<environment>` per environment, which syncs automatically, prunes what was removed from git and undoes changes made by hand.

The canary replaces the Deployment with an Argo Rollouts `Rollout` : the new version first gets a quarter, then half of the pods, with a pause after each step, and then all of them. Its pods must become ready within three minutes, or Argo Rollouts aborts and keeps the stable version. Change the steps with `canary.steps` in a chart of your own.

Images from a private registry need an image pull secret in the namespace; GitOps never writes credentials to git. Create it once, or keep it in git encrypted with Sealed Secrets.

Secrets

The `vault` module (*Secrets* category) keeps the application's secrets in HashiCorp Vault, one KV secret per environment:

```
secret/<app>/staging      e.g. GREETING=..., SPRING_DATASOURCE_PASSWORD=...
secret/<app>/production
```

The deploy stage reads the secret of its environment (`app-secrets.sh`) and the application gets each key as an environment variable, which Spring Boot maps to its properties. Where they go depends on the deployment:

Deployment	Where the secrets are
<code>docker-host</code>	<code>secrets.yml</code> next to the compose file on the machine, readable only by the deploy user
<code>kubernetes</code>	the Secret <code><app>-env</code> of the Helm release; new pods when it changes
<code>argocd with sealed-secrets</code>	a SealedSecret in <code>environments/<environment>/environment.yaml</code> of the GitOps branch, encrypted for the cluster
<code>argocd alone</code>	the Secret <code><app>-env</code> , put in the cluster by the pipeline, not in git

A deployment picks up changed secrets the next time it runs; a rollback with Helm (`rollback` without `--to`) brings back the secrets of that revision.

Question	Default	Meaning
<code>VAULT_SERVER_URL</code>	<code>empty</code>	an existing Vault; empty: Vault in Docker

Question	Default	Meaning
VAULT_TOKEN		existing Vault: a token that may read <code><mount>/data/<app>/*</code> ; it should be periodic or long-lived
VAULT_HOST_PORT	8200	Vault in Docker: its port on this machine
VAULT_KV_MOUNT	secret	mount path of the KV version 2 secrets engine

Vault in Docker keeps its data in a volume. `configure` initialises it once (one unseal key; the key and the root token stay in `.devops/values`), enables the KV engine, and gives the pipeline a token that may only read the application's secrets and is renewed on every run. Vault starts sealed after every restart: run `devops.sh configure` to unseal it. Write secrets in the web console (`devops.sh urls`) or with the Vault CLI in the container:

```
./devops.sh compose exec -e VAULT_ADDR=http://127.0.0.1:8200 \
-e VAULT_TOKEN="$(./devops.sh get VAULT_ROOT_TOKEN)" vault \
vault kv put secret/hello-api/staging GREETING=Hi
```

Sealed Secrets (`sealed-secrets`, adds `kubernetes`): `configure` installs the controller in `kube-system`. With Argo CD the pipeline encrypts each secret with `kubeseal` for its namespace and name (strict scope), so the release in git carries it, and only the controller in the cluster can decrypt it. Seal secrets of your own the same way:

```
kubectl create secret generic db --dry-run=client --output yaml --from-literal=password=... \
| kubeseal --controller-namespace kube-system --format yaml > sealed-db.yaml
```

Database

The `postgresql` module (*Database* category) gives the application a PostgreSQL database in each environment and checks its schema migrations before anything is deployed. The schema lives in Flyway migrations in the project (`src/main/resources/db/migration/V1_...sql`), and the application applies them when it starts, as Spring Boot does with `flyway-core` on the classpath.

Stage	Phase	What happens
migrate (35)	ci	empties the ci database and applies every migration from scratch with the Flyway Maven plugin, then validates them; a broken migration fails the build

Each deployment runs a PostgreSQL next to the application, its data kept across releases and rollbacks:

Deployment	The database
docker-host	a <code>db</code> container in the application's compose project on the machine, its data in a volume there
kubernetes , argocd	a StatefulSet <code><app>-db</code> with a persistent volume in the environment's namespace

The application gets `SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME` and `SPRING_DATASOURCE_PASSWORD` with the environment's secrets, so a Spring Boot application needs no settings of its own. With Vault, the same names in the environment's secret win: set them there to use a managed database instead. When a release does not become healthy because a

migration fails, the previous version runs on: PostgreSQL rolls the failed migration back. Write migrations the previous release can live with (add a column first, remove the old one a release later), because a rollback does not undo them.

Question	Default	Meaning
DATABASE_NAME	the image name, e.g. <code>hello_data</code>	database and user name in every environment
DATABASE_HOST_PORT	5433	the ci database on this machine
FLYWAY_VERSION	12.4.0	the Flyway version of the project, which Spring Boot manages (4.1: 12.4.0)
FLYWAY_LOCATIONS	filesystem:src/main/resources/db/migration	where the migrations are

The passwords of the ci database and of each environment's database are generated (`devops.sh get DATABASE_PRODUCTION_PASSWORD`). A database keeps the password it was created with: after changing one, change it in the database too (`ALTER USER`).

Monitoring

The *Monitoring* modules watch the deployed application in each environment. They run in Docker next to the other tools.

Prometheus and Grafana (`prometheus`): Prometheus scrapes the application's metrics in each environment every 15 seconds, labelled `application` and `environment` , and keeps them for 15 days. Grafana opens on the dashboard *Application*: whether each environment is up, requests per second, failed requests, response times (95th percentile and average), JVM heap, CPU and database connections. Prometheus raises two alerts, visible at `/alerts` : *ApplicationDown* when an environment does not answer for two minutes, and *HighErrorRate* when more than 5% of its requests fail.

The application needs Micrometer's Prometheus registry, as in the examples:

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
  <scope>runtime</scope>
</dependency>
```

The deployments expose its endpoint (`/actuator/prometheus`) and turn on the response time histograms through the application's environment (`MANAGEMENT_ENDPOINTS_WEB_EXPOSURE_INCLUDE`). Prometheus finds the environments from the deployment: the machines and ports of `docker-host` , or the ports of k3s. A cluster of your own is out of its reach; run Prometheus in the cluster there.

Question	Default	Meaning
PROMETHEUS_HOST_PORT	9090	Prometheus on this machine
GRAFANA_HOST_PORT	3000	Grafana on this machine (<code>admin</code> , <code>devops.sh get GRAFANA_ADMIN_PASSWORD</code>)
METRICS_PATH	<code>/actuator/prometheus</code>	where the application's metrics are

Loki (`loki` , adds `prometheus`): Loki keeps the application's logs and Grafana searches them, on the dashboard *Application logs* or in Explore with queries such as `{environment="production"} |= "ERROR"` . Grafana Alloy collects them with the labels `application` , `environment` , `container` (`app` , `db`) and `source` : from the containers of the simulated machine, which run on the same Docker, and from the pods of k3s. Machines and clusters of your own need an agent there, such as Alloy, that pushes to Loki's push URL (`devops.sh urls`).

Question	Default	Meaning
<code>LOKI_HOST_PORT</code>	3100	Loki on this machine

The configuration of Prometheus, Grafana and Alloy is written to `.devops/monitoring` by `devops.sh up` ; run `up` and `configure` again after changing the deployment.

Load test

The `k6` module (*Load test* category) load tests one environment (`LOAD_TEST_ENVIRONMENT` , by default the one before the last, e.g. staging) with `k6` right after each deployment there:

Stage	Phase	What happens
<code>load-test</code> (85)	that environment	<code>LOAD_TEST_VUS</code> virtual users call it for <code>LOAD_TEST_DURATION</code> ; the stage fails when a threshold fails

Without a script of its own, the project gets `load-test.js`: each virtual user calls every path of `LOAD_TEST_PATHS` and pauses a second. It fails when more than `LOAD_TEST_MAX_ERROR_RATE` of the requests fail or the 95th percentile of the response times is above `LOAD_TEST_P95_MS` . A failed load test stops the release there: the environments after it keep the version they have.

Question	Default	Meaning
<code>LOAD_TEST_ENVIRONMENT</code>	the one before the last	environment the load test calls
<code>LOAD_TEST_URL</code>	found from the deployment	that environment as the pipeline reaches it; asked for a cluster of its own
<code>LOAD_TEST_SCRIPT</code>	empty	the project's k6 script, e.g. <code>src/test/k6/load.js</code> ; it gets <code>BASE_URL</code> and the values below as <code>__ENV</code>
<code>LOAD_TEST_PATHS</code>	<code>/actuator/health</code>	paths the generic script calls, comma separated, e.g. <code>/hello,/actuator/health</code>
<code>LOAD_TEST_VUS</code>	10	virtual users
<code>LOAD_TEST_DURATION</code>	30s	how long
<code>LOAD_TEST_P95_MS</code>	500	highest 95th percentile response time in milliseconds
<code>LOAD_TEST_MAX_ERROR_RATE</code>	0.01	highest share of failed requests

The summary is written to `target/load-test.json` . With the `prometheus` module k6 also sends its metrics to Prometheus (`k6_http_reqs_total` , `k6_http_req_duration_p95` and others, labelled `environment`), so a load test shows next to the application's own metrics in Grafana. k6 is downloaded where the pipeline runs, like the other tools.

Project site on GitHub Pages

The `github-pages` module builds the Maven site and pushes it to a `site` branch, which GitHub Pages serves. Two things have to be done once by hand.

1. Create the `site` branch in your project repository. It is an orphan branch that only holds the published site:

```
git checkout --orphan site
git rm -rf --cached . > /dev/null
echo "site" > index.html
git add index.html
git commit -m "Initialize site"
git push origin site
git checkout -f main
```

`git checkout -f main` restores your working tree; untracked files such as `.devops/` are left alone. Use another branch name by answering the `SITE_BRANCH` question in `secrets`.

2. Point GitHub Pages at it. In the GitHub repository go to Settings > Pages, and under Build and deployment set:

- Source: Deploy from a branch
- Branch: `site`, folder `/` (root)

The site is then served at `https://<owner>.github.io/<repo>/`.

Publishing. The site stages run in the `cd` phase, after the ci stages:

```
./devops.sh run --phase cd          # site, stage-site, publish-site and the deploys
./devops.sh run --only publish-site # push an already built and staged site again
```

Maven by hand. The same steps as plain Maven commands, run in the project root. No profiles or settings are needed:

```
## build the site, including module sites
mvn -B org.apache.maven.plugins:maven-site-plugin:3.22.0:site

## preview it at http://localhost:8000
mvn org.apache.maven.plugins:maven-site-plugin:3.22.0:run -Dport=8000

## publish target/staging to the site branch (after ./devops.sh run --only stage-site)
mvn -B -N org.apache.maven.plugins:maven-scm-publish-plugin:3.3.0:publish-scm \
  -Dscmpublish.pubScmUrl=scm:git:git@github.com:<owner>/<repo>.git \
  -Dscmpublish.scmBranch=site
```

`./devops.sh stages` prints the exact command of every stage for your selection. Pushing uses SSH: your own key with the `maven` orchestrator, and a deploy key that `configure` registers on the repository with Jenkins or Concourse.

Releasing your project

`devops.sh release` releases the Maven project without `maven-release-plugin`, so the pom needs no `<scm>` and no `<distributionManagement>` :

1. sets the release version in every module (`1.2.0-SNAPSHOT` becomes `1.2.0`), commits "Release 1.2.0" and tags `v1.2.0`
2. runs the deploy stages of the selected artifact repositories
3. sets the next development version (`1.2.1-SNAPSHOT`) and commits it
4. pushes the branch and the tag

```
devops.sh release --dry-run          # show what would happen
devops.sh release                   # 1.2.0, then 1.2.1-SNAPSHOT
devops.sh release --version 2.0.0 --next 2.1.0-SNAPSHOT
devops.sh release --no-push         # check locally, push yourself
```

It runs on your machine with `mvn` and `git`, needs a clean working tree and pushes with your own `git` credentials or SSH key. Nothing is pushed before step 4: if a step fails, the release commit and the tag are rolled back. To undo a release that was already pushed, delete the tag (`git push origin :refs/tags/v1.2.0`) and revert the two commits.

Reaching tools on your machine from the internet (ngrok)

Only needed when GitHub has to call a tool that runs on your machine or a VM without a public address, which is the Jenkins push webhook (`JENKINS_TRIGGER=webhook`). Polling (`poll` , the default) and Concourse need nothing from outside.

1. Create an account at ngrok.com and install ngrok: - Windows: unzip it into, e.g., `C:\sdk\ngrok` and add that folder to `PATH` . - Linux: `sudo snap install ngrok` , or unpack the tgz into `/opt/ngrok` and add it to `PATH` . - macOS: `brew install ngrok` .
2. Add your auth token from the ngrok dashboard: `bash ngrok config add-authtoken <token>`
3. Start a tunnel to Jenkins (use your `JENKINS_HOST_PORT` , 8080 by default): `bash ngrok http 8080`
With a free account the URL changes on every start; a reserved domain (`ngrok http --url=<name>.ngrok-free.app 8080`) keeps it stable.
4. Give that URL to mvn-devops and register the webhook: `bash devops.sh secrets --reconfigure`
`# JENKINS_TRIGGER=webhook, JENKINS_PUBLIC_URL=https://<name>.ngrok-free.app devops.sh publish #`
registers `<url>/github-webhook/` on the repository The GitHub token needs `admin:repo_hook` for this ([github-setup.md](#)).

Several tools at once can be described in the ngrok config file (`ngrok config edit`) and started with `ngrok start --all` :

```
tunnels:
  jenkins: { addr: 8080, proto: http }
  sonarqube: { addr: 9000, proto: http }
  nexus: { addr: 8084, proto: http }
```

When the URL changes, run `secrets --reconfigure` and `publish` again; the old webhook can be removed under Settings > Webhooks.

IDE settings (IntelliJ IDEA)

Checkstyle

The `checkstyle` stage checks the code with `MAVEN_CHECKSTYLE_CONFIG` (`google_checks.xml` unless you chose your own file). To see the same warnings while you type:

1. Install the [CheckStyle-IDEA](#) plugin.
2. Settings > Tools > Checkstyle: add a configuration file. Use the project's own file (e.g. `code-style/checkstyle.xml`) or the bundled Google checks, whichever the pipeline uses, and make it active.
3. To format code the same way: Settings > Editor > Code Style > Java > (gear icon) > Import Scheme > CheckStyle Configuration, and pick the same file.

Test coverage

If IntelliJ's own coverage runner reports wrong numbers or slows tests down, open Help > Find Action > `Registry...` and turn off:

- `idea.coverage.new.sampling.enable`
- `idea.coverage.test.tracking.enable`
- `idea.coverage.tracing.enable`

Pipeline variables in the IDE

Run configurations started from the IDE do not see the pipeline variables. `devops.sh render` writes `.devops/generated/pipeline.sh` (run it as a shell run configuration), and on Windows `devops.sh env --windows` writes `.devops/generated/set-env.bat` , which sets them with `setx` for the IDE.

Troubleshooting

Start with `devops.sh doctor` : it checks the tools, Docker, the Java and Maven versions, your SSH key on GitHub and the line endings of the framework files. `devops.sh logs <service>` shows a tool's own log, and `devops.sh status` its containers.

Running devops.sh

`mvn-devops` needs **Bash 4 or newer** on macOS, whose own Bash is 3.2. Install a newer one with `brew install bash` and open a new terminal; `#!/usr/bin/env bash` then finds it.

`$'\r': command not found, bad interpreter OR invalid option name` : the scripts got Windows (CRLF) line endings, usually from git on Windows with `core.autocrlf=true`. `doctor` lists the files and prints the command that fixes them. Keep `mvn-devops/.gitattributes` committed so it does not happen again; [installation.md](#) shows how to repair a checkout.

Cannot connect to the Docker daemon : start Docker Desktop, or the docker service on Linux. With `DOCKER_HOST` set, check that the other machine is reachable (`docker info`).

A question is not asked any more, or a teammate's answer is used: answers in the committed `devops.conf` win over your own. Run `devops.sh secrets --reconfigure` to answer again (the new answer is written back to `devops.conf` for everyone), or edit `devops.conf` and commit it.

Starting the tools

port is already allocated during `up` : another program (or another project's tool) uses the port. Change it, e.g. `NEXUS_HOST_PORT`, with `devops.sh secrets --reconfigure` or in `devops.conf`, then run `up` again.

SonarQube never comes up and its log shows `high disk watermark` or

`no_shard_available_action_exception` : SonarQube's search engine does not start when the disk holding Docker is more than 90% full. Free space, for example with `docker system prune`, then `devops.sh down` and `up`.

A tool keeps restarting after its version changed: the tools' data is kept in Docker volumes, and a newer version may not read an older one's data. `devops.sh destroy` removes the containers and volumes, then `up` and `configure` start them fresh.

Jenkins in Docker on another machine does not see its configuration: it mounts files from `.devops/`, which only exist on the machine running `devops.sh`. Run `devops.sh` on the Docker machine for Jenkins.

Running the pipeline

Deploy to Nexus fails with 400 or 403 although the password is right: Nexus Community Edition accepts uploads only after its EULA is accepted. Set `NEXUS_ACCEPT_EULA` to `yes` with `secrets --reconfigure` and run `configure`.

Deploy to Artifactory fails with 404: Artifactory OSS does not let devops.sh create repositories. Open Artifactory, choose Quick Setup > Maven and enter the prefix from `JFROG_ARTIFACTORY_REPOSITORY_PREFIX` ([modules.md](#)).

Checkstyle fails: the default rules are `google_checks.xml`. Point `MAVEN_CHECKSTYLE_CONFIG` at your own file, or set `MAVEN_CHECKSTYLE` to `no` ([project-requirements.md](#)).

Jenkins does not build on push: with `poll` or `webhook`, the trigger starts working after the first build, which records the repository. Run `devops.sh run` once. A webhook also needs Jenkins reachable from GitHub ([ngrok.md](#)).

publish-site is refused: the `site` branch must exist and your SSH key (with the `maven` orchestrator) or the deploy key must be allowed to push ([github-pages.md](#)).

Starting over

```
devops.sh destroy    # containers, volumes, deploy key and webhook; asks before deleting .devops/
devops.sh setup      # everything again; devops.conf keeps the team's answers
```

Design

The idea

Each tool is a self-contained module with its own scripts. `devops.sh` asks which tool you want from each category, then runs only the scripts of those tools, in order, until everything is running, configured and wired into a pipeline that is ready to run.

Categories and modules

Category	Choice	Modules
Source control	required	github
Build	required	maven (validate, package, test, checkstyle, install)
Pipeline orchestrator	one	maven (on your machine), maven-container, jenkins, concourse
Code quality	any	sonarqube
Artifact repositories	any	jfrog, nexus, github-packages
Project site	any	github-pages

A new tool only needs a new directory under `modules/<category>/<tool>/` ; the menu finds it on its own ([module-guide.md](#)).

A module

```
modules/artifact/nexus/  
  module.conf    name, description, modules it depends on  
  compose.yml    the tool's containers  
  module.sh      hooks
```

Every hook is one step of the life cycle, and every hook is optional:

Hook	Job
<code>module_secrets</code>	ask for the values the tool needs (port, passwords, ...)
<code>module_prepare</code>	prepare files before the containers start
<code>module_configure</code>	finish setup once the tool is up: admin password, tokens, repositories
<code>module_env</code>	declare the variables the pipeline needs
<code>module_stages</code>	add Maven stages to the pipeline
<code>module_destroy</code>	undo what <code>configure</code> did outside Docker
<code>module_render</code> , <code>module_publish</code> , <code>module_run</code>	orchestrators only: build, install and run the pipeline

Each hook runs in its own subshell, so modules cannot interfere with each other.

Stages come from the modules

Every module declares its stages with an order number and a phase (`ci` , `cd` , or a deployment environment such as `staging`). For example sonarqube says "stage 45 in phase ci: `sonar-maven-plugin:sonar` ". The orchestrator takes the sorted list and turns it into its own format:

- **maven** runs every stage with `mvn` on your machine.
- **maven-container** writes a script that checks the commit out of GitHub and runs the stages with `mvn` in a container on the Docker machine.
- **jenkins** writes a Jenkinsfile and a `casc.yaml` that defines the admin user, the credentials and the job. No setup wizard and nothing to click.
- **concourse** writes a `pipeline.yml` with a `ci` job that runs on every push, a `cd` job you start by hand after `ci` passed, and a job per environment that needs approval.

So a new tool shows up in every orchestrator at once.

The project's pom.xml needs no profiles

`profiles.xml` only existed in Maven 2. Profiles in `settings.xml` only accept properties, repositories and activation, not plugins or `distributionManagement` .

So stages call every plugin by its full coordinates and configure it with `-D` , for example `maven-deploy-plugin:3.2.0:deploy -DaltSnapshotDeploymentRepository=nexus-snapshots::<url>` . Credentials come from the framework's own `templates/settings.xml` , passed with `-gs` (global settings). A project adds no profile, `distributionManagement` or settings to its pom. A project that has its own profiles or settings file can still use them through `MAVEN_PROFILES` and `MAVEN_SETTINGS` .

How it fits together

```
devops.sh — lib/ (menu, value store, env files, stages, compose)
├── modules/<category>/<tool>/
│   ├── module.conf    title, description, requirements
│   ├── module.sh      hooks: module_secrets, module_prepare, module_configure,
│                       module_env, module_stages, module_urls
│                       (+ module_render, module_publish, module_run for orchestrators)
│   └── compose.yml     containers of the tool (optional)
```

The selected tools and every answer that is not a secret or personal are kept in

`<project>/devops.conf` , which you commit. Everything else is kept in `<project>/devops/` (it git-ignores itself):

```
.devops/
├── values/<KEY>        one file per value, chmod 600
├── env/pipeline.env    variables handed to the pipeline (compose env_file format)
├── env/pipeline.sh     the same as bash exports
├── env/compose.env     everything, for ${VAR} substitution in compose files
├── generated/         Jenkinsfile, jenkins/casc.yaml, concourse/*.yaml, pipeline.sh, maven-
├── container/run.sh
└── keys/              generated SSH deploy key
```

Nothing is written to `~/.bashrc` or to system environment variables, and secrets are never printed: `env --show` masks them and the orchestrators bind them as masked credentials.

URLs handed to the pipeline depend on where it runs. A tool on an existing server is always reached at its own URL. A tool in Docker is reached at `DEVOPS_HOST:<port>` by the `maven` orchestrator and by an existing Jenkins or Concourse server, and at its compose service name (`http://sonarqube:9000`) by maven-container, and Jenkins or Concourse in Docker, which share its network. See [Where the tools run](#).

Where the tools run

Every tool with a server (SonarQube, Nexus, Artifactory, Jenkins, Concourse) has two modes:

- **In Docker:** `devops.sh` starts it, on this machine or on the machine `DOCKER_HOST` points to. It is reached through `DEVOPS_HOST`.
- **Existing server:** anywhere, with its own URL. `secrets` asks for `<TOOL>_SERVER_URL` and credentials, no container is created, and `configure` only checks the credentials and repositories.

Each tool is decided on its own, so any mix works. Details in [where-tools-run.md](#).

Life cycle

```
init → secrets → up → configure → publish → run
menu  questions Docker tokens install pipeline run it
```

`devops.sh setup` runs the first five steps in a row.

Writing a module

A module is a directory under `modules/<category>/<name>/`. The menu discovers it automatically; no other file has to change.

Files

`module.conf` (required)

```
MODULE_TITLE="Nexus Repository"
MODULE_DESCRIPTION="Sonatype Nexus 3 for Maven releases and snapshots" # shown in the menu
MODULE_REQUIRES="scm/github"      # other modules that must be selected too (optional)
MODULE_RUNS_IN=docker              # orchestrators only: host or docker
MODULE_SERVER=NEXUS                # tools with a server: prefix of <PREFIX>_SERVER_URL (optional)
```

With `MODULE_SERVER`, the tool can also be an existing server: when `<PREFIX>_SERVER_URL` has a value, the module's `compose.yml` is left out and the hooks talk to that URL instead.

`compose.yml` (optional)

A docker compose fragment. The fragments of all selected modules are merged into one compose project (`devops-<project>`), so services reach each other by service name. Use named volumes for data. Every stored value is available for `${VAR}` substitution, plus `DEVOPS_HOME`, `DEVOPS_STATE`, `PROJECT_DIR` and `PROJECT_NAME`. Use them for paths, since relative paths are resolved against `.devops/`.

`module.sh` (optional)

Hook functions. Each hook runs in its own subshell with the library loaded, `MODULE_ID` and `MODULE_DIR` set and the module's `module.conf` sourced.

Hook	Called by	Purpose
<code>module_secrets</code>	<code>secrets</code>	ask for values with <code>ask</code> / <code>ask_secret</code>
<code>module_prepare</code>	<code>up</code> , before containers start	render config files, create keys
<code>module_configure</code>	<code>configure</code> , after containers start	admin passwords, tokens, repositories
<code>module_env</code>	every command	export pipeline variables with <code>pipeline_var</code> / <code>pipeline_secret</code>
<code>module_stages</code>	<code>stages</code> , <code>render</code> , <code>run</code>	contribute stages with <code>stage</code>
<code>module_urls</code>	<code>urls</code> , end of <code>setup</code>	print the web console and how to log in
<code>module_rollback</code>	<code>rollback</code>	put the previous release back (deployment modules)
<code>module_destroy</code>	<code>destroy</code> , before containers are removed	undo what <code>configure</code> did outside Docker (deploy keys, webhooks)
<code>module_render</code>	orchestrators: <code>render</code> , <code>publish</code>	write the pipeline definition
<code>module_publish</code>	orchestrators: <code>publish</code>	install the pipeline
<code>module_run</code>	orchestrators: <code>run</code>	run it

For orchestrators, `module_configure` runs during `publish` (after the tools are configured) instead of during `configure`.

Helpers

```
ask KEY "Question" [default]           # stored in .devops/values/KEY and shared through
devops.conf
ask_local KEY "Question" [default]      # same, but never shared: user names, addresses of this
machine
ask_secret KEY "Question" [default]     # hidden input, masked everywhere
value KEY [fallback]                   # read a value
require_value KEY                       # read or fail
set_value KEY VALUE [secret]            # store a computed value, e.g. a token
random_password                         # a random default password

pipeline_var KEY VALUE                  # hand a variable to the pipeline
pipeline_secret KEY VALUE               # same, masked

stage ORDER PHASE NAME "MAVEN ARGS"    # PHASE is ci, cd or an environment ("prod": the last); args
may use $VARS
shell_stage ORDER PHASE NAME "CMD"      # a POSIX shell command in the project root instead of mvn
$ENVIRONMENTS                          # the deployment environments, in order, e.g. "staging
production"
upper NAME                             # STAGING: per-environment values are
<PREFIX>_<NAME>_<SETTING>
env_offset ENV                          # 0 for the last environment, 1 for the one before, ...
(default ports)
env_gates                              # the environments that wait for approval
rollback_args "$@"                     # parse [environment] [--to TAG] into ROLLBACK_ENV,
ROLLBACK_TAG
mvn_plugin KEY group:artifact VERSION GOAL # full plugin coordinates, version overridable per
project
mvn_deploy_args SNAP_ID SNAP_URL REL_ID REL_URL # package + attach + deploy without
distributionManagement
ask_server PREFIX "Title" [example]     # ask <PREFIX>_SERVER_URL; empty means Docker
server_external PREFIX                  # true when an existing server is used
server_url PREFIX HOST_PORT [PATH]      # URL for configure hooks: the server, or DEVOPS_HOST:port
server_pipeline_url PREFIX SERVICE PORT HOST_PORT [PATH] # URL for the pipeline
pipeline_url SERVICE PORT HOST_PORT [PATH] # service name or DEVOPS_HOST, depending on the
orchestrator
host_url HOST_PORT [PATH]               # DEVOPS_HOST URL
github_url / github_host / github_api # github.com or GitHub Enterprise

image_secrets                           # image modules: name, module, builder, base image, port
image_env PUSH_REPO DEPLOY_REPO [USER] [PASSWORD] # IMAGE_REPOSITORY and the rest for the
pipeline
image_stages INSECURE AUTH              # the image stage (Jib, or the project's Dockerfile)

compose ...                             # docker compose of the project, e.g. compose exec -T nexus
...
wait_http URL [timeout] [status regex]
log_step / log_ok / log_warn / log_dim / die / confirm
```

Call plugins by their coordinates and configure them with `-D` properties, so projects need no profiles (see [project-requirements.md](#)). A deploy target also needs a `<server>` with its id in `templates/settings.xml`.

Stage arguments are embedded in single-quoted strings by the orchestrators, so they may not contain `'`, `\`, `|` or `${}`. Use `$VAR` instead of `${VAR}`.

A stage that needs more than one command calls a script from `templates/scripts/`: `shell_stage`

```
80 cd deploy "sh \"\$DEVOPS_SCRIPTS/deploy.sh\"". DEVOPS_SCRIPTS is that directory when the
pipeline runs on this machine; maven-container, Jenkins and Concourse get a copy in
.devops/scripts before the first stage. Scripts get every pipeline variable in the environment.
```

Example: a new artifact repository

```
modules/artifact/reposilite/
  module.conf
  compose.yml
  module.sh
```

```
## module.sh
## module.conf has MODULE_SERVER=REPOSILITE
module_secrets() {
  ask_server REPOSILITE Reposilite https://repo.example.com
  server_external REPOSILITE || ask REPOSILITE_HOST_PORT "Reposilite port on the Docker machine"
8085
  ask_secret REPOSILITE_TOKEN "Reposilite deploy token" "${random_password}"
}

module_env() {
  pipeline_var REPOSILITE_URL "${server_pipeline_url REPOSILITE reposilite 8080 "${value
REPOSILITE_HOST_PORT 8085}"/snapshots"
  pipeline_secret REPOSILITE_TOKEN "${value REPOSILITE_TOKEN}"
}

module_stages() {
  stage 73 cd deploy-reposilite "${mvn_deploy_args reposilite '$REPOSILITE_URL' reposilite
'$REPOSILITE_URL')"}
}
```

A new category is a directory with a `category.conf` :

```
CATEGORY_TITLE="Security scanning"
CATEGORY_ORDER=45           # position in the menu
CATEGORY_MODE=multi         # required | single (exactly one) | optional (at most one) | multi
```

Run `tests/smoke.sh` and `shellcheck` after adding a module.

Development

Testing

```
shellcheck devops.sh lib/*.sh modules/**/*.sh tests/*.sh packaging/*.sh
tests/smoke.sh                # no Docker needed: init, secrets, stages, render,
devops.conf, upgrade
tests/e2e.sh maven sonarqube,nexus # real tools in Docker: setup, the whole pipeline, results in
the tools
tests/e2e.sh pipeline maven-docker-host # a ready-made pipeline
E2E_ENVIRONMENTS="dev test staging prod" E2E_APPROVALS="staging prod" \
  tests/e2e.sh maven docker-registry,docker-host # environments of your own
```

`tests/smoke.sh` runs on Linux, macOS and Windows (Git Bash) in the `ci` workflow. `tests/e2e.sh` builds `examples/hello-maven`, or `examples/hello-api` for the deployment modules, with each orchestrator in the `e2e` workflow, on every push to `main` and once a week; it serves the project from a local git server, so it needs no GitHub token. With `jfrog` it checks the setup only, because Artifactory OSS repositories have to be created in its Quick Setup wizard. Run it locally with Docker and internet access; `E2E_KEEP=1` leaves the containers running for a look.

Releasing mvn-devops

Every change worth telling users goes under `## Unreleased` in `CHANGELOG.md`. To release, rename that heading to the new version (`## 1.1.0`), set the same version in `VERSION`, commit, and push a tag, or start the release workflow by hand under Actions > release > Run workflow with the version. The workflow stops when `CHANGELOG.md` has no section for the version, and uses that section as the text of the GitHub release.

```
git tag v1.1.0 && git push origin v1.1.0
```

The release workflow runs the checks, builds the files with `packaging/build.sh` (zip, tar.gz, and deb and rpm with `nfpm`), installs the deb as a check and publishes everything as a GitHub release. `packaging/build.sh` also works locally and writes to `dist/`.

Changelog

Every release of mvn-devops. The section of a version is also the text of its GitHub release.

1.0.0

First release.

Setup

- A menu of modules per category: GitHub; Maven; SonarQube; Nexus, Artifactory OSS and GitHub Packages; GitHub Pages; container images; image security; deployment; GitOps; secrets; a database; monitoring; a load test.
- Orchestrators: `maven` (on this machine), `maven-container` (in a container on the Docker machine, from a fresh checkout of GitHub), Jenkins (configuration as code, no setup wizard) and Concourse.
- `setup` asks the questions, starts the tools in Docker, replaces their default passwords, creates tokens and repositories, and installs the pipeline. Every tool runs in Docker, on this machine or another one, or is an existing server with its own URL; GitHub Enterprise is supported.
- `devops.conf` in the project root holds the selected tools and every answer that is neither secret nor personal. Commit it: a teammate's `setup` asks only for their own tokens and user names. Passwords and keys stay in `.devops/`.
- 16 ready-made pipelines in `pipelines/`, from Maven with SonarQube and Nexus up to Jenkins with every kind of tool. `setup --pipeline <name or file>` sets one up and generates new passwords, tokens and keys for each project.
- No profiles, plugins or `distributionManagement` in the project's pom: plugins are called by their coordinates.

Pipeline

- Container images built with Jib (no Docker or Dockerfile needed) or with the project's Dockerfile, pushed to a registry in Docker, an existing one or the GitHub Container Registry.
- Image security: Trivy scans the image, Syft writes its SBOM, Cosign signs it and checks the signature before an approved environment gets it.
- Deployment environments of the project's own (`ENVIRONMENTS`, e.g. `dev test staging prod`), each with its own machine or namespace, port, secrets and database, and an approval where `ENV_<NAME>_APPROVAL=yes`. The same image goes through them in order.
- Deployment with Docker Compose over SSH, with Helm on Kubernetes (an existing cluster or k3s in Docker), or with Argo CD from a GitOps branch, the last environment as a canary with Argo Rollouts. Each deployment is checked and rolled back when it fails; `rollback` puts the previous image back.
- Secrets from Vault, passed to the application as environment variables, a Kubernetes Secret or a SealedSecret.
- PostgreSQL per environment, with the Flyway migrations checked against an empty database in ci.

- Prometheus, Grafana and Loki watch each environment; k6 load tests one of them after each deployment.
- `release` releases the project without maven-release-plugin.

Tools and packaging

- `doctor` checks the prerequisites; `doctor --fix` repairs CRLF line endings.
- `upgrade` replaces the copy of mvn-devops inside a project with a release, checked against SHA256SUMS .
- `export-compose` writes the selected tools as one docker-compose.yml.
- Packages: zip, tar.gz, deb and rpm. Runs on Linux, macOS and Windows (Git Bash).
- `docs/getting-started.md` has step-by-step guides; `examples/` has sample projects.
- Tested on every push: a smoke test on Linux, macOS and Windows, and an end-to-end test of every orchestrator and the ready-made pipelines with the real tools on GitHub's runners.